Model-Theoretic Dependency Grammar

Adam Przepiórkowski

Abstract

The aim of this paper is to provide a formalization of Dependency Grammars (DGs), one that may be useful to dependency linguists with different formalization needs and with different theoretical backgrounds. The formalization is set within the Model-Theoretic Syntax paradigm, as assumed in most current DGs. The proposed formalism is used in an analysis of a small fragment of English, corresponding roughly to that treated with a "feature-based dependency grammar formalism" in Gibson 2025, but extending it with more explicit analyses of coordination (including coordination of unlike categories) and long-distance dependencies. Numerous empirical and theoretical advantages of the current model-theoretic approach over the rule-based approach proposed there are pointed out.

Keywords: Model-Theoretic Syntax, Dependency Grammar, Head-driven Phrase Structure Grammar, linguistic theories, unlike category coordination

The aim of this paper is to provide a formalization of Dependency Grammars (DGs), one that may be useful to dependency linguists with different formalization needs and with different theoretical backgrounds.

We assume that the need for replacing the imprecise and ambiguous natural language with a precise and unambiguous formal language when expressing scientific statements does not require justification, but we support it with the usual quotation from *Syntactic Structures* (Chomsky 1957: 5):

> The search for rigorous formulation in linguistics has a much more serious motivation than mere concern for logical niceties or the desire to purify well-established methods of linguistic analysis. Precisely constructed models for linguistic structure can play an important role, both negative and positive, in the process of discovery itself. By pushing a precise but inadequate formulation to an unacceptable conclusion, we can often expose the exact source of this inadequacy and, consequently, gain a deeper understanding of the linguistic data. More positively, a formalized theory may automatically provide solutions for many problems other than those for which it was explicitly designed. Obscure and intuition-bound notions can neither lead to absurd conclusions nor provide new and correct ones, and hence they fail to be useful in two important respects. I think that some of those linguists who have questioned the value of precise and technical development of linguistic theory may have failed to recognize the productive potential in the method of rigorously stating a proposed theory and applying it strictly to linguistic material with no attempt to avoid unacceptable conclusions by *ad hoc* adjustments or loose formulation.

In order to justify the need for the particular model-theoretic formalization of DGs proposed in this paper, we start by describing the contemporary landscape of syntactic frameworks in §1, and then move to a brief characterization of extant attempts at formalizing DGs in §2. The basics of the proposed model-theoretic formalization of DGs are covered in §3, and a preliminary treatment of various linguistic phenomena is presented in §4. This presentation covers roughly the same empirical ground as the "feature-based dependency grammar formalism" of Gibson 2025, discussed in §5. Finally, the concluding §6 considers the issue of simplicity of DGs. Appendices provide the definition of the underlying logic (§A), a translation of that logic into the fully standard First Order Logic (§B), a formal description of an example of a model (§C), and additional axioms attempting to minimize resulting models (§D).

1 Introduction

Current syntactic frameworks may be classified along various dimensions. One is the kind of syntactic structures they assume: some frameworks are based on constituency, i.e., on parthood relations between linguistic expressions, others on dependency, i.e., on direct relations between words or other linguistic units. Well-known examples of constituency frameworks are the many versions of transformational grammars (TGs), from that of *Syntactic Structures* (Chomsky 1957) to the Minimalist Program (Chomsky 1995, 2000) and Minimalist Grammars (Stabler 1996, 2011), but also – with various caveats – different types of Categorial Grammars (CGs; Ajdukiewicz 1935, Lambek 1958, Steedman 1987, Jacobson 1999), Lexical Functional Grammar (LFG; Kaplan and Bresnan 1982, Bresnan et al. 2016, Dalrymple et al. 2019, Dalrymple 2023), and Head-driven Phrase Structure Grammar (HPSG; Pollard and Sag 1994, Müller et al. 2024), as well as its predecessor, Generalized Phrase Structure Grammar (GPSG; Gazdar et al. 1985). Examples of dependency frameworks are what we will call Surface Syntactic Dependency Grammar (SSDG), stemming from the foundational work of Tesnière 1959, 2015 (so the "SS" in "SSDG" may alternatively be understood as "Structural Syntax", after the title of Tesnière's magnum opus) and currently pursued by Timothy Osborne and colleagues (e.g., Osborne 2006, 2019, 2026, Groß and Osborne 2009), but also Word Grammar (WG; Hudson 1984, 1990, 2010), Meaning–Text Theory (MTT; Mel'čuk 1988, 2009), and Functional Generative Description (FGD; Sgall et al. 1986).[1]

Not all of these frameworks are "pure" in the sense of assuming only one kind of syntactic structure and completely excluding the other. For example, while the c-structure of LFG is a pure constituency structure, the other syntactic representation assumed in LFG in parallel, f-structure, contains certain dependency information. Conversely, three of the four dependency frameworks mentioned above assume constituents in their analyses of coordinate structures (SSDG, WG, and – to a lesser extent – MTT; see also Kahane 1997).

The second dimension is the level of formalization assumed in particular frameworks. Constituency frameworks are typically highly formalized, often to such an extent that grammars stated in these frameworks may be relatively directly implemented as parsers; this is especially true about LFG and HPSG, for which grammar implementation platforms have been developed[2] and large-scale implemented grammars of various languages are available,[3] but also for CGs and Minimalist Grammars, for which at least toy manual implementations exist.[4] This contrasts with dependency frameworks, where the two most general and perhaps best known approaches, namely SSDG and WG, are not formalized at all; there, linguistic principles are stated in prose only and there is no formal procedure for deciding whether a given dependency structure is described by the grammar. This difference between

[1] While there are important differences between the vintage work of Tesnière (1959, 2015) and the modern work of Osborne and colleagues, bundling these two approaches under the SSDG moniker is justified by the fact that they are both monostratal (unlike MTT and FGD) and both assume relatively simple graphs representing surface syntax, unlike the more complex "networks" within contemporary WG which also represent many other kinds of linguistic information. There is also a popular corpus annotation standard based on dependency structures, Universal Dependencies (de Marneffe et al. 2021), but it has been developed with engineering applications – rather than theoretical linguistic research – in mind, and it is rather controversial from the theoretical point of view (Przepiórkowski and Patejuk 2018, 2019, Osborne and Gerdes 2019), so we mostly ignore it here.

[2] For example, XLE for LFG, see Crouch et al. 2011, and LKB and TRALE for HPSG, see Copestake 2002 and Meurers et al. 2002, respectively.

[3] Such grammars have been developed especially within the LFG ParGram project (https://wiki.uni-konstanz.de/pargram/; Butt et al. 2002) and the HPSG DELPH-IN (https://github.com/delph-in/docs/wiki/; Bender et al. 2002) and CoreGram (https://hpsg.hu-berlin.de/Projects/CoreGram.html; Müller 2015) projects.

[4] See, for example, Stanojević and Stabler 2018 for an implementation of a toy Minimalist Grammar. Such implementations of manually constructed grammars should be sharply distinguished from parsers machine-learned from annotated corpora. Such large-scale parsers exist for CG (Lewis and Steedman 2014) and for Minimalist Grammars (Torr et al. 2019), as well as for DGs. In fact, due to the existence of a great number of dependency corpora within the Universal Dependencies initiative (see fn. 1), machine-learned dependency parsers are popular in Natural Language Processing (NLP) and have been implemented within various NLP libraries, including Stanza (Qi et al. 2020) and Trankit (Nguyen et al. 2021).

constituency frameworks and (most extant) dependency frameworks is acknowledged by dependency linguists, e.g., in the following quote from de Marneffe and Nivre 2019: 199:

> [A] clarification about the term grammar seems in order. In formal and generative linguistics, this term is often understood in the sense of a formal declarative system capable of generating languages and making predictions about grammaticality. Most frameworks in the dependency grammar tradition do not make use of grammars in this sense but are better described as syntactic analysis schemes that may be more or less formalized.

This difference is also noted by formal and computational grammarians, e.g.: "dependency syntax has remained something of an island from a formal point of view" (Kuhlmann 2013: 355).[5]

The third dimension is the "side of logic" that a given framework assumes: is the (potential) formalization based on – or analogous to – the proof theory, where certain objects (strings, formulae, etc.) are basic and others are derived from them via finite sequences of well-defined rewriting operations, or is it based on the model theory, where grammar is a collection of statements jointly defining certain structures (Johnson 1994, Pullum and Scholz 2001)? The first approach is sometimes called "Generative-Enumerative Syntax" (GES; Pullum and Scholz 2001), but we will use the term "Proof-Theoretic Syntax" (PTS; e.g., Moot 2023; cf. Partee et al. 1993: 179–191, §§8.1–8.3), by analogy to the usual name for the second approach, "Model-Theoretic Syntax" (e.g., Rogers and Kepser 2007 and various papers there); two vaguer terms sometimes used as synonymous with "proof-theoretic" and "model-theoretic" are, respectively, "rule-based" and "constraint-based".[6] Most of the constituency frameworks listed above – with the sole uncontroversial exception of HPSG and a more complicated status of LFG (Przepiórkowski 2023) – are natively proof-theoretic, although they may in principle be reformulated as model-theoretic systems (McCawley 1968, Rogers 1994, 1997, Kaplan 1995). In contrast, with the exception of the "rule-based" FGD (Sgall et al. 1969), the other three dependency frameworks considered here either explicitly define themselves as "constraint-based" (WG, MTT), or propose principles which are most naturally understood as "constraints" (SSDG).[7] However, as noted above, these dependency frameworks do not attempt to actually provide a model-theoretic formalization of the "constraint-based" approach to grammar that they seem to assume.

The fourth relevant dimension on which syntactic frameworks differ is more subtle and subjective, and it has to do with the relative difficulty of adopting a given framework for one's linguistic work. The constituency frameworks are relatively easy to start working with. This is especially true about Transformational Grammars, as they are clearly dominant in the field and taught at many linguistics departments. There are also multiple textbooks of particular versions of TG at different levels of advancement, e.g., the entry-level Carnie 2013 or more detailed Minimalist textbooks of Adger 2003 and Radford 2004, as well as numerous handbooks, e.g., Boeckx 2011. But also HPSG and LFG have various textbooks (e.g., Sag et al. 2003 introducing roughly HPSG machinery and the LFG textbooks Bresnan

[5] On the other hand, this statement is not fully accurate, as the other two dependency frameworks, MTT and FGD, are relatively formal – MTT is implemented in a (proprietary) NLP system, ETAP-3 (Apresjan et al. 2003), while a rule-based formalization of FGD is described in Sgall et al. 1969. However, as discussed in the "fourth dimension" paragraphs below, they are not as easy to adopt as SSDG and WG.

[6] The terms "rule-based" is sometimes used very generally, in a way that also encompasses constraints; cf. the similarly general "rule-bound" in Nefdt 2024.

[7] WG: "A language is a network of entities related by propositions" (Hudson 1984: 1), "... 'constraint-based' theories such as Word Grammar..." (Hudson 2010: 217), "WG, just like HPSG, is a constraint-based theory" (Hudson 2024: 1546), etc. MTT: "The [MTT] is by no means a generative or, for that matter, transformational system: it is a purely EQUATIVE (or translative) device. The rules of the [MTT] do not generate (i.e., enumerate, specify) the set of all and only grammatically correct or meaningful texts. They simply match any given SemR [i.e., semantic representation] with all PhonRs [i.e., phonetic representations] which, in accordance with native speakers' [linguistic] intuition, can convey the corresponding meaning; inversely, they match any given PhonR with all SemRs that can be expressed by the corresponding text" (Mel'čuk 1988: 45). It is more difficult to find such unequivocal declarations in the SSDG literature, but SSDG principles are naturally understood as constraints on structures, rather than as specifications of rewriting rules, e.g., the Principle of Full Clusivity in Osborne 2019: 342: "A constituent preceding a root of a coordinate structure must be included in, or excluded from, that coordinate structure entirely".

et al. 2016 and Börjars et al. 2019), as well as reference monographs (Müller et al. 2024 for HPSG and Dalrymple et al. 2019 and Dalrymple 2023 in the case of LFG), and similarly for various kinds of CGs (e.g., Wood 1993, Moot and Retoré 2012, Steedman 2000, 2024).

The other extreme on this dimension is the FGD dependency framework, with apparently no textbooks or reference monographs after the *locus classicus* Sgall et al. 1986.[8] MTT fares better in this respect, as there are a number of MTT monographs published after Mel'čuk 1988, including Polguère and Mel'čuk 2009 with a lengthy introductory article (Mel'čuk 2009) and the more recent Mel'čuk 2021. However, there are three additional reasons why MTT is not an easy framework to start working with. First, MTT is not a lean syntactic framework, but rather a rich linguistic system with many specific built-in assumptions about the structure of language and the lexicon. In particular, MTT is heavily multistratal, with seven different representation levels, each with its own data structures and mechanisms. On top of that, there is a very detailed and empirically rich approach to the lexicon associated with MTT, called Explanatory Combinatorial Dictionary (e.g., Mel'čuk and Zholkovsky 1984, Mel'čuk 2006). This "embarrassment of riches" makes MTT feel "heavy", perhaps too heavy for the taste of many theoretical linguists. Second, MTT employs a great deal of non-standard abbreviations and notational conventions that also make the learning curve steep. Finally, and relatedly, one of the sometimes perceived advantages of dependency approaches over constituency approaches is the relative simplicity (discussed below and in the Conclusion), but many MTT analyses seem – at least at first sight – anything but simple.

This problem is absent from WG and, especially, SSDG, which are both monostratal approaches, with particularly simple structures in the case of SSDG. After over half a century, an English translation of the foundational SSDG work Tesnière 1959 is now available (Tesnière 2015), as well as a comprehensive introduction ("and beyond"; Osborne 2019). Similarly, a number of WG monographs make learning the framework relatively easy, e.g., Hudson 2010, 2007, as well as the collection of papers in Sugayama and Hudson 2006. A monostratal approach is also assumed in the recent syntax textbook aimed at cognitive scientists, Gibson 2025, which is less complex than that of WG, as it mostly assumes simple trees rather than more complex networks of WG, but more complex than that of SSDG, as it assumes labelled dependencies and (somewhat implicitly) a rich internal structure of words.

We hope that the above considerations – perhaps too lengthy but at the same time too concise to do full justice to contemporary syntactic frameworks[9] – make the need for a model-theoretic formalization of dependency grammars clear. In brief, while all major constituency frameworks are more or less formalized, only one of the three dependency frameworks present in contemporary theoretical linguistic literature is partially formalized, namely, MTT; the other two current dependency approaches, SSDG and WG, lack any formalization. Additionally, many linguists may fail to consider MTT as a host framework for their work because of the linguistic "heaviness" of MTT, its somewhat opaque notation, and the perceived lack of simplicity. Hence, there is a gap to be filled, given that there is no formal DG framework comparable to SSDG or WG (or the approach assumed in Gibson 2025). As all three contemporary approaches – SSDG, WG, and MTT – explicitly or implicitly subscribe to the "constraint-based" view of grammar, the formalization of such a framework should be "model-theoretic", rather than "proof-theoretic".

The secondary need that even a cursory review of dependency literature makes clear is the need for simplicity. The simplicity of dependency approaches is hailed in the recent

[8] On the other hand, FGD is still alive in various Prague Dependency Treebanks, e.g., Bejček et al. 2013.

[9] Also because some important syntactic frameworks have not been mentioned here at all, including Tree Adjoining Grammars (Joshi 1985, 1987), various kinds of Construction Grammar (Goldberg 1995, Kay and Fillmore 1997, Kay 1998, Croft 2001, Boas and Sag 2012), and Simpler Syntax (Culicover and Jackendoff 2005), all of which should be classified as constituency rather than dependency frameworks. See Müller 2023 for a monograph-length comparison of grammatical theories, as well as Nefdt 2024: ch.3 for philosophical metatheoretical considerations.

paper Gibson 2026, but also emphasized in other strands of dependency literature, e.g., in Hudson 2007: 117,[10] in Osborne 2008: 1159,[11] and in de Marneffe and Nivre 2019: 205.[12] However, as discussed in the Conclusion, there is a tension between simplicity on one hand and explicitness and precision which accompany formalization on the other. For this reason, the current paper assumes a tiered approach to grammatical statements, with three levels of formalization, of increasing explicitness and precision, but decreasing perceived simplicity.

The first level, Simple, is limited to simple tree diagrams and vocabulary lists and it requires some additional prose to make the intention of these diagrams and lists fully clear. The second level, AVM, uses the language of Attribute-Value Matrices (AVMs) that should be easy to understand for linguists familiar with LFG, HPSG, or some strands of TGs. Statements at the AVM level are considerably more precise and unambiguous than those at the Simple level, but they still require an introduction of certain conventions and, in some cases, additional explication in prose. The third level, Logical, uses the language of a variant of First Order Logic (FOL) proposed here for formalizing DGs and called $\mathrm{L_{DG}}$ below. It is to some extent based on the logic for formalizing HPSG grammars (King 1989, 1994, 1999, Richter 1999, 2004, 2007), but differs from it in important respects. While statements at all three levels are meant to be synonymous, only statements in $\mathrm{L_{DG}}$ never require any additional interpretation in prose, as their interpretation follows from the definition of $\mathrm{L_{DG}}$ (in Appendix A).

2 Previous formalizations

Previous attempts to formalize dependency grammars are of two kinds: "rule-based" and "constraint-based". While constraint-based approaches show more affinity to our proposal in §§3–4, rule-based approaches are more directly related to the most recent attempt, Gibson 2025: ch.3, discussed in §5. Both approaches are briefly characterized below.

2.1 Rule-based formalizations

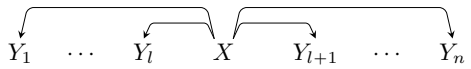
Rule-based formalizations of dependency grammars, described in Hays 1964 and Gaifman 1965, stem from work on Machine Translation around the year 1960. Formally, such dependency grammars may be represented similarly to formal constituency grammars:

(1) Formal Dependency Grammar – a quintuple $\langle N, T, P, S, L\rangle$, where:

a. N is a finite set of non-terminal symbols (e.g., grammatical categories),
b. T is a finite set of terminal symbols (e.g., words of a given language), $N \cap T = \varnothing$,
c. P is a set of rules (defined below),
d. $S \subseteq N$ is the set of start symbols (i.e., possible roots of dependency trees representing sentences),
e. L is a function from N to non-empty subsets of T (i.e., a lexicon).

Rules are presented in Hays 1964: 513 and Gaifman 1965: 305 as in (2a), but they may alternatively be represented in the more readable form in (2b).

(2) a. $X(Y_1 \cdots Y_l * Y_{l+1} \cdots Y_n)$

b. 

[10] "[D]ependency structures [...] are *very much simpler* than phrase structures." (Here and in the two following quotations emphasis is ours.)

[11] "Thus if both the constituency- and dependency-based accounts of the major constituent phenomena examined in this paper succeed at making the right predictions, then according to Occam's Razor, the dependency-based account should be preferred since [it] is *the simpler one*."

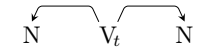
[12] "These reasons can be viewed as falling under three main advantages that dependency representations offer: generalization across languages, a convenient operationalization of human sentence processing facts, and *the transparency and simplicity of the representation*."

In such rules, $n \geqslant 0$; when $n = 0$, i.e., in the case of the rule $X(*)$, X has no dependents (i.e., it is a leaf in the dependency tree).

The way such grammars "generate" structures of sentences should be clear (see again Hays 1964 and Gaifman 1965 for step-by-step explications). For example, the grammar $\langle N, T, P, S, L\rangle$ in (3), with the rules visualized in (4) and the lexicon in (5), generates exactly 20 structures – 4 such as (6), rooted in the start symbol V_i lexicalized as *fell* and differing only in the lexical choice of the determiner (*the* or *a*) and the noun (*girl* or *boy*), and 16 such as (7), rooted in the other start symbol, V_t.

(3) a. $N = \{\mathrm{V}_i, \mathrm{V}_t, \mathrm{N}, \mathrm{D}\}$
b. $T = \{\mathit{fell}, \mathit{saw}, \mathit{boy}, \mathit{girl}, \mathit{the}, \mathit{a}\}$
c. $P = \{\mathrm{V}_i(\mathrm{N}\,*), \mathrm{V}_t(\mathrm{N}\,*\,\mathrm{N}), \mathrm{N}(\mathrm{D}\,*), \mathrm{D}(*)\}$ (see (4))
d. $S = \{\mathrm{V}_i, \mathrm{V}_t\}$
e. $L = \{\langle \mathrm{V}_i, \{\mathit{fell}\}\rangle, \langle \mathrm{V}_t, \{\mathit{saw}\}\rangle, \langle \mathrm{N}, \{\mathit{boy}, \mathit{girl}\}\rangle, \langle \mathrm{D}, \{\mathit{the}, \mathit{a}\}\rangle\}$ (see (5))

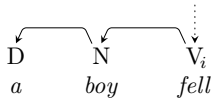
(4) a. 

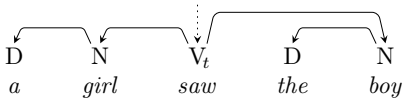
b. 

c. 

d. D

(5) a. V_i: *fell*
b. V_t: *saw*
c. N: *boy*, *girl*
d. D: *a*, *the*

(6) 

(7) 

As shown in Gaifman 1965, this Hays–Gaifman formalization of DGs is weakly equipotent to the formalism of context-free grammars (CFGs), i.e., for any grammar in one of these two formalisms there exists one in another that generates exactly the same sentences, and it is claimed to be strongly equipotent to a proper subset of CFGs, i.e., for each Hays–Gaifman dependency grammar there exists a context-free grammar that generates trees equivalent (in some well-defined sense) to the trees generated by that dependency grammar (but see Miller 1999: 74–75). These results might have created the impression that dependency grammars are simply imperfect notational variants of constituency grammars (Nefdt and Baggio 2024) and they might have been responsible for the fact that early attempts at incorporating DGs into formal theoretical linguistics, as in Robinson 1970, have been largely ignored.

However, the relation of Hays–Gaifman dependency grammars (HGDGs) to contemporary full-fledged dependency frameworks such as Word Grammar, Meaning–Text Theory, or Functional Generative Description is similar to the relation of bare CFGs to full-fledged constituency frameworks such as Government and Binding Theory, Lexical Functional Grammar, or Head-driven Phrase Structure Grammar – the mechanisms assumed in such linguistic frameworks go well beyond the basics of HGDGs and CFGs, so the equivalence or complexity results about these basic formal grammars do not say anything about the equivalence or complexity of corresponding full-fledged linguistic frameworks. This also concerns the rare attempts at more general – not CFG-equipotent – rule-based formalizations of DGs, as in Kuhlmann 2013, where mildly non-projective DGs are formalized via linear context-free rewriting systems (Vijay-Shanker et al. 1987), or as in Dekhtyar et al. 2015 and Kogkalidis 2023, where two very different formalizations based on Categorial Grammars are proposed. This is because such formalizations operate on very simple linguistic objects, typically just categories (nonterminals) and wordforms (terminals), and have nothing to say about phenomena that constitute the bread and butter of working syntacticians, such as agreement, case assignment, selectional restrictions, raising and control, long-distance dependencies, etc. Constraint-based formalizations, discussed in the following subsection, seem closer to the needs of theoretical dependency linguists.

2.2 Constraint-based formalizations

Some constraint-based formalisations of dependency grammars may be found in computationally-oriented work in 1990s and early 2000s, starting with Maruyama's (1990) Constraint Dependency Grammar (CDG). While, from the linguistic point of view, the original CDG did not offer any interesting analyses and was limited to simple toy grammars, it demonstrated that such grammars may be formalized via a set of constraints. One such toy grammar is limited to 3-word sentences consisting of a determiner (D), a noun (N), and an intransitive verb (V). Given the set of dependency relations: {*subj*, *det*, *root*}, as well as an appropriate assignment of categories D, N, and V to words, the grammar can be stated via the following four constraints, given both as logical formulae and in prose (cf. Maruyama 1990: 32–33):[13]

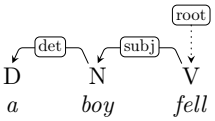
(8) a. $\forall x.\, \mathrm{cat}(x) = \mathrm{D} \rightarrow (\mathrm{rel}(x) = \mathit{det} \wedge \mathrm{cat}(\mathrm{gov}(x)) = \mathrm{N} \wedge x < \mathrm{gov}(x))$
b. 'Each determiner has the incoming dependency relation *det* and is governed by a noun occurring on the right.'

(9) a. $\forall x.\, \mathrm{cat}(x) = \mathrm{N} \rightarrow (\mathrm{rel}(x) = \mathit{subj} \wedge \mathrm{cat}(\mathrm{gov}(x)) = \mathrm{V} \wedge x < \mathrm{gov}(x))$
b. 'Each noun has the incoming dependency relation *subj* and is governed by a verb occurring on the right.'

(10) a. $\forall x.\, \mathrm{cat}(x) = \mathrm{V} \rightarrow (\mathrm{rel}(x) = \mathit{root} \wedge \mathrm{gov}(x) = \mathit{nil})$
b. 'Each verb has the dependency label *root* and is not governed.'

(11) a. $\forall x. \forall y.\, (\mathrm{gov}(x) = \mathrm{gov}(y) \wedge \mathrm{rel}(x) = \mathrm{rel}(y)) \rightarrow x = y$
b. 'No two different words can be governed by the same word and have the same incoming dependency relation.'

In these formulae, cat is a function from words to categories, rel is a function from words to dependency relations, gov is a partial function from words to words (the lack of value is represented by the special value *nil*), and $<$ is a relation on words representing word order. For example, assuming appropriate categories of words: *a*, *boy*, and *fell*, the following dependency tree satisfies ("is licensed by") these constraints.

(12)


Conversely, there are no dependency trees that would satisfy all constraints in (8)–(11) for sequences **A fell* (violates at least (8)) or **Boy boy fell* (violates at least either (9) or (11)).

Schröder et al. 2000 extended CDG by adding weights to constraints and, importantly, by considering linguistically more interesting constraints, on agreement and selectional restrictions, formulated on the basis of German.[14] For example, one of their constraints says that determiners agree with nouns in grammatical case; it may be reformulated using our conventions as in (13) (cf. Schröder et al. 2000: 24, ex. (C17)).

(13) a. $\forall x.\, \mathrm{rel}(x) = \mathit{det} \rightarrow \mathrm{case}(x) = \mathrm{case}(\mathrm{gov}(x))$
b. 'Each determiner agrees with its governor in case.'

Schröder et al. 2000 also assume a rich internal structure of words, encoded as attribute-value matrices (AVMs). For example, the structures of *wir* 'we' and *verschieben* 'postpone' may be represented as in (14)–(15), using the HPSG typesetting conventions adopted in the remainder of the current paper.[15]

[13] Various conventions of Maruyama 1990 have been modified here in order to make them more in line with our proposals in §§3–4.

[14] An implementation of an English (Weighted) CDG is briefly described in By 2004, where a number of constraints on selectional restrictions are proposed.

[15] However, unlike in HPSG, we represent paths – sequences of attributes – by separating attributes with spaces rather than the pipe symbol, e.g., as MORPH CASE in (15) rather than MORPH|CASE.

(14)
$$\begin{bmatrix} \textsc{word}\ \textit{wir} \\ \textsc{cat}\ \textit{pronoun} \\ \textsc{morph}\ \begin{bmatrix} \textsc{case}\ \textit{nom} \\ \textsc{num}\ \textit{pl} \\ \textsc{pers}\ \textit{1} \end{bmatrix} \end{bmatrix}$$

(15)
$$\begin{bmatrix} \textsc{word}\ \textit{verschieben} \\ \textsc{cat}\ \textit{finverb} \\ \textsc{args}\ \begin{bmatrix} 1 \begin{bmatrix} \textsc{optional}\ \textit{no} \\ \textsc{morph}\ \begin{bmatrix} \textsc{case}\ \textit{nom} \\ \textsc{num}\ \textit{pl} \\ \textsc{pers}\ \textit{1} \end{bmatrix} \end{bmatrix} \\ 2 \begin{bmatrix} \textsc{optional}\ \textit{no} \\ \textsc{morph case}\ \textit{acc} \end{bmatrix} \\ 3 \begin{bmatrix} \textsc{optional}\ \textit{yes} \\ \textsc{cat}\ \textit{prep} \\ \textsc{prep}\ \textit{auf} \\ \textsc{morph case}\ \textit{acc} \end{bmatrix} \end{bmatrix} \end{bmatrix}$$

Given such AVM representations, (13) could be rewritten with some syntactic sugar added as in (16), where attributes in small capitals follow terms denoting objects having these attributes.

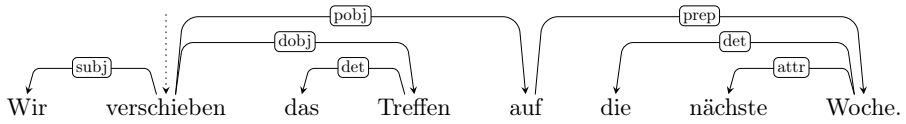
(16) a. $\forall x.\, \mathrm{rel}(x) = \textit{det} \rightarrow x\ \textsc{case} = \mathrm{gov}(x)\ \textsc{case}$
b. 'Each determiner agrees with its governor in case.'

Such syntactic sugar becomes more useful in complex constraints such as (17), which says that the subject's category and morphosyntactic features must satisfy the requirements imposed by its governor (cf. Schröder et al. 2000: 10, ex. (C3)).

(17) a. $\forall x.\, \mathrm{rel}(x) = \textit{subj} \rightarrow$
$(\mathrm{gov}(x)\ \textsc{args}\ 1\ \textsc{cat} = x\ \textsc{cat} \wedge$
$\mathrm{gov}(x)\ \textsc{args}\ 1\ \textsc{morph case} = x\ \textsc{morph case} \wedge$
$\mathrm{gov}(x)\ \textsc{args}\ 1\ \textsc{morph num} = x\ \textsc{morph num} \wedge$
$\mathrm{gov}(x)\ \textsc{args}\ 1\ \textsc{morph pers} = x\ \textsc{morph pers})$
b. 'Each subject must satisfy selectional restrictions on its governor's first argument.'

These constraints are satisfied by the simplified structure in (19) for the German sentence in (18) (cf. Schröder et al. 2000: 11, Figure 6).

(18) *Wir* *verschieben* *das* *Treffen* *auf* *die* *nächste* *Woche.*
we.NOM postpone the.ACC meeting.ACC on the.ACC next week.ACC
'We're postponing the meeting until next week.'

(19)


Importantly, the above structure is just a shorthand for a much more complex structure, where each word is represented by AVMs such as (14)–(15) in the case of the first two words in (19). As will become clear, our proposal below bears more than a passing resemblance to this approach.

Unfortunately, this CDG work has not influenced theoretical linguistic approaches. While some of it was relatively sophisticated linguistically, at least compared to the early CDG work and other work within NLP, it never approached the level of sophistication of full-fledged dependency frameworks – nor did it intend to. Instead, the goal of this work was practical rather than theoretical, with much emphasis on parsing efficiency (see, e.g., Foth et al. 2006, Beuck et al. 2011, and Köhn and Menzel 2013), and with various limitations of the assumed constraint language (e.g., unavailability of existential quantification)

introduced in order to increase efficiency, but making it cumbersome for actual linguistic analysis (see Schröder et al. 2000). Also the related formalism of Extensible Dependency Grammar (Debusmann et al. 2004) concentrated on its formal and computational properties and remained similarly unnoticed within theoretical linguistics.[16] Moreover, none of these computational DG approaches developed – or even hinted at – an explicit model theory of the kind assumed in HPSG. This is what we will do in the remainder of this paper.

3 Basics

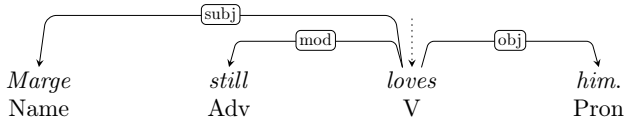
Let us start with the basics of Model-Theoretic Dependency Grammar (MTDG).

3.1 Models

The gist of MTDG dependency representations is presented in (21)–(23), which show different visualizations of the dependency structure of (20):

(20) *Marge still loves him.*

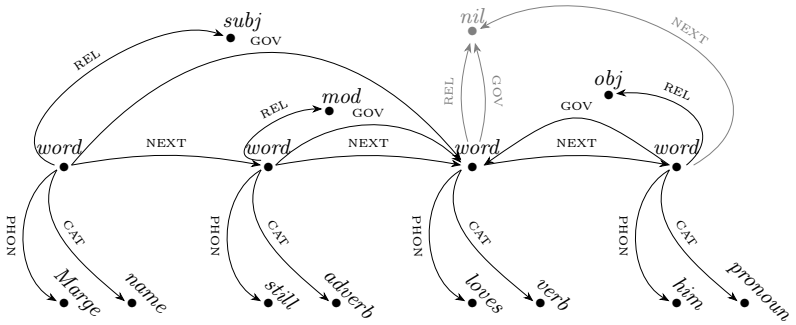
(21)



(22)

$$\boxed{1}\begin{bmatrix} word \\ \text{PHON } Marge \\ \text{CAT } name \\ \text{REL } subj \\ \text{GOV } \boxed{3} \\ \text{NEXT } \boxed{2} \end{bmatrix} \boxed{2}\begin{bmatrix} word \\ \text{PHON } still \\ \text{CAT } adverb \\ \text{REL } mod \\ \text{GOV } \boxed{3} \\ \text{NEXT } \boxed{3} \end{bmatrix} \boxed{3}\begin{bmatrix} word \\ \text{PHON } loves \\ \text{CAT } verb \\ \text{REL } nil \\ \text{GOV } nil \\ \text{NEXT } \boxed{4} \end{bmatrix} \boxed{4}\begin{bmatrix} word \\ \text{PHON } him \\ \text{CAT } pronoun \\ \text{REL } obj \\ \text{GOV } \boxed{3} \\ \text{NEXT } nil \end{bmatrix}$$

(23)



While (21)–(23) are different representations of the assumed *model* of (20), they correspond to the three levels of *descriptions* (constraints, formulae) introduced in §1: Simple, AVM, and Logical.

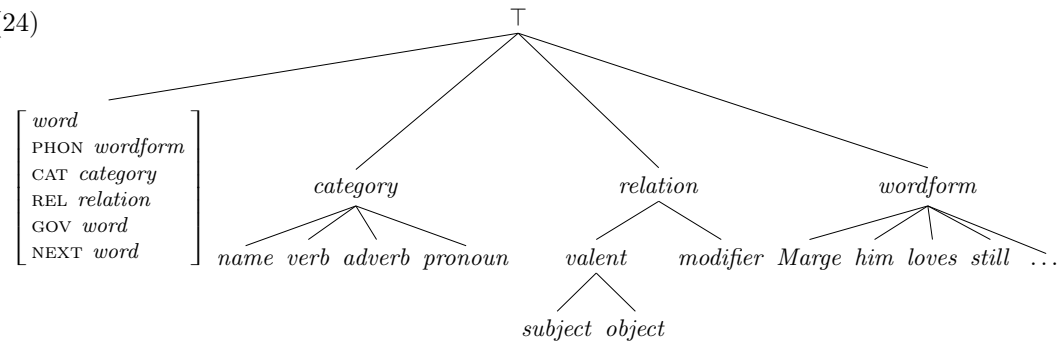
The tree in (21), corresponding to the Simple level of description, is about the simplest possible dependency representation of (20). It says that *loves* is the root of this tree and that it has three dependents: the subject *Marge*, the direct object *him*, and the modifier *still*. Morphosyntactic information about particular words is often present in such trees, here limited to part-of-speech (Name for proper noun, Adv for adverb, V for verb, Pron for pronoun). This indicates that nodes in dependency trees are not just word forms, but have some internal structure in MTDG. Additionally, also linear information is represented in (21).

[16] As well as completely abandoned by around 2010; see https://www.ps.uni-saarland.de/~rade/xdg.html.

In theoretical linguistics, structured information is frequently represented via attribute-value matrices, used in our AVM level of statements. The AVMs in (22) follow certain typographical conventions and substantive ideas of HPSG. In particular, such AVMs are of certain sorts, here the *word* sort. Moreover, values of attributes (often also called "features") are also understood as – possibly atomic – objects of certain sorts, so *name*, *subj*, *adverb*, *mod*, etc., are also sorts.[17]

Also as in HPSG, in MTDG such sorts are organized in a sort hierarchy, called *signature*. The hierarchy in (24), with $\top$ as the most general sort, is compatible with the representation in (22). It will be extended in the following sections.

(24)



Such signatures not only define hierarchies of sorts, but also specify which sorts correspond to AVMs – what attributes must be present in such AVMs and what sorts of values these attributes may have – and which are atomic. Here, only the sort *word* introduces attributes and constrains their values. (Other sorts of AVMs will be introduced in the following sections.)

Apart from sorts explicitly defined in the signature, we assume a special sort, *nil*, which can be the value of any attribute. The role of this special sort is to make attributes optional in principle; when the value of an attribute is *nil*, it is understood as in a sense absent (hence, in grey in (22)). For example, while the sort *word* is specified for the attribute NEXT, the last word in the utterance has no following word, so NEXT is not needed there. Also, while almost all words have a governor and an appropriate grammatical function, the root word does not, so for this – and only this – word the attributes GOV and REL, although specified as generally appropriate for any *word*, are not needed and, hence, should be *nil*-valued.

Again as in HPSG, boxed numbers are understood at the AVM level of MTDG as variables referring to particular objects, e.g., 2 refers to the *word* structure for *still*. Hence, in the *word* structure 1 for *Marge*, the value of the attribute GOV is the *word* structure 3 (representing the word *loves*), and the value of NEXT is the *word* structure 2 (representing *still*).

It is easy to see that (22) constitutes a minimal and complete AVM representation of the tree (21). Word forms and their categories are presented via the attributes PHON and CAT, their dependency labels and governors via REL and GOV, and the linear order via NEXT.

Such AVMs are still not models in the sense of first order logic (FOL), but they represent such models relatively directly, as illustrated in (23) and formalized in Appendix C. Such models correspond directly to the Logical level of descriptions in MTDG. In this case, the domain of the model contains 16 objects: four objects of type *word* corresponding to the AVMs 1–4 in (22), four objects representing word forms and another four representing categories of these words, three objects representing dependency labels, and an object of the special sort *nil*. As in HPSG, each object in the model is of exactly one minimal sort: either *word*, or *name*, or *verb*, …, or *subject*, …, or *Marge*, etc. For example, if a given

[17] Technically, also values of PHON (*Marge*, *still*, etc.) are understood as sorts here, *in lieu* of a more adequate phonological representation. For a possible representation of phonology in such a system, see Höhle 1999.

object is of the non-minimal sort *relation*, it must be of exactly one of its minimal sorts: *subject*, *object*, or *modifier*.[18] As formally defined in Appendices A–C, such sorts are simply unary predicates, while attributes are encoded as partial functions.

In summary, simple dependency trees such as (21) correspond to structures which are more explicitly represented by AVMs such as (22). These structures are FOL models, which may be visualized as graphs, as in (23), or given more formally, as in Appendix C. In what follows, we assume similar three levels of description – Simple, AVM, and Logical – in formulating linguistic constraints that such models must satisfy.

3.2 Theories

In logic, theories are sets of closed formulae (what linguists sometimes call "constraints" or "principles") that must be true in a model. It is useful to distinguish two types of such formulae: framework constraints and linguistic constraints.[19] We present examples of the former kind in the remainder of this section, using only Logical statements and their English paraphrases, while examples of linguistic constraints – formulated in the Simple way, in the language of AVMs, and via Logical formulae – are proposed in §4.

Framework constraints define what kinds of dependency structures are assumed in a given dependency framework, for example, whether an utterance is represented by a single syntactic dependency structure (as in monostratal theories such as the Surface Syntactic approaches of Tesnière 1959, 2015 and Osborne 2019 or as Hudson's 1984, 1990, 2010 Word Grammar), or multiple dependency structures (as in multistratal theories such as Meaning–Text Theory of Mel'čuk 1988, 2009 or Functional Generative Description of Sgall et al. 1986), whether the structures are simple dependency trees (as mostly assumed in SSDG) or more complex graphs (as commonly assumed in WG), whether non-projectivity ("crossing dependencies") is allowed or not. In this section, we formalize constraints defining monostratal projective dependency trees, as often assumed in SSDG (and in Gibson 2025: ch.3).

Regardless of the specific dependency framework, nodes – here assumed to be words – must be linearly ordered. The following constraint – first given in prose and then as a Logical statement – ensures such a total ordering:[20]

(25) Total Word Order Constraint (TWOC)

a. 'For any two distinct words, one asymmetrically precedes the other.'

b. $\forall x.\forall y.\,(x : \mathit{word} \wedge\ y : \mathit{word} \wedge\ x \neq y) \rightarrow (\texttt{precede}(x, y)\ \veebar\ \texttt{precede}(y, x))$

The binary relation `precede` employed in (25) is defined in such a way that it holds of two words if the first one linearly precedes the second:[21]

(26) a. 'x precedes y iff both are words and either x immediately precedes y, or it immediately precedes some z that precedes y.'

b. $\forall x.\forall y.\,\texttt{precede}(x, y) \leftrightarrow (x : \mathit{word} \wedge\ y : \mathit{word} \wedge\ (x\ \textsc{next} = y\ \vee\ \exists z.\, x\ \textsc{next} = z \wedge \texttt{precede}(z, y)))$

---

[18] Our *minimal* sorts are minimal in the sense of the partial order $\sqsubseteq$ encoded in hierarchies such as (24), where $\top$ is the single largest (hence, also maximal) element, and the sorts *word*, *name*, etc. (but not *category*, *valent*, ..., $\top$) are minimal. Such minimal sorts are also maximally informative, which is probably the reason why they are called *maximal* in HPSG.

[19] In Appendix D, we also discuss another kind of constraints, "model constraints", which ensure minimality of models.

[20] $\veebar$ in (25b) stands for exclusive *or* (XOR) and may be defined as: $p \veebar q \stackrel{df}{\equiv} p \leftrightarrow \neg q$.

[21] According to the signature (24), the value of NEXT is of sort *word* or the special sort *nil*. At most one word – the last word – may be [NEXT *nil*], as otherwise two [NEXT *nil*] words would not stand in the `precede` relation, contrary to TWOC in (25). Also, the value of the last word's NEXT cannot be any earlier word, as then both words would symmetrically precede each other, again contrary to TWOC. However, nothing so far precludes the possibility that the last word's NEXT is cyclically that last word itself (rather than the intended *nil*), so we need to preclude it explicitly:

(i) $\forall x.\, x\ \textsc{next} \neq x$

These Logical formulae use some syntactic sugar in order to make them a little more readable to linguists. In particular, we distinguish two kinds of relation symbols: unary predicate symbols corresponding to sorts, written in italics and using the colon notation (e.g., $y : \mathit{word}$), and others, of any positive arity, defined as part of the theory, written in the typewriter font (e.g., `precede`). Moreover, "x NEXT" in (26b) stands for the value of the attribute NEXT (written in small capitals) on object x. Formally, attributes are (partial) functions, so a more standard way of writing "x NEXT $= z$" would be "$\textsc{next}(x) = z$", but this more standard notation would get cumbersome in the case of some linguistic constraints in §4 which involve sequences of attributes.[22] Without such syntactic sugar, the formula in (26b) would look as in (27):

(27) Formula (26b) without syntactic sugar

$$\forall x.\forall y.\, \mathrm{precede}(x,y) \leftrightarrow (\mathrm{word}(x) \wedge \mathrm{word}(y) \wedge (\mathrm{next}(x) = y \vee \exists z.\, \mathrm{next}(x) = z \wedge \mathrm{precede}(z,y)))$$

Dependency trees must satisfy three axioms, the first of which does not have to be formalized as a separate constraint, as it is already encoded in the signature:

(28) Single Governor Constraint

'Every word has at most one governor.'[23]

(29) Root Constraint[24]

a. 'There is exactly one root.'
b. $\exists!x.\, \mathtt{root}(x)$

(30) Connectivity Constraint

a. 'Every non-root word must be a descendant of the root.'
b. $\forall x.\, x : \mathit{word} \rightarrow (\mathtt{root}(x) \vee \exists y.\, \mathtt{root}(y) \wedge \mathtt{descendant}(x,y))$

The above constraints make use of relations `root` and `descendant` defined as follows:

(31) a. 'Root is a word without a governor.'
b. $\forall x.\, \mathtt{root}(x) \leftrightarrow x\ \textsc{gov} : \mathit{nil}$

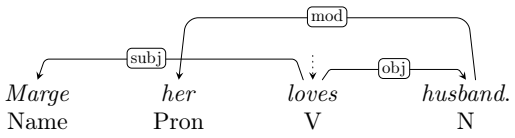
(32) a. 'x is a descendant of y iff either y is a governor of x or some other z is a governor of x and, at the same time, a descendant of y.'
b. $\forall x.\forall y.\, \mathtt{descendant}(x,y) \leftrightarrow (x\ \textsc{gov} = y \vee \exists z.\, x\ \textsc{gov} = z \wedge \mathtt{descendant}(z,y))$

Of course, GOV and REL, which jointly define a labelled dependency, must go hand in hand – either both are defined or neither is:

(33) a. 'A word has no governor iff it has no grammatical function.'; alternatively: 'A node has no incoming arc iff it has no dependency label.'
b. $\forall x.\, x\ \textsc{gov} : \mathit{nil} \leftrightarrow x\ \textsc{rel} : \mathit{nil}$

The above tree axioms do not yet rule out non-projective trees such as (34).

(34)



[22] For example, without this syntactic sugar, the atomic subformula $x\ \textsc{val rest rest} : \mathit{elist}$ in (85) in §4 below would be: elist(rest(rest(val(x)))).

[23] This requirement is already present in (24): each *word* has GOV whose value is a single *word* or *nil*.

[24] The formula in (29b) uses the ∃! convention for "exists exactly one"; it can be defined as usual, as: $\exists!x.\, \phi(x) \stackrel{df}{\equiv} \exists x.\, \phi(x) \wedge \forall y.\, \phi(y) \rightarrow y = x$.

For this, a projectivity constraint is needed:[25]

(35) Projectivity Constraint

a. 'If x is governed by y and some z is between them, then z must also be a descendant of y.'

b. $\forall x.\forall y.\forall z.\,(x\ \textsc{gov} = y \wedge \texttt{between}(z,x,y)) \rightarrow \texttt{descendant}(z,y)$

The relation `between` is defined as might be expected:

(36) a. 'z is between x and y iff either it follows x and precedes y or it follows y and precedes x.'

b. $\forall x.\forall y.\forall z.\,\texttt{between}(z,x,y) \leftrightarrow$
$((\texttt{precede}(x,z) \wedge \texttt{precede}(z,y)) \vee$
$(\texttt{precede}(y,z) \wedge \texttt{precede}(z,x)))$

Having defined a dependency framework, we are ready to move to the more interesting linguistic constraints.

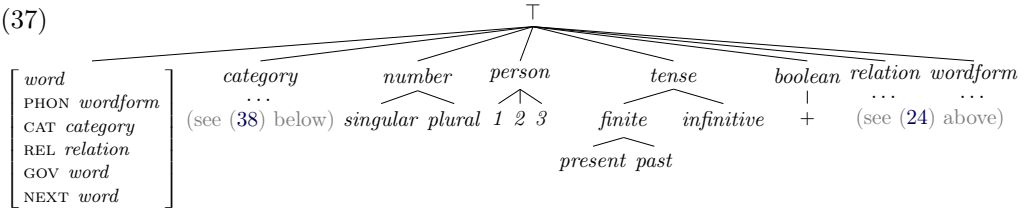
4 Linguistic constraints

It is not an aim of this paper to provide a comprehensive analysis of any particular phenomenon; the aim is only to illustrate how such analyses might be formulated in the model-theoretic approach to dependency grammars proposed here. A recent syntax textbook targeted at cognitive scientists, Gibson 2025, has a similar aim: to show how certain phenomena may be analysed in a "feature-based dependency grammar formalism" (p. 94). Unfortunately, there is very little formalism in that proposal and, as discussed in §5 below, whatever formalism there is, it is not very precise or even fully coherent (which – incidentally – demonstrates the need for a more formal approach such as that proposed in the current paper). Nevertheless, given the similarities of goals, we aim to cover roughly the same empirical ground as in Gibson 2025: ch.3.

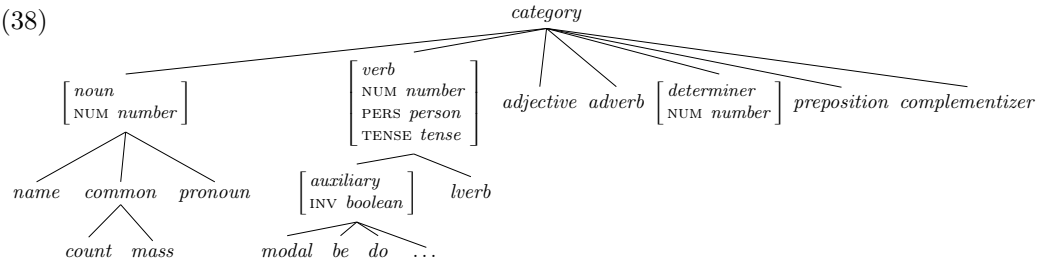
4.1 Categories

Let us start with extending morphosyntactic parts of the signature in (24), where the only subsorts of *category* were *name*, *pronoun*, *verb*, and *adverb*, and which did not define any morphosyntactic features (such as number or tense) – see (37)–(38).

(37)



(38)

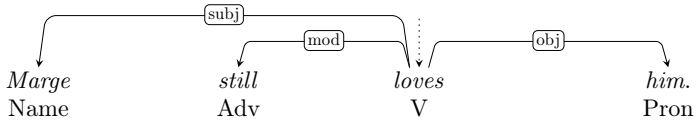


[25] This assumes that projectivity should be handled in grammar proper, while it actually seems to follow to a large extent from human sentence processing constraints (see, e.g., Ferrer-I-Cancho and Gómez-Rodríguez 2016 and Yadav et al. 2022). This means that a constraint such as (35) might not be needed in grammar proper.

As in Gibson 2025: ch.3, there are 7 parts of speech defined in this signature (see the 7 immediate subsorts of *category* in (38)), with *noun* further subdivided into *name* (proper noun), *count* and *mass common* nouns, and *pronoun*, and *verb* subdivided into lexical verbs of sort *lverb* and various sorts of – in principle invertible – *auxiliary* verbs. Different parts of speech introduce possibly different morphosyntactic features, e.g., nouns, verbs, and determiners may inflect for number, with all verbs additionally inflecting for person and tense, and auxiliary verbs having the boolean (binary) INV feature on top of that.[26] Possible values of these features are defined in (37).[27]

Given the subhierarchy in (38), some *category* objects are atomic, e.g., those of minimal sorts *adverb* or *complementizer*, and some correspond to AVMs, e.g., *name*, which inherits the attribute NUM from *noun*, or *modal*, which inherits four attributes from its supersorts: NUM, PERS, TENSE, and INV. With these extensions in hand, the Simple tree (21) for the running example *Marge still loves him*, repeated below as (39), will now have the more explicit AVM representation in (40) and the correspondingly extended Logical model in (41).[28]

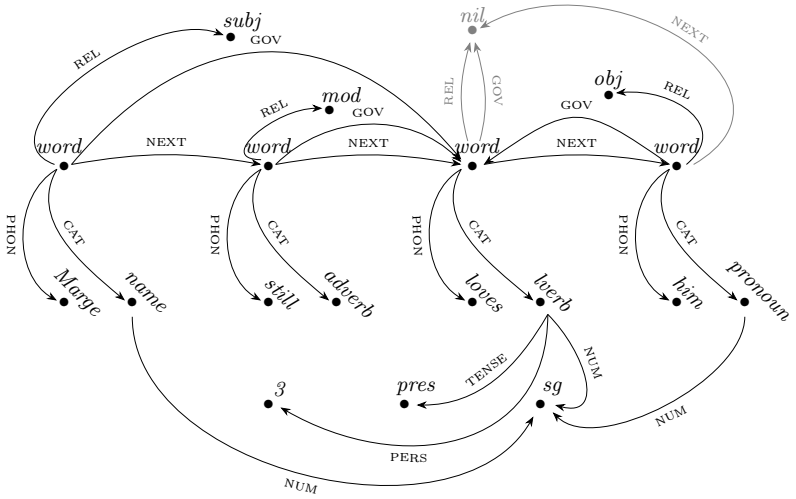
(39)



(40)

$$\boxed{1}\begin{bmatrix} word \\ \text{PHON } Marge \\ \text{CAT } \begin{bmatrix} name \\ \text{NUM } sg \end{bmatrix} \\ \text{REL } subj \\ \text{GOV } \boxed{3} \\ \text{NEXT } \boxed{2} \end{bmatrix} \quad \boxed{2}\begin{bmatrix} word \\ \text{PHON } still \\ \text{CAT } adverb \\ \text{REL } mod \\ \text{GOV } \boxed{3} \\ \text{NEXT } \boxed{3} \end{bmatrix} \quad \boxed{3}\begin{bmatrix} word \\ \text{PHON } loves \\ \text{CAT } \begin{bmatrix} lverb \\ \text{NUM } sg \\ \text{PERS } 3 \\ \text{TENSE } pres \end{bmatrix} \\ \text{NEXT } \boxed{4} \end{bmatrix} \quad \boxed{4}\begin{bmatrix} word \\ \text{PHON } him \\ \text{CAT } \begin{bmatrix} pronoun \\ \text{NUM } sg \end{bmatrix} \\ \text{REL } obj \\ \text{GOV } \boxed{3} \end{bmatrix}$$

(41)



4.2 Agreement

Only two instances of agreement are considered in Gibson 2025: ch.3: subject–verb agreement and noun–determiner agreement (both in number). As discussed in §5, subject–verb

[26] The sort *boolean* has only one subsort, +, as − is represented by *nil*, which may be the value of any attribute, unless explicitly prohibited (more on this in fn. 33 below).

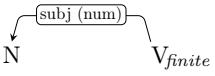
[27] Gibson 2025: 54 additionally mentions the boolean COUNT feature of determiners, but it is not used in any analyses or representations there, so we ignore it here.

[28] The usual abbreviations are used here, e.g., *sg* for *singular*, etc. Additionally, *nil*-valued attributes are now omitted in the AVM representation.

agreement (e.g., *The president lies* vs. *The presidents lie*) is formalized there within a relatively large number of syntactic rules. By contrast, a single statement is sufficient in MTDG:[29]

(42) Subject–Verb Agreement

a. 'Finite verbs agree in number with their nominal subjects.'

b. 

c. $\begin{bmatrix} \text{CAT } \textit{noun} \\ \text{REL } \textit{subj} \\ \text{GOV CAT TENSE } \textit{finite} \end{bmatrix} \rightarrow \begin{bmatrix} \text{CAT NUM } \boxed{1} \\ \text{GOV CAT NUM } \boxed{1} \end{bmatrix}$

d. $\forall x.\,(x \text{ CAT} : \textit{noun} \wedge x \text{ REL} : \textit{subj} \wedge x \text{ GOV CAT TENSE} : \textit{finite}) \rightarrow$
$\exists y.\, x \text{ CAT NUM} = y \wedge x \text{ GOV CAT NUM} = y$

This is the first linguistic constraint formalized in MTDG, so let us look at the various ways of expressing it. The first, in (42a), is in English prose and it requires no explanation.

The second, in (42b), is our attempt at a Simple formalization, as close to Gibson's (2025: 94) "feature-based dependency grammar formalism" as possible. This attempt is a little confusing, as it is not immediately clear what statement (42b) expresses – it could as well be 'If a nominal subject agrees in number with its governor, then this governor is a finite verb', among other statements reasonably expressed by (42b). Also, it suggests that directionality matters, as the subject is drawn as preceding the verb, while the constraint is meant to apply also in inverted constructions (e.g., *Does the president lie?* vs. *Do the presidents lie?*).

The third encoding of the constraint, in (42c), is at the AVM level. It is much more precise and explicit than the Simple diagram in (42b): it says that if some noun is the subject of a finite governor, then this noun and this governor have the same number. However, as in similar HPSG constraints (e.g., Davis and Koenig 2024: 154–155), it relies on implicit quantification: universal quantification over the whole formula ("every object satisfying the AVM description in the antecedent of the implication must also satisfy the description in the consequent") and existential quantification within the consequent of the implication ("there exists an object $\boxed{1}$ that is the simultaneous value of the two attribute paths").

Finally, the Logical formula in (42d), while using some syntactic sugar explained above, is fully explicit. The logical structure of this formula mirrors the AVM formalization in (42c), but of course other formulae expressing the same constraint are also possible, as in (42d′) and (42d″) below.

(42) d′. $\forall x.\,(x \text{ CAT} : \textit{noun} \wedge x \text{ REL} : \textit{subj} \wedge x \text{ GOV CAT TENSE} : \textit{finite}) \rightarrow$
$x \text{ CAT NUM} = x \text{ GOV CAT NUM}$

d″. $\forall x.\forall y.\,(x \text{ CAT} : \textit{noun} \wedge x \text{ REL} : \textit{subj} \wedge x \text{ GOV} = y \wedge y \text{ CAT TENSE} : \textit{finite}) \rightarrow$
$x \text{ CAT NUM} = y \text{ CAT NUM}$

Note that this constraint is true in the model shown in (41). Concentrating on the formalization in (42d′), for almost all values of x the implication under $\forall x$ is true because these values make the antecedent false (most objects in the model are not nominal subjects governed by a finite verb). The only object that makes the antecedent true is the *word* object for *Marge* (marked as $\boxed{1}$ in the AVM representation in (40)): its CAT value is of sort *noun* (given that *name* is a subsort of *noun*), its REL is *subj*, and its GOV is the *word* object for *loves* (marked as $\boxed{3}$ in (40)), so its GOV CAT TENSE is *finite* (as it is *pres*, which is a subsort of *finite*). But for x equal to this object, also the consequent of this implication is true, as this object's CAT NUM is equal to its governor's CAT NUM – they are both the

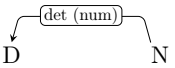
[29] Recall that the empirical scope of MTDG analyses proposed in the current §4 is limited to that covered in Gibson 2025: ch.3 and that we do not aim at comprehensiveness or novelty.

object of sort *sg* (i.e., *singular*). Hence, the implication under the universal quantifier $\forall x$ is true for all values of x, so the whole formula is true in this model.

The other instance of agreement, concerning nouns and determiners (e.g., *this president* vs. *these presidents*), is fully analogous:[30]

(43) Noun–Determiner Agreement

a. 'Nouns agree in number with their determiners.'

b. 

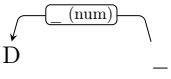
c. $\begin{bmatrix} \text{CAT } \textit{determiner} \\ \text{REL } \textit{det} \\ \text{GOV CAT } \textit{noun} \end{bmatrix} \rightarrow \begin{bmatrix} \text{CAT NUM } \boxed{1} \\ \text{GOV CAT NUM } \boxed{1} \end{bmatrix}$

d. $\forall x.\,(x \text{ CAT} : \textit{determiner} \wedge x \text{ REL} : \textit{det} \wedge x \text{ GOV CAT} : \textit{noun}) \rightarrow$
$x \text{ CAT NUM} = x \text{ GOV CAT NUM}$

Depending on other assumptions made in the grammar, such constraints may be simplified or may need to be more complex. For example, if determiners are always governed by nouns and if the exact label they bear does not matter, than the category of the governor and the dependency label may be dropped in (43), resulting in (44):

(44) Noun–Determiner Agreement (simplified)

a. 'Determiners agree in number with their governors.'

b. 

c. $\left[\text{CAT } \textit{determiner}\right] \rightarrow \begin{bmatrix} \text{CAT NUM } \boxed{1} \\ \text{GOV CAT NUM } \boxed{1} \end{bmatrix}$

d. $\forall x.\, x \text{ CAT} : \textit{determiner} \rightarrow x \text{ CAT NUM} = x \text{ GOV CAT NUM}$

In what follows, we will not attempt to arrive at such simplest possible versions of the proposed constraints.

4.3 Lexicon

In the Simple representation, the lexicon may be given as a list of categories and words bearing these categories, as in (45)–(59) (based on Gibson 2025: 55, 62–63, 69–73).[31]

(45) $\text{N}_{name,sg}$: *Lena*, *Ollie*

(46) $\text{N}_{count,sg}$: *girl*, *boy*, *cat*, *ball*

(47) $\text{N}_{count,pl}$: *girls*, *boys*, *cats*, *balls*

(48) $\text{N}_{count,sg/pl}$: *sheep*

(49) $\text{N}_{pronoun,sg}$: *I*, *me*, *you*, *he*, *him*, *she*, *her*

(50) $\text{N}_{pronoun,pl}$: *we*, *us*, *you*, *they*, *them*

(51) D_{sg}: *a*, *every*, *this*

(52) D_{pl}: *these*, *many*

(53) $\text{D}_{sg/pl}$: *the*, *some*

(54) $\text{V}_{sg,1/2,pres}$: *chase*, *eat*

(55) $\text{V}_{sg,3,pres}$: *chases*, *eats*

[30] This constraint assumes a dependency label, *det*, not introduced explicitly above. See §4.6 for an explicit introduction of dependency labels of determiners.

[31] In Gibson 2025: ch.3, comma sometimes means conjunction (e.g., *name,sg*), and at other times disjunction (e.g., *sg,pl*); in order to avoid this confusion, disjunction is marked here with a slash (e.g., *sg/pl*), and it is assumed to bind stronger than conjunction.

(56) $\mathrm{V}_{pl,1/2/3,pres}$: *chase*, *eat*

(57) $\mathrm{V}_{sg/pl,1/2/3,past}$: *chased*, *ate*

(58) $\mathrm{V}_{be,sg,3,pres,\pm inv}$: *is*

(59) $\mathrm{V}_{be,sg/pl,1,pres,+inv}$: *aren't*

In the AVM representation, lexical entries (LEs) for *Ollie*, *girls*, and *sheep* could be defined more explicitly as follows:[32]

(60) $\mathrm{LE}_{Ollie} \overset{df}{\equiv} \begin{bmatrix} \textsc{phon} & \textit{Ollie} \\ \textsc{cat} & \begin{bmatrix} \textit{name} \\ \textsc{num} \ \textit{sg} \end{bmatrix} \end{bmatrix}$

(61) $\mathrm{LE}_{girls} \overset{df}{\equiv} \begin{bmatrix} \textsc{phon} & \textit{girls} \\ \textsc{cat} & \begin{bmatrix} \textit{count} \\ \textsc{num} \ \textit{pl} \end{bmatrix} \end{bmatrix}$

(62) $\mathrm{LE}_{sheep} \overset{df}{\equiv} \begin{bmatrix} \textsc{phon} & \textit{sheep} \\ \textsc{cat} & \textit{count} \end{bmatrix}$

Note that, instead of specifying the grammatical number of *sheep* disjunctively, it is left here underspecified. According to (38), since *count* is a subsort of *noun*, it has the attribute NUM, whose value is one of the maximally specific subsorts of *number*, i.e., *singular* (abbreviated above to *sg*) or *plural* (abbreviated to *pl*).[33] Similarly, NUM values do not have to be stated explicitly for some determiners (cf. (53)), for past forms of verbs (cf. (57)), and for the inverted copula *aren't* (cf. (59); e.g., *Aren't I honest?*), the value of PERS does not have to be mentioned in lexical entries of past and plural present forms of verbs (cf. (56)–(57)), the boolean value of INV is not needed in the lexical entry of most auxiliaries (cf. (58)), etc. Note also that such lexical entries describe words (given the presence of *word*-only attributes PHON and CAT), but they say nothing about values of the "structural" attributes GOV, REL, and NEXT, as these are set via grammatical constraints (on which more below).

Lexical entries such as (60)–(62) are not complete constraints, but rather parts of a single constraint on the lexicon:[34]

(63) Lexicon Constraint (AVM)

a. 'There are following words: *Ollie* (defined as in (60)), *girls* (as in (61)), *sheep* (as in (62)), . . . '

b. $word \rightarrow (\mathrm{LE}_{Ollie} \vee \mathrm{LE}_{girls} \vee \mathrm{LE}_{sheep} \vee \dots)$

As such an "unstructured list of lexical entries" view of the lexicon seems to be assumed in Gibson 2025, we also adopt it here. However, given that MTDG inherits the crucial mechanisms of HPSG, also a more structured – "hierarchical" – view of the lexicon sometimes assumed in HPSG may be formalized.[35]

The Logical versions of the above AVM statements are relatively straightforward. Formally, lexical AVM entries such as (60)–(62) are abbreviations of open formulae such as (64)–(66).

(64) $\mathrm{LE}_{Ollie}(x) \overset{df}{\equiv} x \ \textsc{phon} : \textit{Ollie} \wedge x \ \textsc{cat} : \textit{name} \wedge x \ \textsc{cat num} : \textit{sg}$

(65) $\mathrm{LE}_{girls}(x) \overset{df}{\equiv} x \ \textsc{phon} : \textit{girls} \wedge x \ \textsc{cat} : \textit{count} \wedge x \ \textsc{cat num} : \textit{pl}$

[32] These definitions assume the presence of appropriate subsorts of *wordform*, including *Ollie*, *girls*, and *sheep*.

[33] Technically, the possibility of *nil*-valued NUM must also be excluded in such cases, for example, by constraining it to infinitival verbs only:

(i) $\forall x.\, x \ \textsc{num} : \textit{nil} \leftrightarrow x \ \textsc{tense} : \textit{infinitive}$

In some other cases, *nil* should never be allowed, e.g.:

(ii) $\neg\exists x.\, x \ \textsc{tense} : \textit{nil}$

[34] In HPSG, an analogous constraint is known as the Word Principle (Höhle 1994, Richter 2024: 101).

[35] See, e.g., Davis and Koenig 2024 and references therein.

(66) $\text{LE}_{sheep}(x) \stackrel{df}{\equiv} x\ \textsc{phon} : sheep \wedge x\ \textsc{cat} : count$

These open formulae are parts of the Logical encoding of the Lexicon Constraint:

(67) Lexicon Constraint (Logical)

a. 'There are following words: *Ollie* (defined as in (64)), *girls* (as in (65)), *sheep* (as in (66)), . . . '

b. $\forall x.\, x : word \rightarrow (\text{LE}_{Ollie}(x) \vee \text{LE}_{girls}(x) \vee \text{LE}_{sheep}(x) \vee \dots)$

As mentioned above, most lexical entries do not include linearization information (perhaps with the exception of parts of some idioms or cranberry words) or information about their governors (with the exception of modifiers, to be discussed in §4.5). However, lexical entries are normally assumed to contain information about some of their dependents, i.e., about their arguments (often called "valents" in dependency grammars), as discussed in the next subsection.

4.4 Valency

Here are some examples of Simple valency information, roughly in the style of Gibson 2025 (with some extensions):

(68) $\text{V}_{\langle it \rangle}$: intransitive verbs with expletive subjects, e.g., *dawn*

(69) $\text{V}_{\langle N \rangle}$: intransitive verbs with nominal subjects, e.g., *sleep*

(70) $\text{V}_{\langle N,N \rangle}$: transitive verbs with nominal subjects, e.g., *chase*

(71) $\text{V}_{\langle N/Comp,N \rangle}$: transitive verbs with nominal or clausal subjects, e.g., *bother*

(72) $\text{V}_{\langle N,N,N \rangle}$: ditransitive verbs, e.g., *give (the dog a stick)*

(73) $\text{V}_{\langle N,N,prep(to) \rangle}$: ditransitive verbs, e.g., *give (a stick to the dog)*

(74) $\text{V}_{\langle N,Comp \rangle}$: verbs combining with CPs, e.g., *believe (that Allan liked the pizza)*

In the above lexical entries, only valency information is given, but complete lexical entries would also contain morphosyntactic information, e.g.:

(75) $\text{V}_{\langle it \rangle,sg,3,pres}$: *dawns*, *rains*

(76) $\text{V}_{\langle N,N \rangle,sg,3,pres}$: *chases*, *eats*

(77) $\text{V}_{\langle N/Comp,N \rangle,sg,3,pres}$: *bothers*, *surprises*

More precisely, such valencies may be encoded as values of a dedicated list-valued VALENCY attribute (abbreviated to VAL), as illustrated in the AVMs (78)–(80); full category information is provided only for the first of these, to reduce clutter.

(78) $\text{LE}_{dawns} \stackrel{df}{\equiv} \left[\begin{array}{l} \textsc{phon}\ dawns \\ \textsc{cat}\ \left[\begin{array}{l} lverb \\ \textsc{num}\ sg \\ \textsc{pers}\ 3 \\ \textsc{tense}\ pres \end{array}\right] \\ \textsc{val}\ \langle [\textsc{phon}\ it] \rangle \end{array}\right]$

(79) $\text{LE}_{chases} \stackrel{df}{\equiv} \left[\begin{array}{l} \textsc{phon}\ chases \\ \textsc{cat}\ lverb \\ \textsc{val}\ \langle [\textsc{cat}\ noun], [\textsc{cat}\ noun] \rangle \end{array}\right]$

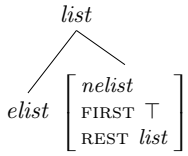
(80) $\text{LE}_{bothers} \stackrel{df}{\equiv} \left[\begin{array}{l} \textsc{phon}\ bothers \\ \textsc{cat}\ lverb \\ \textsc{val}\ \langle [\textsc{cat}\ noun] \vee \left[\begin{array}{l} \textsc{phon}\ that \\ \textsc{cat}\ comp \end{array}\right], [\textsc{cat}\ noun] \rangle \end{array}\right]$

The attribute VAL appearing in these lexical entries must be explicitly defined in the signature, so the definition of sort *word* in (37) needs to be extended as shown in (81) (or (82), to be explained below).

(81)
$$\begin{bmatrix} word \\ \text{PHON}\ wordform \\ \text{CAT}\ category \\ \text{VAL}\ list \\ \text{REL}\ relation \\ \text{GOV}\ word \\ \text{NEXT}\ word \end{bmatrix}$$

(82)
$$\begin{bmatrix} word \\ \text{PHON}\ wordform \\ \text{CAT}\ category \\ \text{VAL}\ list(word) \\ \text{REL}\ relation \\ \text{GOV}\ word \\ \text{NEXT}\ word \end{bmatrix}$$

Also, the signature needs to be extended by adding the sort *list* (an immediate subsort of $\top$) defined – as in HPSG – as follows:[36]

(83)


Thus, a list is either the empty list or a pair consisting of a list element and the rest of the list (i.e., a list, possibly empty). For example, an AVM visualization of $\text{LE}_{bothers}$ which does not employ the shorthand $\langle \ldots \rangle$ notation used in (80) would look as in (84).

(84)
$$\text{LE}_{bothers} \stackrel{df}{\equiv} \begin{bmatrix} \text{PHON}\ bothers \\ \text{CAT}\ lverb \\ \text{VAL} \begin{bmatrix} nelist \\ \text{FIRST} \quad [\text{CAT}\ noun] \vee \begin{bmatrix} \text{PHON}\ that \\ \text{CAT}\ comp \end{bmatrix} \\ \text{REST} \quad \begin{bmatrix} nelist \\ \text{FIRST} \quad [\text{CAT}\ noun] \\ \text{REST}\ elist \end{bmatrix} \end{bmatrix} \end{bmatrix}$$

As in the case of previous AVM lexical entries, those in (78)–(80) are AVM visualizations of open Logical formulae which are part of the Lexicon Constraint in (67). For example, such a formula corresponding to (80) (i.e., still omitting some morphosyntactic information) is given in (85).

(85)
$$\begin{aligned} \text{LE}_{bothers}(x) \stackrel{df}{\equiv}\ & x\ \text{PHON} : bothers \wedge x\ \text{CAT} : lverb \wedge \\ & \exists y.\, y = x\ \text{VAL FIRST} \wedge \\ & \quad (y\ \text{CAT} : noun \vee (y\ \text{CAT} : comp \wedge y\ \text{PHON} : that)) \wedge \\ & \exists z.\, z = x\ \text{VAL REST FIRST} \wedge z\ \text{CAT} : noun \wedge \\ & x\ \text{VAL REST REST} : elist \end{aligned}$$

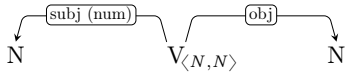
The valency list only contains *word* objects (and not, for example, *boolean* objects); this requirement may be somewhat informally expressed as in (82), by specifying values of VAL as *list*(*word*). At the Logical level, this requirement may be formalized as in (86)–(87).

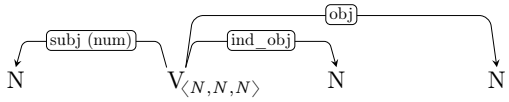
(86) a. 'Every value of VAL must be a list of words.'
b. $\forall x. \forall y.\, x\ \text{VAL} = y \rightarrow \texttt{list\_of\_words}(y)$

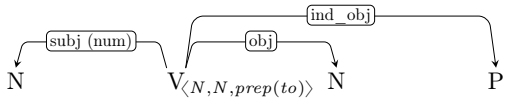
(87) a. 'A list of words is either the empty list or a non-empty list whose first element is a word and the rest of the list is a list of words.'
b. $\forall x. \texttt{list\_of\_words}(x) \leftrightarrow (x : elist \vee (x\ \text{FIRST} : word \wedge \texttt{list\_of\_words}(x\ \text{REST})))$

[36] Here, *elist* stands for "empty list" and *nelist* for "non-empty list".

Gibson (2025) assumes that information about grammatical functions of valents is provided via general "rules" such as (88)–(90), which also encode word order and agreement information.

(88) 

(89) 

(90) 

We do not assume such "rules" here. As discussed in §5, having many such rules misses a number of generalizations (e.g., "finite verbs agree with their subjects", "subjects are usually pre-verbal and other valents post-verbal"), also about the mapping between morpho-syntactically defined valencies and grammatical functions (e.g., "first valents of verbs are subjects", "second valents are objects, if they are nominal and not followed by another nominal valent"). Such generalizations may be expressed in MTDG directly, for example, in the following AVM and Logical statements:

(91) First Valents of Verbs are Subjects

a. 'The first element on a verb's valency list is its subject.'

b. $\begin{bmatrix} \textsc{cat}\ \textit{verb} \\ \textsc{val}\ \langle \boxed{1}, \ldots \rangle \end{bmatrix} \rightarrow \boxed{1}\big[\,\textsc{rel}\ \textit{subj}\,\big]$

c. $\forall x.\forall y.\,(x\ \textsc{cat} : \textit{verb} \wedge x\ \textsc{val}\ \textsc{first} = y) \rightarrow y\ \textsc{rel} : \textit{subj}$

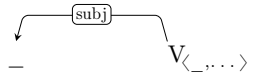
(92) Second Valents of Verbs are Normally Direct Objects[37]

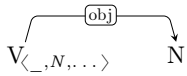
a. 'The second element on a verb's valency list is its direct object, if it is nominal and not followed by another nominal valent.'

b. $\begin{bmatrix} \textsc{cat}\ \textit{verb} \\ \textsc{val}\ \langle \boxed{1}, \boxed{2}\big[\,\textsc{cat}\ \textit{noun}\,\big] \rangle \oplus \boxed{3} \end{bmatrix} \wedge \neg\boxed{3}\langle\big[\,\textsc{cat}\ \textit{noun}\,\big], \ldots\rangle \rightarrow \boxed{2}\big[\,\textsc{rel}\ \textit{obj}\,\big]$

c. $\forall x.\forall y.\,(x\ \textsc{cat} : \textit{verb} \wedge y\ \textsc{cat} : \textit{noun} \wedge x\ \textsc{val}\ \textsc{rest}\ \textsc{first} = y \wedge \neg(x\ \textsc{val}\ \textsc{rest}\ \textsc{rest}\ \textsc{first}\ \textsc{cat} : \textit{noun})) \rightarrow y\ \textsc{rel} : \textit{obj}$

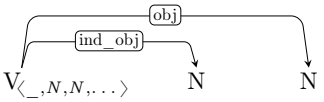
In the Simple notation, such mappings could perhaps be expressed as in (93)–(94), with directionality of arcs again *not* indicating any hard word order constraints.

(93) 

(94) 

The mapping in (93) should be understood as saying that first valents of verbs are their subjects, while the mapping in (94) that second valents of verbs, if they are nominal, are their objects. This second mapping is less precise, as it does not mention the exception of ditransitive verbs, but a Pāṇinian convention could be introduced that more specific mappings block the less specific ones, in which case (94) would be blocked by (95).

[37] In (92c), $\oplus$ indicates list concatenation, i.e., $\boxed{3}$ is a list. The second conjunct in the antecedent specifies that this list does not start with a noun, i.e., that there is no nominal third element of the verb's valency; this is compatible with $\boxed{3}$ being the empty list, i.e., with the verb having exactly two valents, as in (88). This additional condition on the potential third valent is needed because of ditransitive verbs, as in (89), where the second valent is an indirect object, rather than a direct object.

(95) 

This approach to valency and dependency labels is sufficient for our purposes, i.e., for presenting a model-theoretic dependency grammar with roughly the same empirical coverage as Gibson 2025: ch.3. However, in some traditions, morphosyntactic valency information does not determine dependency labels. One such theoretical tradition is LFG, where grammatical functions are primitive notions, claimed not be fully determined by morphosyntactic properties (but see the discussion in Patejuk and Przepiórkowski 2016 and Kaplan 2017). Also, in descriptions of some languages, direct objects are defined with reference to passivization: only those second valents of active verbs which correspond to the subjects of their passive counterparts are called direct objects. Applying this criterion to English, the non-subject nominal valent of the verb *receive* is its direct object (cf. *This book was received by John*), while that of *have* is not (cf. **This book was had by John*). If so, the morphosyntax of the argument and its governor does not determine the grammatical function of that argument. Moreover, on some analyses, some finite verbs do not have a subject among its valents (see, e.g., Babby 2009: §1.3 on some subjectless verbs in Russian). In all such traditions and analyses, instead of assuming mappings such as (88)–(92), it makes sense to specify grammatical functions of valents directly in the lexical entries of their governors, as in the AVM entries in (96)–(97).[38]

(96) $\text{LE}_{receive} \stackrel{df}{\equiv} \left[\begin{array}{l} \textsc{phon}\ \textit{receive} \\ \textsc{cat}\ \textit{lverb} \\ \textsc{val}\ \langle \left[\begin{array}{l}\textsc{cat}\ \textit{noun}\\ \textsc{rel}\ \textit{subj}\end{array}\right], \left[\begin{array}{l}\textsc{cat}\ \textit{noun}\\ \textsc{rel}\ \textit{obj}\end{array}\right] \rangle \end{array}\right]$

(97) $\text{LE}_{have} \stackrel{df}{\equiv} \left[\begin{array}{l} \textsc{phon}\ \textit{have} \\ \textsc{cat}\ \textit{lverb} \\ \textsc{val}\ \langle \left[\begin{array}{l}\textsc{cat}\ \textit{noun}\\ \textsc{rel}\ \textit{subj}\end{array}\right], \left[\begin{array}{l}\textsc{cat}\ \textit{noun}\\ \textsc{rel}\ \textit{obl}\end{array}\right] \rangle \end{array}\right]$

While it is not fully clear whether lexical entries should mention grammatical functions of their valents or not, it is clear that lexical entries of particular words do not need to specify that these words are governors of their valents, as this may be expressed by the single constraint given as AVM and Logical statements in (98), with the `member` relation defined in the standard way in (99).

(98) Valency and Governor[39]

a. 'For any word, every member of this word's valency list has this word as its governor.'

b. $\boxed{1}[\textsc{val}\ \boxed{2}] \rightarrow \boxed{2}\langle[\textsc{gov}\ \boxed{1}], \ldots, [\textsc{gov}\ \boxed{1}]\rangle$

c. $\forall x.\forall y.\, \texttt{member}(y, x\ \textsc{val}) \rightarrow y\ \textsc{gov} = x$

(99) a. 'Some entity is a member of a list iff either it is this list's first element or it is a member of the rest of this list.'

b. $\forall x.\forall y.\, \texttt{member}(y, x) \leftrightarrow (y = x\ \textsc{first} \vee \texttt{member}(y, x\ \textsc{rest}))$

Let us finally note that, while the idea of VALENCY lists is borrowed from HPSG, no specialized Valency Principles of the kind assumed in HPSG (e.g., Pollard and Sag 1994: 348) are needed to make sure that VALENCY elements are actually realized. This already follows from the earlier principles, especially, from the three dependency tree axioms

[38] In (97), *obl* stands for oblique, for the lack of a better dependency label.

[39] As (98) is a technical principle, relating VAL and GOV, it has no Simple encoding. (98c) implies that only words are present on VAL lists (because only *word* has the GOV attribute), so it makes the "list of words" requirement in (86)–(87) redundant. The unidirectionality of the implication in this principle is explained in §4.5 immediately below.

in (28)–(30) (Single Governor Constraint, Root Constraint, and Connectivity Constraint), which state that all *word* objects in the model must form a dependency tree, and from the Total Word Order Constraint (TWOC) in (25), which says that all *word* objects must be linearly ordered. As VALENCY elements are of sort *word*, they must satisfy all these constraints, i.e., they must be nodes in the dependency tree.

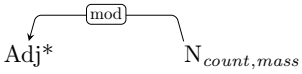
4.5 Modifiers

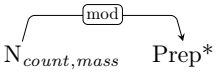
Given the standard valent–modifier distinction (more frequently called the "argument–adjunct dichotomy"), (98) must contain a one-way implication, as modifiers satisfy its consequent (they have a governor), but not its antecedent (they are not on this governor's VAL list). For example, after adding VAL to the list of attributes of *word*, the running example *Marge still loves him* will now have the AVM representation in (100), replacing (40) (with *nil*-valued attributes omitted again).

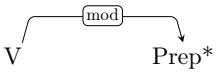
(100)
$$\boxed{1}\begin{bmatrix} word \\ \text{PHON } Marge \\ \text{CAT } \begin{bmatrix} name \\ \text{NUM } sg \end{bmatrix} \\ \text{REL } subj \\ \text{GOV } \boxed{3} \\ \text{NEXT } \boxed{2} \end{bmatrix} \quad \boxed{2}\begin{bmatrix} word \\ \text{PHON } still \\ \text{CAT } adverb \\ \text{REL } mod \\ \text{GOV } \boxed{3} \\ \text{NEXT } \boxed{3} \end{bmatrix} \quad \boxed{3}\begin{bmatrix} word \\ \text{PHON } loves \\ \text{CAT } \begin{bmatrix} lverb \\ \text{NUM } sg \\ \text{PERS } 3 \\ \text{TENSE } pres \end{bmatrix} \\ \text{VAL } \langle \boxed{1}, \boxed{4} \rangle \\ \text{NEXT } \boxed{4} \end{bmatrix} \quad \boxed{4}\begin{bmatrix} word \\ \text{PHON } him \\ \text{CAT } \begin{bmatrix} pronoun \\ \text{NUM } sg \end{bmatrix} \\ \text{REL } obj \\ \text{GOV } \boxed{3} \end{bmatrix}$$

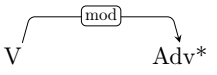
Note that *Marge*, *still*, and *him* all specify *loves* as the governor, but only *Marge* and *him* are on *loves*' valency list.

Also the source of this GOV value is different for valents and modifiers. As specified in (98), the value of GOV of valents is a direct reflex of their presence on the governor's VAL list, i.e., it reflects the governor's lexical entry. In contrast, modifiers may be – to the first approximation – introduced at the Simple level of description by more general rules such as (101)–(104) (based on those in Gibson 2025: 67–68).

(101) 

(102) 

(103) 

(104) 

For example, (101) is meant to be saying that adjectives are (preceding) modifiers of nouns, and similarly for (102)–(104).

As discussed in §5 below, such rules are imprecise or misleading in a number of ways, so AVM constraints such as following – and corresponding Logical formulae – should be preferred in more precise discourse:

(105) a. 'Adjectival modifiers modify common nouns.'

b. $\begin{bmatrix} \text{CAT } adj \\ \text{REL } mod \end{bmatrix} \rightarrow \left[\text{GOV CAT } common \right]$

c. $\forall x. (x \text{ CAT} : adj \wedge x \text{ REL} : mod) \rightarrow x \text{ GOV CAT} : common$

(106) a. 'Prepositional modifiers modify common nouns and verbs.'

b. $\begin{bmatrix} \text{CAT } prep \\ \text{REL } mod \end{bmatrix} \rightarrow \left[\text{GOV CAT } common \vee verb \right]$

c. $\forall x.\,(x\ \textsc{cat} : \mathit{prep} \wedge x\ \textsc{rel} : \mathit{mod}) \rightarrow$
$(x\ \textsc{gov cat} : \mathit{common} \vee x\ \textsc{gov cat} : \mathit{verb})$

(107) a. 'Adverbs modify verbs.'[40]

b. $\begin{bmatrix} \textsc{cat}\ \mathit{adv} \\ \textsc{rel}\ \mathit{mod} \end{bmatrix} \rightarrow \left[\textsc{gov cat}\ \mathit{verb}\right]$

c. $\forall x.\,(x\ \textsc{cat} : \mathit{adv} \wedge x\ \textsc{rel} : \mathit{mod}) \rightarrow x\ \textsc{gov cat} : \mathit{verb}$

What constraints stated this way make clear, unlike the Simple statements in (101)–(104), is that adjectives, prepositions, and adverbs do not need to be modifiers – in other constructions they may be valents. For example, adjectives may be arguments of copulas (to be discussed in §4.9 below), prepositions may head indirect objects (cf. (73) and (90) above), and adverbs may be arguments of verbs such as *treat* and *word* (e.g., *He treated me badly*, *She worded this so perfectly*). Also, the statements in (101)–(104) – but not those in (105)–(107) – simultaneously encode word order in a way that seems roughly right in the case of modifiers of nouns, but not in the case of verbs, whose modifiers do not have to follow them (e.g., *Over the holidays, we frequently visited our grandparents*)

Again, this approach to modification is sufficient for our purposes, as its level of linguistic detail corresponds to that of Gibson 2025: ch.3. However, a more detailed approach might rely on the particular modifiers lexically selecting for their governors, as assumed in some approaches (e.g., Pollard and Sag 1994, Bruening 2010). For example, while it is generally assumed that adverbs do not modify nouns, some of them apparently do, e.g., *the now king* or *the once Prince of Wales* (see Przepiórkowski and Patejuk 2025 for numerous attested examples). If so, instead of general statements such as those above, modifiers might individually specify their selectional restrictions. On this approach, while most words will not specify their governors lexically, modifiers will have lexical entries such as the following Simple and AVM versions of the lexical entry for the adverbs *quickly* and *once*:

(108) a. Adv_V: *quickly*

b. $\text{LE}_{quickly} \stackrel{df}{\equiv} \begin{bmatrix} \textsc{phon}\ \mathit{quickly} \\ \textsc{cat}\ \mathit{adverb} \\ \textsc{val}\ \mathit{nil} \\ \textsc{rel}\ \mathit{mod} \\ \textsc{gov cat}\ \mathit{verb} \end{bmatrix}$

(109) a. $\text{Adv}_{V/N}$: *once*

b. $\text{LE}_{once} \stackrel{df}{\equiv} \begin{bmatrix} \textsc{phon}\ \mathit{once} \\ \textsc{cat}\ \mathit{adverb} \\ \textsc{val}\ \mathit{nil} \\ \textsc{rel}\ \mathit{mod} \\ \textsc{gov cat}\ \mathit{verb} \vee \mathit{noun} \end{bmatrix}$

Whatever the level of detail and specific approach, one more – framework-level – constraint is needed that encodes the difference between valents and modifiers: only the former appear on valency lists and only the latter do not:

(110) Valents

a. 'All – and only – valents appear on VALENCY lists.'

b. $\forall x.\, x\ \textsc{rel} : \mathit{valent} \leftrightarrow \exists y.\, \texttt{member}(x, y\ \textsc{val})$

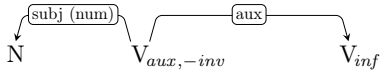
The above constraint assumes the *relation* sort hierarchy in (24), according to which dependents may be split into valents (including subjects and objects) and modifiers. It will be extended with other valents in the following subsection (in (116) below).[41]

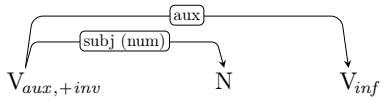
[40] This should be extended to adverbs modifying other parts of speech, especially adjectives.

[41] An interesting third approach, but not considered here as this would take us too far afield, is to negate the argument–adjunct dichotomy and treat all dependents alike, as sometimes done in HPSG (Bouma et al. 2001, Przepiórkowski 1999a,b), and also proposed within LFG (Przepiórkowski 2016).

4.6 Word order

As discussed in §5, word order information is specified in Gibson 2025: ch.3 via simplified "rules" such as those in (88)–(90) and (101)–(104) above or (111)–(112) below.

(111) 

(112) 

This may be understood as a simplified description of word order, but it is one that clearly misses generalizations. For example, out of 17 "rules" in Gibson 2025: ch.3, 10 mention subjects: in 8 of them the subject is pre-verbal, while in 2 – exactly the "rules" rooted in a $+inv$ verb (as in (112)) – it is post-verbal. For this reason, we do not assume such "rules" here.

This generalization may be expressed succinctly at the more formalized levels of description:

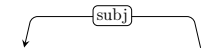
(113) Linearization of Subjects

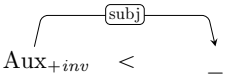
a. 'Subjects precede their governors, with the exception of $+inv$ governors.'

b. $\begin{bmatrix} \text{REL } \textit{subj} \\ \text{GOV } \boxed{1} \end{bmatrix} < \boxed{1}$ unless $\boxed{1}\big[\,\text{CAT INV } +\,\big]$

c. $\forall x.\forall y.\,(x \text{ REL} : \textit{subj} \wedge x \text{ GOV} = y) \rightarrow (\texttt{precede}(y, x) \leftrightarrow y \text{ CAT INV} : +)$

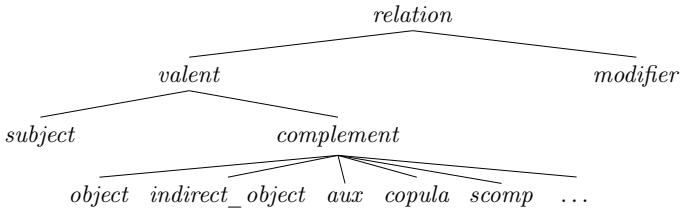
As for the Simple level of description, we propose to capture verb–subject linearization with the following two word order rules, explicitly indicating linear order with $<$:

(114) 

(115) 

The first rule, (114), says that subjects generally precede their governors, whatever the categories of the subjects and the governors, while the second rule, (115), overrides it in a Pāṇinian way, by saying that subjects follow $+inv$ auxiliary verbs, whatever the category of the subjects.

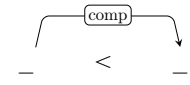
Stating other word order constraints, encoded in particular "rules" in Gibson 2025: ch.3, is equally easy. For this, we need to extend the *relation* part of the signature (24) to other kinds of valents assumed in such "rules", apart from *subject* and *object* (cf. Hudson 2024: 1546, 1565 and Hudson et al. 2026: 369):

(116) 

With this extension, the following constraint covers the generalization – distributed over 12 out of 17 rules in Gibson 2025: ch.3 – that complements follow their governors:

(117) Linearization of Complements

a. 'Complements follow their governors.'

b. 

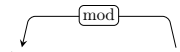
c. $\boxed{1} < \begin{bmatrix} \text{REL} & \textit{comp} \\ \text{GOV} & \boxed{1} \end{bmatrix}$

d. $\forall x.\forall y.\,(x\ \text{REL} : \textit{comp} \wedge x\ \text{GOV} = y) \rightarrow \texttt{precede}(y, x)$

As noted above, linearization constraints involving modifiers are more complex, and they seem to depend on semantics and information structure, but at least in the case of adjectives modifying nouns the general constraint is simple:

(118) Linearization of Adjectival Modifiers

a. 'Adjectival modifiers precede their governors.'

b. 

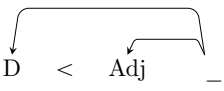
c. $\begin{bmatrix} \text{CAT} & \textit{adjective} \\ \text{REL} & \textit{mod} \\ \text{GOV} & \boxed{1} \end{bmatrix} < \boxed{1}$

d. $\forall x.\forall y.\,(x\ \text{REL} : \textit{mod} \wedge x\ \text{CAT} : \textit{adjective}) \wedge x\ \text{GOV} = y) \rightarrow \texttt{precede}(x, y)$

Moreover, adjectives follow determiners (if any):

(119) Linearization of Determiners (wrt Adjectives)

a. 'Determiners precede their co-dependent adjectives.'

b. 

c. $\begin{bmatrix} \text{CAT} & \textit{determiner} \\ \text{GOV} & \boxed{1} \end{bmatrix} < \begin{bmatrix} \text{CAT} & \textit{adjective} \\ \text{GOV} & \boxed{1} \end{bmatrix}$

d. $\forall x.\forall y.\,(x\ \text{CAT} : \textit{determiner} \wedge y\ \text{CAT} : \textit{adjective} \wedge x\ \text{GOV} = y\ \text{GOV}) \rightarrow$
$\texttt{precede}(x, y)$

These linearization constraints seem not to cover the simple case of determiners preceding nouns (even in the absence of adjectives). However, we assume for simplicity that, as in standard HPSG (Pollard and Sag 1994), nouns select for determiners (cf. Van Eynde 2024) and, apparently unlike in any previous literature, that determiners bear the dependency label *subject*. One technical advantage of this non-standard proposal is that the Linearization of Subjects constraint (113) automatically handles the pre-nominal position of determiners. A theoretical advantage is that an arbitrary decision is avoided regarding the exact place where *subject* is replaced with another dependency label in the following sequence of examples:

(120) a. *Krusty arrived in Springfield; it was unexpected.*
b. *Krusty arriving in Springfield was unexpected.*
c. *Krusty's arriving in Springfield was unexpected.*
d. *Krusty's arrival in Springfield was unexpected.*
e. *As for Krusty, his arrival in Springfield was unexpected.*
f. *As for Krusty, the arrival in Springfield was unexpected.*

Krusty is the subject in (120a), but does it stop being the subject already in (120b), or only somewhere down this list? In the absence of strong arguments to the contrary, a dramatic change of label is avoided, if determiners are assumed to be nouns' subjects.[42]

In summary, the model-theoretic approach to dependency grammar makes it possible to state certain simple generalizations that would be missed in other formalizations.

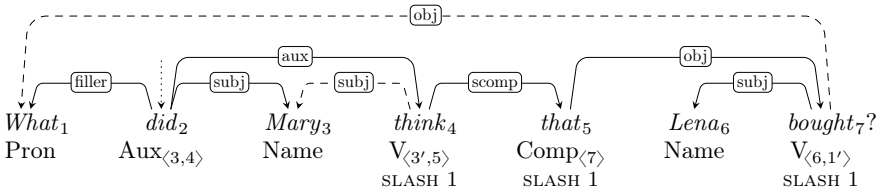
4.7 Long-Distance Dependencies

The analysis of long-distance dependencies (LDDs; sometimes called Unbounded Dependency Constructions, UDCs; including *wh*-questions, relative clauses, and topicalization) is only very roughly sketched in Gibson 2025: ch.3, but with a clear reference to the SLASH mechanism known from GPSG and HPSG. As with other phenomena, it is not our aim to provide a comprehensive analysis of unbounded dependencies, only to indicate how this phenomenon could be approached in MTDG.[43]

One of the LDD examples given in Gibson 2025: 96, ex. (96), is (121) below, with a structure suggested there similar to that in (122).

(121) *What did Mary think that Lena bought?*

(122)



The numbers in this Simple graph refer to particular words. For example, the valency list of the second word, *did*, contains information about word 3, *Mary*, and about word 4, *think*.

One thing to note in (122) is the presence of the dashed *obj* arc from *bought* to *what*, which "is not a dependency arc of the same type as the others" (Gibson 2025: 96), and similarly for the dashed *subj* arc from *think* to *Mary*. Such arcs indicate the original grammatical functions of *what* and *Mary*, but not the actual "functions" these words have at the point they are overtly realized, i.e., as the filler of *did* and the (raised) subject of *did*, respectively. Without these dashed arcs, the graph in (122) is still a projective dependency tree.

We follow Gibson 2025 (and GPSG and HPSG) in assuming the SLASH feature, appropriate for *word*s.[44] For most words, the value of SLASH is *nil*, but for words introducing the dependency and passing it up it is *word*.[45] In (122), three nodes in the tree have a non-*nil* SLASH: *bought*, where one of the arguments, indicated as $1'$, corresponds to the SLASH value indicated as 1 (more on this "correspondence" below), as well as *that* and *think*; these three nodes pass the SLASH value up. At the top of this unbounded dependency, *did* "cancels" it by realizing the SLASH value brought by one of its valents, *think*, as its filler – the word *what*.

Now, in the tree-based dependency framework assumed here, the second valency member of *bought*, $1'$, cannot be the same *word* as the filler *what*, 1, as then this *word* would have two governors: the local governor via *filler* and the non-local governor via *obj*. The resulting graph would not be a tree. For this reason, we distinguish between the direct object selected

[42] As noted by Dick Hudson (p.c.), on this approach *his* will be the subject in *his execution*, also when it is understood as the patient in the execution event. We do not find this problematic, as patients may be subjects anyway, namely, in passive constructions.

[43] For dependency descriptions of LDDs which are more comprehensive than that in Gibson 2025: ch.3, but also in wont of greater precision (and any formalization) – see, e.g., Hudson 1984: 124–130 and Groß and Osborne 2009.

[44] One advantage of this approach over that of, say, Groß and Osborne 2009, where such a feature sharing mechanism is not explicitly assumed, is that it makes it easier to account for languages which exhibit extraction-sensitive phenomena, as argued in Bouma et al. 2001.

[45] Unlike in HPSG, SLASH values are limited to at most single words here, so the preliminary formalization proposed in this paper does not cover overlapping unbounded dependencies (see, e.g., Pollard and Sag 1994: 159), not discussed in Gibson 2025: ch.3 either.

by *bought* and the word actually realized as the filler: they are related, but different elements in the model.

How are they related? The filler should satisfy the selectional restrictions of the original governor, so at least their CAT (for syntactic selections) and PHON (for lexical selections) values should be equal.[46] We also assume the identity of VAL values, as on some accounts governors may select for valents with specific valencies.[47] However, what is not expected to be equal is REL (for 1′ in (122) it is *obj*, for 1 it is *filler*) or GOV (for 1′ it is the word *bought*, for 1 it is *did*). Moreover, the value of NEXT is bound to differ, as it is the word *did* in the case of the locally realized word 1, *what*, while this attribute is irrelevant (and, hence, *nil*-valued) in the case of 1′, which is not locally realized at all (and, as such, is irrelevant to word order constraints). Hence, the desired structures of the key words in the running example (121)–(122) are as in (123) (with only partial values of CAT here and with the new attribute REAL – standing for "locally realized" – to be introduced presently).

(123)

$$\boxed{1}\begin{bmatrix} \textit{word} \\ \text{PHON } \boxed{1a}\textit{what} \\ \text{CAT } \boxed{1b}\textit{pron} \\ \text{VAL } \boxed{1c}\textit{nil} \\ \text{REL } \textit{filler} \\ \text{GOV } \boxed{2} \\ \text{REAL } + \\ \text{SLASH } \textit{nil} \\ \text{NEXT } \boxed{2} \end{bmatrix} \boxed{2}\begin{bmatrix} \textit{word} \\ \text{PHON } \textit{did} \\ \text{CAT } \textit{aux} \\ \text{VAL } \langle \boxed{3}, \boxed{4} \rangle \\ \text{REL } \textit{nil} \\ \text{GOV } \textit{nil} \\ \text{REAL } + \\ \text{SLASH } \textit{nil} \\ \text{NEXT } \boxed{3} \end{bmatrix} \ldots \boxed{4}\begin{bmatrix} \textit{word} \\ \text{PHON } \textit{think} \\ \text{CAT } \textit{lverb} \\ \text{VAL } \langle \boxed{3'}, \boxed{5} \rangle \\ \text{REL } \textit{aux} \\ \text{GOV } \boxed{2} \\ \text{REAL } + \\ \text{SLASH } \boxed{1} \\ \text{NEXT } \boxed{5} \end{bmatrix} \ldots \boxed{7}\begin{bmatrix} \textit{word} \\ \text{PHON } \textit{bought} \\ \text{CAT } \textit{lverb} \\ \text{VAL } \langle \boxed{6}, \boxed{1'} \begin{bmatrix} \textit{word} \\ \text{PHON } \boxed{1a} \\ \text{CAT } \boxed{1b} \\ \text{VAL } \boxed{1c} \\ \text{REL } \textit{obj} \\ \text{GOV } \boxed{7} \\ \text{REAL } \textit{nil} \\ \text{SLASH } \boxed{1} \\ \text{NEXT } \textit{nil} \end{bmatrix} \rangle \\ \text{REL } \textit{obj} \\ \text{GOV } \boxed{4} \\ \text{REAL } + \\ \text{SLASH } \boxed{1} \\ \text{NEXT } \textit{nil} \end{bmatrix}$$

Formally, the "correspondence" between a locally unrealized word (e.g., $\boxed{1'}$ in (123)) and its related copy ($\boxed{1}$ in (123)) is encoded as the `copy_of_unrealized_word` relation, abbreviated to `couw` and defined in (124).

(124) a. 'A word w is a copy of a locally unrealized word w' iff they share the form, the category, and the valency.'

b. $\forall x.\forall y.\, \texttt{couw}(x, y) \leftrightarrow$
$(y \text{ REAL} : \textit{nil} \wedge y \text{ NEXT} : \textit{nil} \wedge$
$x \text{ PHON} = y \text{ PHON} \wedge x \text{ CAT} = y \text{ CAT} \wedge x \text{ VAL} = y \text{ VAL})$

Once such a distinction is made between locally realized (i.e., REAL : +) words, which are subject to linearization and tree constraints, and locally unrealized (i.e., REAL : *nil*) words, which are not, it is necessary to restate all these constraints by changing all mentions of words to mentions of locally realized words. For example, the Total Word Order Constraint of §3.2, repeated below as (125), should be modified as in (126).

(125) Total Word Order Constraint (TWOC, original)

a. 'For any two distinct words, one precedes the other.'

b. $\forall x.\forall y.\, (x : \textit{word} \wedge y : \textit{word} \wedge x \neq y) \rightarrow$
$(\texttt{precede}(x, y) \veebar \texttt{precede}(y, x))$

[46] We ignore here the so-called "movement paradoxes" (see, e.g., Bresnan et al. 2016: ch.2), noting only that they seem to be inherently more tractable in non-transformational approaches, like the current one, than in approaches relying on movement.

[47] For example, in various languages, prepositions of the same form may select for different grammatical cases, e.g., German *auf* 'on' may combine with accusative or dative, with different meanings. However, the verb *warten* 'wait' combines only with *auf* plus accusative, and one way of stating this selectional restriction is by requiring that the appropriate valent of *warten* have CAT : *preposition*, PHON : *auf*, and VAL = ⟨[CASE *acc*]⟩.

(126) Total Word Order Constraint (TWOC, modified)

a. 'For any two distinct locally realized words, one precedes the other.'

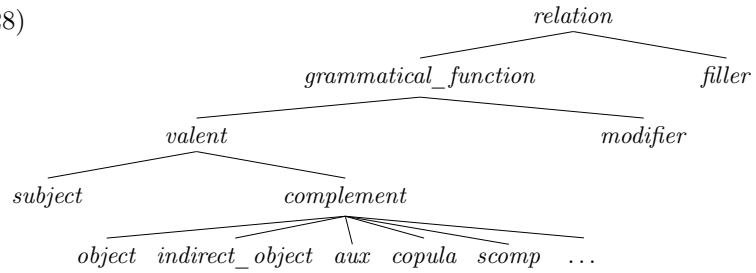
b. $\forall x.\forall y.\,(x \text{ REAL} : + \;\wedge\; y \text{ REAL} : + \;\wedge\; x \neq y) \rightarrow$
$(\texttt{precede}(x, y) \;\veebar\; \texttt{precede}(y, x))$

We leave other modifications of this kind as a tedious but easy exercise.

Given the new attributes SLASH and REAL, and the new kind of dependency label, *filler*, the signature should be modified as indicated in (127), which extends the specification of the sort *word*, and as in (128), which extends the sort *relation* by distinguishing *grammatical function*s from *filler*s.

(127)
$$\begin{bmatrix} \textit{word} \\ \text{PHON } \textit{wordform} \\ \text{CAT } \textit{category} \\ \text{VAL } \textit{list} \\ \text{REL } \textit{relation} \\ \text{GOV } \textit{word} \\ \text{REAL } \textit{boolean} \\ \text{SLASH } \textit{word} \\ \text{NEXT } \textit{word} \end{bmatrix}$$

(128)

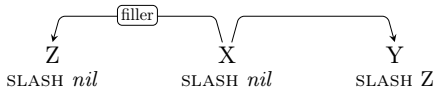


With these preliminaries out of the way, we can provide the following Simple four-part constraint on LDDs:[48]

(129) Long-Distance Dependencies (Simple)

a. 

(no LDD)

b. 

(LDD top)

c. 

(LDD mid)

d. 

(LDD bot)

Each word – whether locally realized or not – should satisfy one of the above four descriptions. In the simplest case, (129a), the word X is not a part of a LDD: the SLASH value of this word and all its dependents is *nil*. At the of top of a LDD, (129b), one of the dependents of X is the filler and another one (perhaps among more) has this filler as its SLASH value. The word with such two dependents has *nil*-valued SLASH itself. At the middle of

[48] As in previous constraints at this level, the direction of arrows does not necessarily indicate the actual word order.

the LDD, (129c), the (non-*nil*) SLASH value is shared by the word X and one of its dependents. Finally, at the bottom of the LDD, (129d), a locally unrealized (as indicated by the apostrophe ′ in X′) word – a valent or a modifier of some locally realized word – introduces a copy of itself, X, as its SLASH value. This copy will eventually be realized as a filler, via the previous two parts of this constraint, (129b–c).

A somewhat more precise AVM statement of this constraint, but abusing the usual AVM notation a little, is given in (130).

(130) Long-Distance Dependencies (AVM)

word →

$$(\boxed{1}[\textsc{slash}\ \mathit{nil}] \wedge \forall \boxed{2}[\textsc{gov}\ \boxed{1}] \rightarrow \boxed{2}[\textsc{slash}\ \mathit{nil}]) \vee \qquad \text{(no LDD)}$$

$$(\boxed{1}[\textsc{slash}\ \mathit{nil}] \wedge \exists \boxed{2}\begin{bmatrix}\textsc{gov}\ \boxed{1}\\ \textsc{slash}\ \boxed{3}\end{bmatrix} \wedge \boxed{3}\begin{bmatrix}\textsc{rel}\ \mathit{filler}\\ \textsc{gov}\ \boxed{1}\\ \textsc{slash}\ \mathit{nil}\end{bmatrix}) \vee \qquad \text{(LDD top)}$$

$$(\boxed{1}[\textsc{slash}\ \boxed{3}] \wedge \exists \boxed{2}\begin{bmatrix}\textsc{gov}\ \boxed{1}\\ \textsc{slash}\ \boxed{3}\mathit{word}\end{bmatrix}) \vee \qquad \text{(LDD mid)}$$

$$\boxed{1'}[\textsc{slash}\ \boxed{1}] \qquad \text{(LDD bot)}$$

It should be understood as saying that, for any *word* object, either of the four disjuncts must be true, i.e., the object must satisfy the description at the beginning of the disjunct (e.g., [SLASH *nil*] in the first two disjuncts), with additional quantificational conditions within these disjuncts also satisfied. For example, the third – LDD mid – disjunct states that the word must have some non-*nil* SLASH value shared with some dependent of this word.

Finally, the Logical statement is fully explicit:

(131) Long-Distance Dependencies (Logical)

a. 'Each word is either 1) outside of LDDs, or 2) at the top of a LDD, or 3) in the middle of a LDD, or 4) at the bottom of a LDD.'

b. $\forall x.\, x : \mathit{word} \rightarrow (\texttt{no\_ldd}(x) \vee \texttt{ldd\_top}(x) \vee \texttt{ldd\_mid}(x) \vee \texttt{ldd\_bot}(x))$

(132) a. 'A word is outside of LDD iff all its dependents (if any) and the word itself have no slash.'

b. $\forall x.\, \texttt{no\_ldd}(x) \leftrightarrow (x\ \textsc{slash} : \mathit{nil} \wedge \forall y.\, y\ \textsc{gov} = x \rightarrow y\ \textsc{slash} : \mathit{nil})$

(133) a. 'A word is at the top of a LDD iff it has no slash and its no-slash filler dependent is equal to the slash of one of its other dependents.'

b. $\forall x.\, \texttt{ldd\_top}(x) \leftrightarrow (x\ \textsc{slash} : \mathit{nil} \wedge \exists y. \exists z.\, y\ \textsc{gov} = x \wedge y\ \textsc{rel} : \mathit{filler} \wedge y\ \textsc{slash} : \mathit{nil} \wedge z\ \textsc{gov} = x \wedge z\ \textsc{slash} = y)$

(134) a. 'A word is on a LDD path iff its slash is equal to that of one of its dependents.'

b. $\forall x.\, \texttt{ldd\_mid}(x) \leftrightarrow \exists y.\, y\ \textsc{gov} = x \wedge y\ \textsc{slash} : \mathit{word} \wedge x\ \textsc{slash} = y\ \textsc{slash}$

(135) a. 'A word is at the bottom of a LDD iff it is not locally realized and introduces its copy as slash.'

b. $\forall x.\, \texttt{ldd\_bot}(x) \leftrightarrow \texttt{couw}(x\ \textsc{slash}, x)$

Two additional constraints are needed to make this analysis waterproof. First, for any word, at most one of its dependents may have slash; otherwise, the LDD mid property (134) would make sure that one of these slashes would be shared with the governor and eventually realized as a filler, but the other one would at this stage simply be ignored. As a result, sentences such as * *What did Mary think that bought?* (cf. *What did Mary think that who*

bought?) and * *Who did Mary think that bought?* (cf. *Who did Mary think that bought what?*) would be predicted to be grammatical.

(136) At Most One SLASH

a. 'For any word, at most one of its dependents has slash.'

b. $\forall x.\forall y.\forall z.\,(y \text{ GOV} = x \wedge y \text{ SLASH} : \mathit{word} \wedge z \text{ GOV} = x \wedge z \text{ SLASH} : \mathit{word}) \rightarrow y = z$

Another constraint makes sure that slashes eventually get realized as fillers, rather than being left unrealized at the root level:

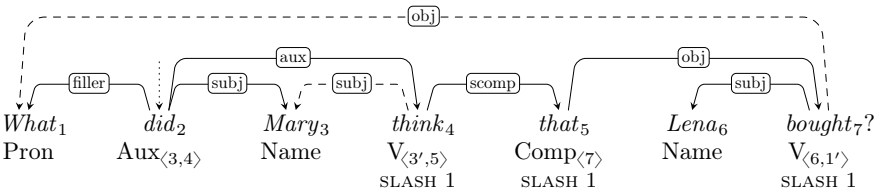
(137) Root Has No SLASH

a. 'The root has no slash.'

b. $\forall x.\,\texttt{root}(x) \rightarrow x \text{ SLASH} : \mathit{nil}$

To finish this subsection, let us return to the running example (121), whose simplified structure (122) is repeated below as (138).

(138)



There are nine different *word* objects in (138): the seven locally realized, which are linearly ordered and constitute the tree, as well as two that exist only on valency lists: the object of *bought* marked in the valency of *bought* as $1'$, but also the subject of *think*, $3'$, realized only as the raised subject of *did*, $Mary_3$ (more on this in the following subsection). All of these nine words satisfy the LDD constraint in (131). The locally unrealized $1'$ introduces SLASH and has the `ldd_bot` property defined in (135). Three words, $bought_7$, $that_5$, $think_4$, collect this SLASH value from one of their dependents and make it their own, satisfying `ldd_mid` in (134). Another, did_2, identifies this SLASH value of one of its dependents as its own filler, satisfying `ldd_top` in (133). Finally, all the other words – $what_1$, $Mary_3$, the related locally unrealized $3'$, and $Lena_6$ – satisfy `no_ldd` in (132): their SLASH is *nil* and they do not have any non-*nil* SLASH dependents (as they have no dependents at all).

4.8 Raising

Gibson 2025 does not discuss raising or control, but at least a preliminary analysis of raising is needed to fully understand the structure of the LDD graph in (138). We adopt the common assumption that auxiliary verbs, including the verb *did* there, are raising verbs: their subjects are – or correspond to – the subjects of the infinitival verbs they select for.

In the Simple notation, the lexical entry of *did* may be defined as in (139).

(139) $\text{V}_{\langle N, V_{\langle N' \ldots \rangle, inf} \rangle, do, past, \pm inv}$: *did*

According to this entry, it is a *do*-auxiliary verb in the past tense, and it may be inverted. Its valency list contains two elements: a noun and an infinitival verb. Additionally, the first valent of that infinitival verb must correspond to the first valent of *did*; as above, this correspondence is expressed via the apostrophe: N' is the locally unrealized valent of the N valent of *did*.

This notation is a little misleading, as it assumes that the raised subject must be nominal, while in fact subjects of any category may be raised. For example, Gibson 2025: 82,

ex. (76), gives (140a) as an example of a sentential subject,[49] and such subjects may also be raised to subjects of auxiliary verbs, as shown in (140b–c).

(140) a. *That Francine ate the pizza bothered Lena.*
b. *That Francine ate the pizza did bother Lena.*
c. ?*Did that Francine ate the pizza bother Lena?*[50]

The following AVM-level lexical entry of *did* does not presuppose the category of the raised subject, but it still relies on the apostrophe convention for relating corresponding word objects:[51]

(141) $\text{LE}_{did} \stackrel{df}{\equiv}$
$$\left[\begin{array}{l} \textsc{phon}\ \textit{did} \\ \textsc{cat}\ \left[\begin{array}{l} \textit{do} \\ \textsc{tense}\ \textit{past} \end{array}\right] \\ \textsc{val}\ \langle \boxed{1}, \left[\begin{array}{ll} \textsc{cat} & \left[\begin{array}{l} \textit{verb} \\ \textsc{tense}\ \textit{inf} \end{array}\right] \\ \textsc{val} & \langle \boxed{1'}, \ldots \rangle \end{array}\right] \rangle \end{array}\right]$$

By comparison, the Logical-level lexical entry in (142) is explicit here, by invoking the `couw` relation defined in (124) above.

(142) $\text{LE}_{did}(x) \stackrel{df}{\equiv} x\ \textsc{phon} : \textit{did} \wedge x\ \textsc{cat} : \textit{do} \wedge x\ \textsc{cat tense} : \textit{past} \wedge \exists y.\exists z.\, y = x\ \textsc{val first} \wedge z = x\ \textsc{val rest first} \wedge z\ \textsc{cat tense} : \textit{inf} \wedge \texttt{couw}(y, z\ \textsc{val first}) \wedge x\ \textsc{val rest rest} : \textit{elist}$

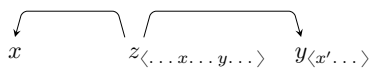
The end result of this lexical entry, however formalized, is that the subject of the infinitival verb is not realized locally (so it is not a node in the tree), but it shares relevant features with the subject of *did*, which is realized locally in (138) (and, hence, a node in the tree).

Note that raising is the second phenomenon, after LDD, which involves locally unrealized *word* objects. Any reasonable grammar will need a theory of such "silent" dependents; in some grammars they may also involve various kinds of implicit arguments (including *pro*-dropped subjects) or elided constituents. In the current fragment, we assume that they are limited to elements at the bottom of LDDs and to raised subjects:

(143) a. 'The only locally unrealized words are LDD gaps and raised subjects.'
b. $\forall x.\, x\ \textsc{real} : \textit{nil} \rightarrow (\texttt{ldd\_bot}(x) \vee \texttt{raised}(x))$

(144) a. 'A word is raised iff it is its governor's subject and it corresponds to a valent of its governor's governor.'
b. $\forall x'.\, \texttt{raised}(x') \leftrightarrow \exists x.\exists y.\exists z.\, x'\ \textsc{rel} : \textit{subj} \wedge x'\ \textsc{gov} = y \wedge \texttt{member}(y, z\ \textsc{val}) \wedge \texttt{member}(x, z\ \textsc{val}) \wedge \texttt{couw}(x, x')$

The configuration assumed in (144b) is presented schematically in (145).

(145)


This completes the analysis illustrated with the LDD example in (138). As in Gibson 2025: ch.3, the dashed arrows there are not part of the tree: $Mary_3$ is the subject of $think_4$ only in the weak sense of corresponding to the actual subject of $think_4$, i.e., the word $3'$ in

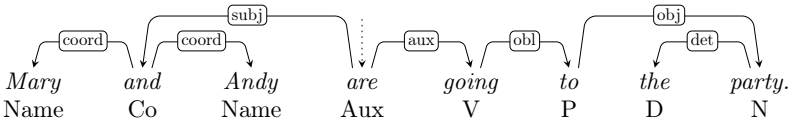
[49] Such subjects are not admitted by the "rules" in Gibson 2025: ch.3 – a fact not commented upon there.
[50] While some generative work denies the possibility of Subject–Auxiliary Inversion when the subject is clausal (Haegeman and Guéron 1999: 115), such examples may be found in corpora, e.g., the following two from the English Web 2021 (enTenTen21) corpus (http://www.sketchengine.eu; Kilgarriff et al. 2008, 2014):
i. Does that it might save lives make it right?
ii. Does that he is a lawyer make him an expert on Onan?
[51] Additionally, "$\langle \boxed{1'}, \ldots \rangle$" stands for "$\langle \boxed{1'} \rangle \oplus \textit{list}$".

the valency list of *think*$_4$, which is not realized locally, and similarly for *what*$_1$, which is the object of *bought*$_7$ only in the sense of corresponding to 1′ in that verb's valency list. The actual structure, built on overtly expressed words, is still a projective tree.

4.9 Coordination

While Gibson (2025: ch.3) does not suggest any formal analysis of coordination, he provides some examples of sentences involving coordinate structures, such as (146), below.[52]

(146)



Such coordinate structures, headed by the coordinator, are the staple of Functional Generative Description (Sgall et al. 1986) and they are employed in numerous dependency treebanks developed at the Charles University in Prague (e.g., Bejček et al. 2013).[53]

This and other examples of coordination in Gibson 2025: ch.3 violate various "rules" there.[54] In the particular case of (146), the *subject* link from the auxiliary *are* to the coordinator *and* violates "rules" which state that subjects of verbs, including auxiliary verbs, are nouns (Gibson 2025: 71–72). This is not easy to repair; for example, adding coordinators as possible subjects of verbs would massively overgenerate: any coordination could then be such a subject, e.g.:[55]

(147) * *Of Mary and of Andy are going to the party.*

In this final subsection, we show how this particular problem may be approached in MTDG. As in the case of other phenomena covered in this section, this is not meant to be a comprehensive analysis of coordination, which is widely recognized as one of the most difficult phenomena for dependency approaches (see, e.g., de Marneffe and Nivre 2019: 204).

Let us start with the internal structure of coordination. The signature assumed so far needs to be extended by another immediate subsort of *category*, namely, *coordinator* (abbreviated to *co*), as well as another immediate subsort of *relation*, namely, *coordinand* (abbreviated to *coord*).[56] Simple lexical entries of some coordinators are given in (148)–(149).

(148) $\mathrm{Co}_{\langle \_,\ \_ \rangle}$: *but*

(149) $\mathrm{Co}_{\langle \_,\ \_,\ \ldots \rangle}$: *and*, *or*

According to (148), *but* is a coordinator that takes exactly two arguments of unspecified categories. According to (149), the coordinators *and* and *or* take at least two but possibly more such unspecified arguments.

AVM-level lexical entries are fully analogous:

(150) $\mathrm{LE}_{but} \stackrel{df}{=} \begin{bmatrix} \textsc{phon} & but \\ \textsc{cat} & co \\ \textsc{val} & \langle \boxed{1}, \boxed{2} \rangle \end{bmatrix}$

[52] In Gibson 2025: 89, the arc from *going* to *to* is labelled *object*, but as elsewhere this label is used only for nominal dependents, we rename it to *oblique* here.

[53] This connection to the Praguian tradition is not explicitly made in Gibson 2025, where a reference is made to Sag et al. 1985 instead – a constituency-based multi-headed analysis of coordination more similar to the multi-headed dependency analysis within Word Grammar (Hudson 1984, 1990, 2010). On different approaches to coordinate structures in different linguistic frameworks and corpora, see, e.g., Popel et al. 2013 and Przepiórkowski and Woźniak 2023.

[54] These violations are not commented on in Gibson 2025.

[55] Note that semantics will not exclude such examples, as *of* is a semantically empty identity function elsewhere, e.g., in *the destruction of Rome*.

[56] We use here terms promoted by Martin Haspelmath (e.g., 2007) as less ambiguous than *conjunction* (does it include or exclude disjunctions?) and *conjunct* (does it include or exclude disjuncts?).

(151) $\text{LE}_{and} \overset{df}{\equiv} \begin{bmatrix} \text{PHON } and \\ \text{CAT } co \\ \text{VAL } \langle \boxed{1}, \boxed{2}, \ldots \rangle \end{bmatrix}$

And similarly for Logical-level descriptions, which specify that the REST of the VAL list after the first two arguments is *elist* (the empty list) in the case of *but*, but any *list* in the case of *and*:

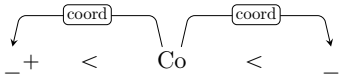
(152) $\text{LE}_{but}(x) \overset{df}{\equiv} x \text{ PHON} : but \wedge x \text{ CAT} : co \wedge x \text{ VAL REST REST} : elist$

(153) $\text{LE}_{and}(x) \overset{df}{\equiv} x \text{ PHON} : and \wedge x \text{ CAT} : co \wedge x \text{ VAL REST REST} : list$

We assume that all – and only – elements with the *coord* function are valents of coordinators:

(154) a. 'A word has the *coordinand* function iff it is a valent of a coordinator.'
b. $\forall x.\, x \text{ REL} : coord \leftrightarrow \exists y.\, y \text{ CAT} : co \wedge \texttt{member}(x, y \text{ VAL})$

As to word order, exactly one coordinand follows the coordinator:

(155) 

In this Simple word order rule, the underscore _ stands for any category, just as in the simplified lexical entries in (148)–(149). Additionally, the Kleene plus + stands for at least one occurrence, just as the Kleene star * was used for zero or more occurrences in some earlier Simple representations.

At the AVM level of description, this word order could be expressed as in (156), where the order of valents within VAL is assumed to be the same as their linear order, while at the Logical level as in (157), where no such assumption is made.

(156) $\boxed{1} < \ldots < \begin{bmatrix} \text{CAT } co \\ \text{VAL } \langle \boxed{1}, \ldots, \boxed{n} \rangle \end{bmatrix} < \boxed{n}$

(157) a. 'Exactly one argument of a coordinator follows it.'
b. $\forall x.\, x \text{ CAT} : co \rightarrow \exists! y.\, \texttt{member}(y, x \text{ VAL}) \wedge \texttt{precede}(x, y)$

We are now in position to deal with the problem this subsection started with, namely, selectional restrictions. Recall the lexical entry for *bothers*, repeated – at various levels of description – below:

(158) $\text{V}_{\langle N/Comp, N \rangle, sg, 3, pres}$: *bothers*

(159) $\text{LE}_{bothers} \overset{df}{\equiv} \begin{bmatrix} \text{PHON } bothers \\ \text{CAT } lverb \\ \text{VAL } \langle \begin{bmatrix} \text{CAT } noun \end{bmatrix} \vee \begin{bmatrix} \text{PHON } that \\ \text{CAT } comp \end{bmatrix}, \begin{bmatrix} \text{CAT } noun \end{bmatrix} \rangle \end{bmatrix}$

(160) $\text{LE}_{bothers}(x) \overset{df}{\equiv} x \text{ PHON} : bothers \wedge x \text{ CAT} : lverb \wedge$
$\exists y.\, y = x \text{ VAL FIRST} \wedge$
$(y \text{ CAT} : noun \vee (y \text{ CAT} : comp \wedge y \text{ PHON} : that)) \wedge$
$\exists z.\, z = x \text{ VAL REST FIRST} \wedge z \text{ CAT} : noun \wedge$
$x \text{ VAL REST REST} : elist$

This lexical entry assumes that selectional restrictions are expressed by directly specifying the features of the valent. For example, the second valent in (160), z, is specified as "z CAT : *noun*", and the first valent, y, as "y CAT : *noun* $\vee$ (y CAT : *comp* $\wedge$ y PHON : *that*)". Words with "CAT : *co*" directly violate such specifications.

To solve this problem, we propose to specify selectional restrictions via separately defined properties, so that the Logical-level lexical entry of *bothers* changes from (160) to (161).

(161) $\text{LE}_{bothers}(x) \overset{df}{\equiv} x \text{ PHON} : bothers \wedge x \text{ CAT} : lverb \wedge$
$\exists y.\, y = x \text{ VAL FIRST} \wedge \texttt{*noun\_or\_comp\_that}(y) \wedge$
$\exists z.\, z = x \text{ VAL REST FIRST} \wedge \texttt{*noun}(z) \wedge$
$x \text{ VAL REST REST} : elist$

If coordination were not an issue, *noun and *noun_or_comp_that could be defined just the way noun and noun_or_comp_that are defined in (162)–(163), making the new lexical entry in (161) fully equivalent to the previous one in (160).

(162) a. 'A linguistic entity is a noun iff its category is *noun*.'
b. $\forall x.\, \texttt{noun}(x) \leftrightarrow x \text{ CAT} : noun$

(163) a. 'A linguistic entity is a noun or the complementizer *that* iff either its category is *noun* or its category is *comp* and its form is *that*.'
b. $\forall x.\, \texttt{noun\_or\_comp\_that}(x) \leftrightarrow$
$(x \text{ CAT} : noun \vee (x \text{ CAT} : comp \wedge x \text{ PHON} : that))$

However, given the possibility of coordination, properties defined in (162)–(163) must be able to distribute to coordinands if necessary, as in (164)–(165).

(164) a. 'A linguistic entity is a noun in the broader sense iff either its category is *noun* or its category is *co*(ordinator) and all its valents are nouns in the broader sense.'
b. $\forall x.\, \texttt{*noun}(x) \leftrightarrow$
$(\texttt{noun}(x) \vee (x \text{ CAT} : co \wedge \forall y.\, \texttt{member}(y, x \text{ VAL}) \rightarrow \texttt{*noun}(y)))$

(165) a. 'A linguistic entity is a noun or the complementizer *that* in the broader sense iff one of the following two conditions are met: either 1) its category is *noun* or its category is *comp* and its form is *that*, or 2) its category is *co*(ordinator) and each of its valents is a noun or complementizer *that* in the broader sense.'
b. $\forall x.\, \texttt{*noun\_or\_comp\_that}(x) \leftrightarrow$
$(\texttt{noun\_or\_comp\_that}(x) \vee$
$(x \text{ CAT} : co \wedge \forall y.\, \texttt{member}(y, x \text{ VAL}) \rightarrow \texttt{*noun\_or\_comp\_that}(y)))$

The logical structure of these two formulae is identical, so let us take a closer look at the shorter (164). There, the distributive property *noun is defined to hold of x iff either its non-distributive version noun holds of x directly or x happens to be a coordinator, in which case the distributive property *noun must hold of all coordinands.[57] The reason for requiring that coordinands satisfy the distributive version of the property is the possibility of nested coordination, as in *Either Mary and Andy or Jane and Tom are going to the party*, where the two coordinands of *or* are themselves coordinate structures.

How about the simpler levels of description? We propose that the Simple lexical entries such as (158) remain as they are, with information that selectional restrictions such as *N/Comp* and *N* in $\text{V}_{\langle N/Comp, N\rangle, sg, 3, pres}$ may distribute into coordinands left implicit. As to the more precise AVM representation, this information could be given more explicitly, as in (166), where "[1]: AVM" is understood as "either [1] satisfies the description in the AVM, or [1] is a coordinate structure and for each coordinand [2] it holds that [2]: AVM".[58]

(166) $\text{LE}_{bothers} \overset{df}{\equiv} \begin{bmatrix} \text{PHON} & bothers \\ \text{CAT} & lverb \\ \text{VAL} & \langle [1], [2] \rangle \end{bmatrix}$
[1]: $\begin{bmatrix} \text{CAT} & noun \end{bmatrix} \vee \begin{bmatrix} \text{PHON} & that \\ \text{CAT} & comp \end{bmatrix}$
[2]: $\begin{bmatrix} \text{CAT} & noun \end{bmatrix}$

[57] The asterisk notation is meant to be reminiscent of the plurality operator of Link 1983: 306, transforming properties of individuals into properties of sums of such individuals. If we used Second Order Logic instead of FOL, we could define this operator once as in (i), removing the need for separate definitions such as (164)–(165).
(i) $\forall \text{P}_{\langle e,t\rangle}.\forall x_e.\, \texttt{*}(\texttt{P})(x) \leftrightarrow (\texttt{P}(x) \vee (x \text{ CAT} : co \wedge \forall y.\, \texttt{member}(y, x \text{ VAL}) \rightarrow \texttt{*}(\texttt{P})(y)))$

[58] This notation is based on that proposed for LFG in Przepiórkowski 2022: §7.1.

Let us illustrate this analysis with the copula, whose valency may be defined in the Simple way as in (167) (cf. Gibson 2025: 72), and similarly in the AVM notation in (168).

(167) $\mathrm{V}_{\langle N, N/Adj/P\rangle, be, sg, 3, pres, \pm inv}$: *is*

(168)
$$\begin{bmatrix} \textsc{phon}\ \textit{is} \\ \textsc{cat}\ \begin{bmatrix} \textit{be} \\ \textsc{num}\ \textit{sg} \\ \textsc{pers}\ \textit{3} \\ \textsc{tense}\ \textit{pres} \end{bmatrix} \\ \textsc{val}\ \langle \boxed{1}, \boxed{2} \rangle \end{bmatrix}$$
$\boxed{1}$: $[\ \textsc{cat}\ \textit{noun}\ ]$
$\boxed{2}$: $[\ \textsc{cat}\ \textit{noun} \vee \textit{adjective} \vee \textit{preposition}\ ]$

In the Logical description, a new selectional property needs to be defined, `noun_adjective_preposition`, abbreviated below to `nap`:

(169) a. 'A linguistic entity has the N/Adj/P property iff its category is either *noun*, or *adjective*, or *preposition*.'
b. $\forall x.\, \texttt{nap}(x) \leftrightarrow (x\ \textsc{cat} : \textit{noun} \vee x\ \textsc{cat} : \textit{adjective} \vee x\ \textsc{cat} : \textit{preposition})$

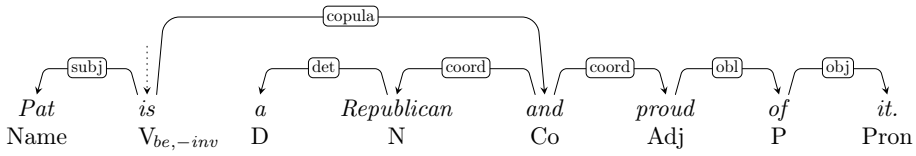
(170) a. 'A linguistic entity has the N/Adj/P property in the broader sense iff one of the following two conditions are met: either 1) its category is *noun*, or *adjective*, or *preposition*, or 2) its category is *co*(ordinator) and each of its valents has the N/Adj/P property in the broader sense.'
b. $\forall x.\, {}^{*}\texttt{nap}(x) \leftrightarrow (\texttt{nap}(x) \vee (x\ \textsc{cat} : co \wedge \forall y.\, \texttt{member}(y, x\ \textsc{val}) \rightarrow {}^{*}\texttt{nap}(y)))$

Then, the lexical entry for *is* may be defined in a way similar to that for *bother* above, as in (171) (ignoring morphosyntactic features, for brevity):

(171) $\mathrm{LE}_{is}(x) \overset{df}{\equiv} x\ \textsc{phon} : \textit{is} \wedge x\ \textsc{cat} : \textit{be} \wedge \exists y.\, y = x\ \textsc{val first} \wedge {}^{*}\texttt{noun}(y) \wedge \exists z.\, z = x\ \textsc{val rest first} \wedge {}^{*}\texttt{nap}(z) \wedge x\ \textsc{val rest rest} : \textit{elist}$

What is important in this approach to selectional restrictions is that disjunctive properties such as *`nap` may be satisfied by particular coordinands differently, leading to unlike category coordination (UCC), as in the famous (172) (Sag et al. 1985: 117), with the simplified structure in (173), where nominal and adjectival coordinands are combined.

(172) *Pat is a Republican and proud of it.*

(173)


Such unlike category coordination is much more widespread than previously thought (Patejuk and Przepiórkowski 2023). While analyses of UCC have been proposed in a number of frameworks (see, e.g., Przepiórkowski 2022 and references therein), this seems to be the first explication of how to formally approach this phenomenon within a dependency framework.

On the other hand, the analysis proposed in this subsection is not meant to be the beginning of a serious MTDG account of coordination. Instead, it simply shows how the structure of coordination assumed in Gibson 2025: ch.3 could be given some substance in MTDG and even provide the basis of the first dependency analysis of unlike category coordination. As noted at the beginning of this subsection, such coordinate structures, headed by coordinators, are only assumed in Functional Generative Description, while other dependency approaches assume very different structures, which may lead to more satisfactory MTDG

accounts of coordination. Developing such a comprehensive MTDG analysis of coordination is left for future research.

5 Gibson 2025

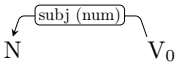
The aim of this paper is similar to that of Gibson 2025: ch.3, i.e., to propose a "feature-based dependency grammar formalism" (p. 94). Given that many rules and other mechanisms are left at an intuitive level there, with some important aspects of formalizations missing altogether (esp., in the case of LDDs and coordination), one fundamental difference between the two approaches is the level of explicitness and formalization. Another important difference is that Gibson's (2025) proposal seems to be set within the "rule-based" (proof-theoretic, generative-enumerative) paradigm, with frequent mentions of "syntactic rules" which "generate all and only the possible structures in the language" (pp. 22–23; also pp. 37, 40, 52–53, etc.). By contrast, our approach is squarely "constraint-based" (model-theoretic).

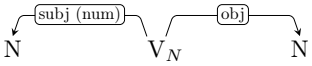
In the following three subsections, we point out that the syntactic "rules" assumed in Gibson 2025: ch.3 lack coherent interpretation (§5.1), we recapitulate and extend the various remarks above that they miss generalizations (§5.2), and we discuss the need for lexical rules (§5.3), assumed in Gibson 2025.

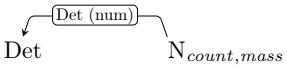
5.1 Syntactic "rules"

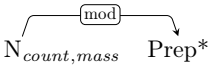
The reason for using scare quotes when referring the syntactic "rules" proposed in Gibson 2025: ch.3 is that it is not explained there how they should be used to "generate" dependency structures, and a coherent understanding of these rules is not easy to unravel.

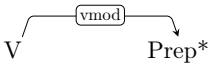
Consider the following "rules", shown here exactly the way they are presented in Gibson 2025: 56, 58, 68:

(174) 

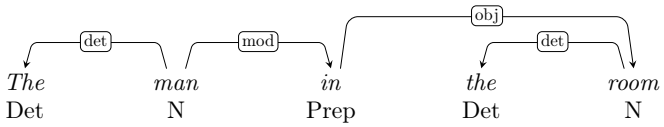
(175) 

(176) 

(177) 

(178) 

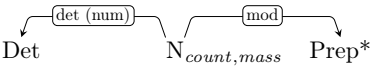
Such rules seem to be a generalization of rules assumed in Hays–Gaifman dependency grammars (HGDGs), discussed in §2.1; the two extensions are 1) labelled dependency arcs and 2) the use of regular expressions ("Prep*" in (177)–(178) stands for any number of prepositional dependents). However, they clearly do not have the interpretation of HGDG rules; if they did, common nouns could have a determiner (when (176) is applied) or prepositional modifiers (when (177) is applied), but not both (as only one rule can apply to any governor), contrary to Gibson's (2025: 68) example illustrating these rules:

(179) 

A fortiori, these rules cannot be understood as allowing "at most" the dependents mentioned in any single rule. Obviously, they cannot be understood as allowing "at least" the

dependents mentioned either, as then they would massively overgenerate, by allowing any dependents – on top of those mentioned in rules – to attach to any governors.

Rather, the intention seems to be that such rules may combine in determining the number and kind of dependents. For example, in order to "generate" (179), the "rules" in (176)–(177) must be used simultaneously, as if they were parts of the single rule in (180):[59]

(180)


This leads to the following intended interpretation of "rules":

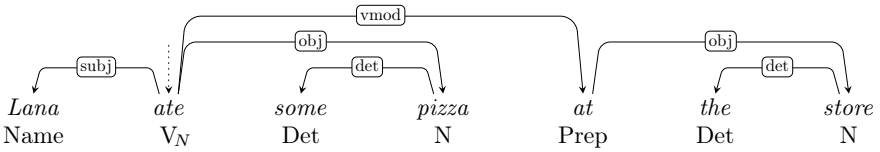
(181) Interpretation of "rules" (insufficient)

A local subtree with the governor X and the dependents $Y_1, \ldots, Y_n$ may be generated by the grammar iff there exist "rules" $R_1, \ldots, R_k$ such that:

a. the governor in each R_i matches X,
b. the set of sets of dependents $\{D_1, \ldots, D_k\}$ in these k rules is a partition of the set of dependents $Y_1, \ldots, Y_n$,
c. word order and agreement constraints imposed by each rule are satisfied.

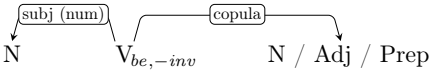
Note that this formulation implies that each rule can be used only once in generating a local subtree; this is as it should be, as otherwise multiple uses of (176) would license multiple determiners, multiple uses of (174) would allow for multiple subjects, etc.

However, this understanding of "rules" is not yet adequate, as it does not distinguish between "rules" that *must* apply obligatorily from those that *may* apply optionally. For example, in the case of singular count nouns, the determiner "rule" in (176) must apply, as otherwise the modifier "rule" (177), applied alone to the governor *man*, would make it possible to generate the ill-formed **Man in the room (yawned)*, in addition to the well-formed *A man in the room (yawned)*. Similarly, the transitive "rule" in (175) must apply to transitive verbs, as otherwise the verbal modification "rule" in (178) would alone "generate" the ill-formed **Ate at the store*, on top of well-formed structures such as (182) (from Gibson 2025: 68).

(182)


No hint at such a theory of obligatory and optional rules is made in Gibson 2025: ch.3, and we will not attempt to formalize such a theory here either; let us only note that such a formalization would most probably need to be stated in terms of constraints, making the resulting grammar at least partially "constraint-based".

Another problem with the apparently intended understanding of "rules" in (181) is that it leaves certain aspects of word order underspecified. For example, when both the transitive "rule" (175) and the verbal modifier "rule" in (178) are used to generate a single local subtree, as in (182), what makes sure that the object precedes the verbal modifier? In this particular case this word order interaction may be left unspecified, as sufficiently long objects may follow sufficiently short modifiers (e.g., *Lena ate there some anchovy pizza*), but there are other cases where the word order of dependents contributed by different "rules" is strict. Consider, for example, the two "rules" from Gibson 2025: 72 for "generating" predicative sentences with non-inverted and inverted copula:[60]

(183)


[59] With the label "Det (num)" in (176) corrected to "det (num)" here.
[60] With respect to the original formulations, "(num)" is added here for explicitness.

(184) 

In combination with the verbal modifier "rule" in (178), the above two "rules" may "generate" sentences in (185)–(186), respectively.

(185) a. *He is very gentle in fact.*
b. *He is in fact very gentle.*

(186) a. *Is he very gentle in fact?*
b. *Is he in fact very gentle?*
c. **Is in fact he very gentle?*

However, the last of these, (186c), seems to be ungrammatical: apparently verbal modifiers are not fully well-formed between the copula and the inverted subject. If so, additional word order constraints, on top of those encoded in rules, seem necessary, again resulting in a formalism that is at least partially "constraint-based".

In summary, it is not clear that there is a coherent understanding of the "rules" proposed in Gibson 2025: ch.3. The best we have come up with is that in (181), but it is clearly not sufficient.

5.2 Missed generalizations

Apart from the formal problems with Gibson's (2025: ch.3) "rules" discussed in the previous subsection, there are also a number of "missed generalization" problems – already alluded to in §4 – that make them less than attractive to the working linguist.

The first kind of missed generalization concerns word order. As mentioned in §4.6, out of 17 "rules" in Gibson 2025: ch.3, 12 encode information that may be expressed with the simple statement "Complements follow their governors" (as in (117) above). Similarly, 10 "rules" mentioning subjects very ineffectively encode the generalization that "Subjects precede their governors, with the exception of *+inv* governors" (see (113) above). In realistic grammars, containing many more rules like these to cover different valency types, these missed generalizations would be massive.

The second kind of missed generalization concerns agreement. All 10 "rules" referring to subjects should encode the simple fact that "Finite verbs agree in number with their nominal subjects" (see (42) in §4.2) – "should", as most of the "rules" in Gibson 2025: ch.3 are simplified by omitting this requirement. Again, a realistic grammar would contain many more "rules" mentioning subjects, so this missed generalization would be even more widespread. Moreover, only agreement in number is currently encoded, while person should also be taken into account (*I/You like grammar* vs. *She/He likes grammar*) and – in many languages – also gender, so the actual agreement rule would be more complex than just the equality of number values. And yet, this same agreement rule would have to be stated in a large number of syntactic "rules" on the approach proposed in Gibson 2025: ch.3. Moreover, in many languages agreement is not limited to finite verbs and their subjects; in many languages adjectives agree with their governing nouns in number, gender, and case, and on Gibson's (2025) approach this fact would have to be encoded over a number of "rules" (for pre-nominal attributive adjectives, for constructions with post-nominal adjectives, for primary and secondary predication, etc.).

We believe that such missed generalizations and – more generally – lack of formal language for expressing such generalizations make the proposed "feature-based dependency grammar formalism" unacceptable for contemporary theoretical linguists. One solution would be to add some mechanism for stating such constraints, but once a language for expressing constraints like the one advocated in the current paper is available, "rules" of the kind proposed in Gibson 2025: ch.3 become spurious.

5.3 Lexical rules

Let us finish this section by considering a formal device assumed in Gibson 2025: ch.3, but not mentioned so far, namely that of lexical rules (LRs), "generating" one lexical form from another (p. 71). The following LRs are proposed in Gibson 2025: 71, 82–83, 95–96:[61]

(187) Lexical Rules in Gibson 2025: ch.3:

a. $\mathrm{V}_{+aux} \Rightarrow \mathrm{V}_{+aux,+inv}$
b. $\mathrm{V}_{Comp} \Rightarrow \mathrm{V}_{V+finite}$
c. $\mathrm{V}_{N} \Rightarrow \mathrm{V}_{prep(by),+passive}$
d. $\mathrm{V}_{N} \Rightarrow \mathrm{V}_{0,SLASH(N)}$
e. $\mathrm{P}_{N} \Rightarrow \mathrm{P}_{0,SLASH(N)}$

Work within HPSG has made it clear that such rules are very difficult to formalize (see, e.g., Lahm 2016 for discussion and references) and that their intended effect may often be achieved via other, more standard means, including an appropriately enriched sort hierarchy and/or underspecification (see Davis and Koenig 2024 for discussion and references). This last mechanism – underspecification – was in fact employed in §4 for achieving the intended effects of (187a) and (187d–e).

The LR in (187a) is supposed to "generate" inverted forms of auxiliaries, on the assumption that only non-inverted forms are stated directly in the lexicon. However, the same effect may be achieved by lexically underspecifying auxiliaries for the value of INV, as was done in (168) on p. 35 in §4.9. Then, given the signature in (37)–(38), the value of INV is bound to be + (i.e., inverted) or *nil* (not inverted), automatically resulting in two different words for each such underspecified lexical entry.

Similarly, the effect of (187d–e) was achieved in §4.7 by underspecifying potential dependents as to whether they are slash-less and locally realized, or slash-bearing and locally unrealized. The general treatment of Long-Distance Dependencies proposed there applies to all kinds of dependents, unlike LRs such as (187d–e), which must be stated individually for each valent of each possible ⟨category, valency frame⟩ combination, resulting in a very large number of such rules in a realistic grammar. What is worse, such a large set of rules would still be insufficient to handle extraction of modifiers, as in *When did you say you want to do it _ ?*, given that modifiers are not valents. LRs adding such modifiers as slash values could be stated, but they would have to encode information about what kinds of modifiers are appropriate for what kinds of governors, repeating information stated in syntactic "rules" and resulting in yet another missed generalization.

A similar missed generalization may be observed in the case of the passivization LR in (187c). This particular rule handles simple transitive verbs, whose valency is expressed as "N", but analogous rules would have to be given for other valencies, e.g., "N, N" (e.g. *John gave Mary a book ⇒ Mary was given a book (by John)*), "$N, prep(to)$" (e.g. *John gave a book to Mary ⇒ A book was given to Mary (by John)*), "$N, Comp$" (e.g., *Mary told John that it's raining ⇒ John was told (by Mary) that it's raining*), etc. Interestingly, while the passivization lexical rule – as opposed to the passive transformation of the day – was championed by Bresnan (1978) and assumed in early LFG (Bresnan 1982b), such lexical rules have fallen out of favour in recent LFG work (Findlay et al. 2023: §3). Instead, appropriate mappings are assumed between valency frames, which encode actual grammatical functions, and a more semantic representation of argument structure.

Finally, the simple rule in (187b) says that a verb that combines with a declarative clause introduced by *that* may alternatively combine with a bare clause, e.g., *Mary said that it's raining ⇒ Mary said it's raining*. Again, more LRs are needed to handle other valencies involving *Comp* (e.g., $N, Comp$, as in *Mary told John that it's raining ⇒ Mary told John it's raining*), and again, the same effect may be achieved by underspecification

[61] One more, responsible for dative alternation, is alluded to (on p. 80), but not stated.

or appropriate mapping between the more semantic argument structure and the syntactic valency.

In summary, the mechanism of lexical rules does not seem to be needed for a successful formalization of dependency grammars. Nevertheless, if it turns out to be useful in a given dependency approach, it can be formalized in a way similar to the HPSG formalization discussed in Meurers 2001 and Davis et al. 2024: §5.3.

In the current setup, we assume an additional attribute appropriate for *word* objects, LR, whose value is usually *nil* (for words not "derived" via lexical rules), but may also be a *word* (for words so "derived").[62] In order to state the passive lexical rule formally, we also need to be able to represent active and passive voice; for this, we assume that lexical verbs have the attribute VOICE, with *active* and *passive* as its values.

Active verbs may now have AVM-level lexical entries as in (188)–(190), with the [LR *nil*] specification (assumed implicitly, if not specified explicitly).

(188) $\text{LE}_{chase} \stackrel{df}{\equiv} \begin{bmatrix} \text{PHON } chase \\ \text{CAT } \begin{bmatrix} \text{TENSE } infinitive \\ \text{VOICE } active \end{bmatrix} \\ \text{VAL } \langle [\text{CAT } noun], [\text{CAT } noun] \rangle \\ \text{LR } nil \end{bmatrix}$

(189) $\text{LE}_{give} \stackrel{df}{\equiv} \begin{bmatrix} \text{PHON } give \\ \text{CAT } \begin{bmatrix} \text{TENSE } infinitive \\ \text{VOICE } active \end{bmatrix} \\ \text{VAL } \langle [\text{CAT } noun], [\text{CAT } noun], [\text{CAT } noun] \rangle \\ \text{LR } nil \end{bmatrix}$

(190) $\text{LE}_{tell} \stackrel{df}{\equiv} \begin{bmatrix} \text{PHON } tell \\ \text{CAT } \begin{bmatrix} \text{TENSE } infinitive \\ \text{VOICE } active \end{bmatrix} \\ \text{VAL } \langle [\text{CAT } noun], [\text{CAT } noun], \begin{bmatrix} \text{PHON } that \\ \text{CAT } comp \end{bmatrix} \rangle \\ \text{LR } nil \end{bmatrix}$

By contrast, lexical entries implementing lexical rules will have their inputs specified via LR, as in the following first approximation of the passive lexical rule:[63]

(191) $\text{LE}_{passive\_LR} \stackrel{df}{\equiv} \begin{bmatrix} \text{PHON } \boxed{2} \\ \text{CAT VOICE } passive \\ \text{VAL } \langle [\text{CAT } noun] \rangle \oplus \boxed{3} \oplus \langle \begin{bmatrix} \text{PHON } by \\ \text{CAT } prep \end{bmatrix} \rangle \\ \text{LR } \begin{bmatrix} \text{PHON } \boxed{1} \\ \text{CAT } \begin{bmatrix} \text{TENSE } infinitive \\ \text{VOICE } active \end{bmatrix} \\ \text{VAL } \langle [\text{CAT } noun], [\text{CAT } noun] \rangle \oplus \boxed{3} \\ \text{REAL } nil \\ \text{SLASH } nil \\ \text{NEXT } nil \end{bmatrix} \end{bmatrix} \wedge$
$\texttt{passive\_morphology}(\boxed{1}, \boxed{2})$

The effect of this passive LR is a passive word whose form is related to the form of the active input via the `passive_morphology` relation, encoding both the regular *-(e)d* suffixation (e.g., *chase* ⇒ *chased*), as well as various subregularities and exceptions (e.g., *give* ⇒ *given*, *tell* ⇒ *told*). Moreover, the valency of the passive word is systematically related to the valency of the active input: the initial two nominal requirements are reduced

[62] "Derived" occurs in scare quotes as, in model-theoretic frameworks such as HPSG or MTDG, lexical rules do not actually derive one structure from another but rather specify static relations between certain kinds of structures.

[63] Once lexical rules are part of the system, it might make more sense to assume that all forms, active and passive alike, are "derived" from representations of lexemes, rather than passive forms being "derived" from active forms. (Thanks to Stefan Müller, p.c., for pointing this out to me.)

to one, any other valency elements of the active input are added to the valency of the passive output, and finally the (optional) *by* valent is appended.[64] For example, if values of LR are the active verbs lexically specified in (188)–(190), the lexical rule entry in (191) will give rise to the words described in (192)–(194).

(192)
$$\left[\begin{array}{l}\textsc{phon}\ \textit{chased}\\ \textsc{cat}\ \textsc{voice}\ \textit{passive}\\ \textsc{val}\ \langle\left[\textsc{cat}\ \textit{noun}\right], \left[\begin{array}{l}\textsc{phon}\ \textit{by}\\ \textsc{cat}\ \textit{prep}\end{array}\right]\rangle\\ \textsc{lr}\ \left[\begin{array}{l}\textsc{phon}\ \textit{chase}\\ \textsc{cat}\ \left[\begin{array}{l}\textsc{tense}\ \textit{infinitive}\\ \textsc{voice}\ \textit{active}\end{array}\right]\\ \textsc{val}\ \langle\left[\textsc{cat}\ \textit{noun}\right], \left[\textsc{cat}\ \textit{noun}\right]\rangle\\ \textsc{lr}\ \textit{nil}\\ \textsc{real}\ \textit{nil}\\ \textsc{slash}\ \textit{nil}\\ \textsc{next}\ \textit{nil}\end{array}\right]\end{array}\right]$$

(193)
$$\left[\begin{array}{l}\textsc{phon}\ \textit{given}\\ \textsc{cat}\ \textsc{voice}\ \textit{passive}\\ \textsc{val}\ \langle\left[\textsc{cat}\ \textit{noun}\right], \left[\textsc{cat}\ \textit{noun}\right], \left[\begin{array}{l}\textsc{phon}\ \textit{by}\\ \textsc{cat}\ \textit{prep}\end{array}\right]\rangle\\ \textsc{lr}\ \left[\begin{array}{l}\textsc{phon}\ \textit{give}\\ \textsc{cat}\ \left[\begin{array}{l}\textsc{tense}\ \textit{infinitive}\\ \textsc{voice}\ \textit{active}\end{array}\right]\\ \textsc{val}\ \langle\left[\textsc{cat}\ \textit{noun}\right], \left[\textsc{cat}\ \textit{noun}\right], \left[\textsc{cat}\ \textit{noun}\right]\rangle\\ \textsc{lr}\ \textit{nil}\\ \textsc{real}\ \textit{nil}\\ \textsc{slash}\ \textit{nil}\\ \textsc{next}\ \textit{nil}\end{array}\right]\end{array}\right]$$

(194)
$$\left[\begin{array}{l}\textsc{phon}\ \textit{told}\\ \textsc{cat}\ \textsc{voice}\ \textit{passive}\\ \textsc{val}\ \langle\left[\textsc{cat}\ \textit{noun}\right], \left[\begin{array}{l}\textsc{phon}\ \textit{that}\\ \textsc{cat}\ \textit{comp}\end{array}\right], \left[\begin{array}{l}\textsc{phon}\ \textit{by}\\ \textsc{cat}\ \textit{prep}\end{array}\right]\rangle\\ \textsc{lr}\ \left[\begin{array}{l}\textsc{phon}\ \textit{tell}\\ \textsc{cat}\ \left[\begin{array}{l}\textsc{tense}\ \textit{infinitive}\\ \textsc{voice}\ \textit{active}\end{array}\right]\\ \textsc{val}\ \langle\left[\textsc{cat}\ \textit{noun}\right], \left[\textsc{cat}\ \textit{noun}\right], \left[\begin{array}{l}\textsc{phon}\ \textit{that}\\ \textsc{cat}\ \textit{comp}\end{array}\right]\rangle\\ \textsc{lr}\ \textit{nil}\\ \textsc{real}\ \textit{nil}\\ \textsc{slash}\ \textit{nil}\\ \textsc{next}\ \textit{nil}\end{array}\right]\end{array}\right]$$

Importantly, the input to lexical rules, including the passive lexical rule in (191), is specified as [REAL *nil*] (and, concomitantly, [SLASH *nil*] and [NEXT *nil*]), so that words from which new words are "derived" will not be nodes in dependency trees (cf. §4.7). On the other hand, as the input is of sort *word*, it must satisfy the Lexicon Constraint in (63), so it must be a word corresponding to some other lexical entry (perhaps another lexical rule entry).

6 Conclusion

No formalization is presently available of the kinds of monostratal Dependency Grammars that are currently developed within theoretical linguistics, especially, within Surface Syntactic Dependency Grammar (Tesnière 1959, 2015, Osborne 2019) and within Word

[64] We do not attempt to model the optionality of the prepositional phrase expressing the agent here, as optionality of (some) valents requires a more general treatment.

Grammar (Hudson 1984, 1990, 2010). Gibson (2025: ch.3) attempts to provide a "feature-based dependency grammar formalism" for such approaches. This intention is to be hailed. Unfortunately, the rule-based execution of this program in Gibson 2025: ch.3 is largely unsuccessful.

The aim of this paper is to provide an alternative execution of this intention, one that adheres to the constraint-based paradigm, commonly assumed in contemporary dependency approaches. The empirical ground covered here is similar to that in Gibson 2025: ch.3, but extends it by providing fully explicit and precise analyses of (aspects of) long-distance dependencies, raising, and coordination.

As mentioned in the Introduction, there is a tension between simplicity on one hand and explicitness and precision on the other. This tension is made conspicuous in the constraints proposed throughout the paper, formulated at three levels of description of increasing explicitness and precision, but decreasing simplicity. Constraints stated at the Simple level, similar to those assumed in Gibson 2025, are often misleading and require additional explanation in prose, to make their intended effect clear. Those stated at the AVM level are more explicit and precise, and probably still sufficiently simple for linguists familiar with other contemporary syntactic frameworks such as LFG and HPSG. Those at the Logical level require fluency in First Order Logic, but are fully explicit and precise. Some of the more technical constraints, especially framework constraints and model constraints, could only be stated in this Logical language.

Hence, this paper contributes to the debate about the perceived simplicity of dependency approaches and the perceived complexity of constituency approaches, stated forcefully in Gibson 2026. In line with the more comprehensive discussion in Müller 2026, we believe that this apparent difference is mostly an epiphenomenon of the degree of formalization – and accompanying explicitness and precision – assumed in both approaches so far. As demonstrated in the current paper, dependency grammars become as complex as constituency grammars when stated at the same level of explicitness and precision, even if the resulting dependency structures are simpler in the sense of containing fewer syntactic nodes.

This, however, is not an argument for abandoning dependency frameworks. On the contrary, dependency approaches are currently dominant in Cognitive Science (as witnessed by Gibson's 2025 textbook) and within Natural Language Processing (see, e.g., Kübler et al. 2009, Hromei et al. 2024, and fn. 4), and they are also commonly used for evaluating the linguistic capabilities of Large Language Models (see, e.g., Hewitt and Manning 2019 and Jumelet et al. 2026), so it is timely that such dependency approaches should also be considered more seriously within theoretical linguistics. We believe that one obstacle to a greater adoption of dependency frameworks by linguists who care about the predictive power of their analyses has been the lack of formalization. We hope to have removed this obstacle.

Acknowledgements. This paper benefited from comments by Bob Borsley, Ted Gibson, Dick Hudson, Stefan Müller, Tim Osborne, Geoff Pullum, and Berke Şenşekerci, as well as the reviewers and the audience of the 33rd International Conference on Head-Driven Phrase Structure Grammar. The usual disclaimers apply.

Appendix A Definition of L_{DG}

The logical language used in the main text, L_{DG}, is a variant of the language of First Order Logic (FOL). However, since it assumes non-standard signatures, some syntactic sugar, and partial functions, we define it formally in this appendix: first, the signature (in §A.1), as it is relevant both for syntax (defined in §A.2), and for semantics (in §A.3).

A.1 Signature

The definition of an L_{DG} *signature* is based on the definition of RSRL signatures in Richter 2004: 156, where RSRL (Relational Speciate Re-entrant Language; King 1989, 1994, 1999, Richter 1999, 2004, 2007) is a logic used in the model-theoretic formalization of HPSG grammars.

Definition 1 (signature)**.** Σ *is a* ***signature*** *iff*
Σ *is a sextuple* $\langle S, \sqsubseteq, A, F, R, Ar\rangle$,
$\langle S, \sqsubseteq\rangle$ *is a partial order such that:*
  $\top \in S$ *and* $nil \in S$,
  for all $\sigma \in S$, $\sigma \sqsubseteq \top$,
  for all $\sigma \in S$, *if* $\sigma \sqsubseteq nil$, *then* $\sigma = nil$,
  for all $\sigma \in S$, *if* $nil \sqsubsetneq \sigma$, *then* $\sigma = \top$,
A *is a set,*
F *is a partial function from* $S \times A$ *to* S,
for each $\sigma_1 \in S$, *for each* $\sigma_2 \in S$, *for each* $\alpha \in A$,
  if $F(\sigma_1, \alpha)$ *is defined and* $\sigma_2 \sqsubseteq \sigma_1$
  then $F(\sigma_2, \alpha)$ *is defined and* $F(\sigma_2, \alpha) \sqsubseteq F(\sigma_1, \alpha)$,
R *is a finite set, and*
Ar *is a total function from* R *to the positive integers.*

According to this definition, a signature determines a partially ordered set of sorts S such that $\top$ is the most general sort and *nil* is its immediate subsort. Moreover, *nil* is a minimal sort, i.e., one of the *species*.

Definition 2 (species)**.** S_{min} *is the set of* ***species*** *of the signature* $\Sigma = \langle S, \sqsubseteq, A, F, R, Ar\rangle$ *iff* $S_{min} = \{\sigma \in S \mid$ *for each* $\sigma' \in S$, *if* $\sigma' \sqsubseteq \sigma$ *then* $\sigma = \sigma'\}$.

Additionally, a signature determines the set of attributes, A, and their appropriateness for particular sorts, F. If an attribute α is defined for a sort σ, then it is also defined for subsorts σ' of this sort. Moreover, if the values of α for σ are defined as being of a certain sort λ, then the values of α for σ' must of a sort $\lambda' \sqsubseteq \lambda$.[65] As a typographical convention, sorts are typeset in italics, e.g., *word*, while attributes are in small capitals, e.g., PHON.

Finally, a signature also determines the set of relations, R, and their arities, Ar. For example, one of the relations defined in the main text (in (99) on p. 21) is `member`, such that $Ar(\texttt{member}) = 2$. As a typographical convention, relations are written in the typewriter font.

A.2 Syntax

Also the definition of the syntax of L_{DG} is to some extent based on that of RSRL (Richter 2004: 162). First, given a countably infinite set of variables, V, terms are defined as follows:

[65] This aspect of HPSG signatures is not taken advantage of in the main text, but given its usefulness in HPSG, it carried over to MTDG.

Definition 3 (terms)**.** *For each signature* $\Sigma = \langle S, \sqsubseteq, A, F, R, Ar\rangle$*, the set of* ***terms*** T^Σ *is the smallest set such that*
for each $x \in V$*,* $x \in T^\Sigma$*,*
for each $\alpha \in A$ *and each* $\tau \in T^\Sigma$*,* $\tau\,\alpha \in T^\Sigma$*.*

That is, terms are variables followed by (perhaps empty) sequences of attributes. Given this definition of terms, the definition of formulae is similar to that of standard FOL with equality, with just one additional kind of atomic formulae involving the : sort operator.

Definition 4 (formulae)**.** *For each signature* $\Sigma = \langle S, \sqsubseteq, A, F, R, Ar\rangle$*, the set of* ***formulae*** D^Σ *is the smallest set such that*
for each $\sigma \in S$*, for each* $\tau \in T^\Sigma$*,* $\tau : \sigma \in D^\Sigma$*,*
for each $\tau_1, \tau_2 \in T^\Sigma$*,* $\tau_1 = \tau_2 \in D^\Sigma$*,*
for each $\rho \in R$*, for each* $\tau_1, \ldots, \tau_{Ar(\rho)} \in T^\Sigma$*,* $\rho(\tau_1, \ldots, \tau_{Ar(\rho)}) \in D^\Sigma$*,*
for each $x \in V$*, for each* $\delta \in D^\Sigma$*,* $\exists x.\delta \in D^\Sigma$*,* *(analogously for* $\forall$*)*
for each $\delta \in D^\Sigma$*,* $\neg\delta \in D^\Sigma$*,*
for each $\delta_1, \delta_2 \in D^\Sigma$*,* $(\delta_1 \wedge \delta_2) \in D^\Sigma$*.* *(analogously for* $\vee, \rightarrow, \leftrightarrow$*)*

Parentheses may be omitted when this does not lead to ambiguity. For example, given the convention that the scope of quantifiers extends as far as possible to the right, the official (A1a) may be simplified to (A1b).

(A1) a. $\forall x.\forall y.\,(\texttt{precede}(x, y) \leftrightarrow$
$(x : \mathit{word} \wedge (y : \mathit{word} \wedge (x \text{ NEXT} = y \vee \exists z.\,(x \text{ NEXT} = z \wedge \texttt{precede}(z, y))))))$

b. $\forall x.\forall y.\,\texttt{precede}(x, y) \leftrightarrow$
$(x : \mathit{word} \wedge y : \mathit{word} \wedge (x \text{ NEXT} = y \vee \exists z.\,x \text{ NEXT} = z \wedge \texttt{precede}(z, y)))$

Finally, an $\mathrm{L_{DG}}$ theory is any finite set of closed formulae:

Definition 5 (theory)**.** *For each signature* Σ*, a* ***theory*** θ^Σ *is any finite subset of* $D_0^\Sigma = \{\delta \in D^\Sigma \mid FV(\delta) = \{\}\}$*, where* $FV(\delta)$ *is the set of free variables in* δ*.*

A.3 Semantics

The definition of *interpretation* is also to some extent based on that for RSRL (Richter 2004: 157–158):

Definition 6 (interpretation)**.** *For each signature* $\Sigma = \langle S, \sqsubseteq, A, F, R, Ar\rangle$*,* $\mathsf{I} = \langle \mathsf{U}, \mathsf{S}, \mathsf{A}, \mathsf{R}\rangle$ *is a* Σ ***interpretation*** *iff*
U *is a set,*
S *is a total function from* U *to* S_{min}*,*
A *is a total function from* A *to the set of partial functions from* U *to* U*,*
for each $\alpha \in A$ *and each* $u \in \mathsf{U}$*,*
if $F(\mathsf{S}(u), \alpha)$ *is defined then* $\mathsf{A}(\alpha)(u)$ *is defined,*
for each $\alpha \in A$ *and each* $u \in \mathsf{U}$
if $\mathsf{A}(\alpha)(u)$ *is defined*
then $F(\mathsf{S}(u), \alpha)$ *is defined, and* $\mathsf{S}(\mathsf{A}(\alpha)(u)) \sqsubseteq F(\mathsf{S}(u), \alpha)$ *or* $\mathsf{S}(\mathsf{A}(\alpha)(u)) = \mathit{nil}$*, and*
R *is a total function from* R *to the power set of* $\bigcup_{n\in\mathbb{N}} \mathsf{U}^n$*, and*
for each $\rho \in R$*,* $\mathsf{R}(\rho) \subseteq \mathsf{U}^{Ar(\rho)}$*.*

An interpretation of the signature $\Sigma = \langle S, \sqsubseteq, A, F, R, Ar\rangle$ is thus a set of objects such that each is assigned a unique minimal sort from $\langle S, \sqsubseteq\rangle$. Attributes A are interpreted as partial functions which satisfy the appropriateness conditions defined in F, with the

additional possibility of the value of any attribute being *nil*. Finally, the relations in R are interpreted as sets of tuples of appropriate arity, defined by Ar.

The definition of a *variable assignment* is standard – it is a total function g from the set of variables V to the set of objects U.

Definition 7 (variable assignments)**.** *For each interpretation* $\mathsf{I} = \langle \mathsf{U}, \mathsf{S}, \mathsf{A}, \mathsf{R} \rangle$*, the set of* ***variable assignments*** *in* I *is* $\mathsf{G}_\mathsf{I} = \mathsf{U}^V$*.*

As is usual, we will overload the $[\![\cdot]\!]$ notation for both *term interpretation* and *formula denotation.*

Definition 8 (term interpretation)**.** *For each signature* $\Sigma = \langle S, \sqsubseteq, A, F, R, Ar \rangle$*, for each* Σ *interpretation* $\mathsf{I} = \langle \mathsf{U}, \mathsf{S}, \mathsf{A}, \mathsf{R} \rangle$*, for each* $g \in \mathsf{G}_\mathsf{I}$*, the* ***term interpretation*** $[\![\cdot]\!]^{g,\mathsf{I}}$ *is the partial function from* T^Σ *to* U *such that:*

for each $x \in V$*,* $[\![x]\!]^{g,\mathsf{I}}$ *is defined and* $[\![x]\!]^{g,\mathsf{I}} = g(x)$*,*
for each $\tau \in T^\Sigma$*, for each* $\alpha \in A$*,*
$[\![\tau\,\alpha]\!]^{g,\mathsf{I}}$ *is defined iff* $[\![\tau]\!]^{g,\mathsf{I}}$ *is defined and* $\mathsf{A}(\alpha)([\![\tau]\!]^{g,\mathsf{I}})$ *is defined, and*
if $[\![\tau\,\alpha]\!]^{g,\mathsf{I}}$ *is defined then* $[\![\tau\,\alpha]\!]^{g,\mathsf{I}} = \mathsf{A}(\alpha)([\![\tau]\!]^{g,\mathsf{I}})$*.*

This definition is relatively complex, as the interpretation of terms must be sensitive to the appropriateness condition and, hence, is only a partial function from terms to objects. For example, assuming signatures as in the main text, if the object denoted by x is of sort *singular*, then the interpretation of the syntactically well defined term x GOV is undefined, as only *word* objects have the GOV attribute.

The definition of *formula denotation* is as in FOL, with some complications due to the fact that not all terms have interpretations and due to special atomic formulae for sort specification:

Definition 9 (formula denotation)**.** *For each signature* $\Sigma = \langle S, \sqsubseteq, A, F, R, Ar \rangle$*, for each* Σ *interpretation* $\mathsf{I} = \langle \mathsf{U}, \mathsf{S}, \mathsf{A}, \mathsf{R} \rangle$*, for each* $g \in \mathsf{G}_\mathsf{I}$*, the* ***formula denotation*** *function* $[\![\cdot]\!]^{g,\mathsf{I}}$ *is the total function from* D^Σ *to* $\{\mathsf{T}, \mathsf{F}\}$ *such that:*

for each $\tau \in T^\Sigma$*, for each* $\sigma \in S$*,*
$[\![\tau : \sigma]\!]^{g,\mathsf{I}} = \mathsf{T}$ *iff* $[\![\tau]\!]^{g,\mathsf{I}}$ *is defined and* $\mathsf{S}\big([\![\tau]\!]^{g,\mathsf{I}}\big) \sqsubseteq \sigma$*,*
for each $\tau_1, \tau_2 \in T^\Sigma$*,*
$[\![\tau_1 = \tau_2]\!]^{g,\mathsf{I}} = \mathsf{T}$ *iff* $[\![\tau_1]\!]^{g,\mathsf{I}}$ *is defined,* $[\![\tau_2]\!]^{g,\mathsf{I}}$ *is defined, and* $[\![\tau_1]\!]^{g,\mathsf{I}} = [\![\tau_2]\!]^{g,\mathsf{I}}$*,*
for each $\rho \in R$*, for each* $\tau_1, \ldots, \tau_{Ar(\rho)} \in T^\Sigma$*,*
$[\![\rho(\tau_1, \ldots, \tau_{Ar(\rho)})]\!]^{g,\mathsf{I}} = \mathsf{T}$ *iff* $\langle [\![\tau_1]\!], \ldots, [\![\tau_{Ar(\rho)}]\!] \rangle \in \mathsf{R}(\rho)$*,*
for each $x \in V$*, for each* $\delta \in D^\Sigma$*,*
$[\![\exists x.\delta]\!]^{g,\mathsf{I}} = \mathsf{T}$ *iff* $[\![\delta]\!]^{g[x \mapsto u],\mathsf{I}} = \mathsf{T}$ *for some* $u \in \mathsf{U}$*,*
$[\![\forall x.\delta]\!]^{g,\mathsf{I}} = \mathsf{T}$ *iff* $[\![\delta]\!]^{g[x \mapsto u],\mathsf{I}} = \mathsf{T}$ *for each* $u \in \mathsf{U}$*,*
for each $\delta \in D^\Sigma$*,*
$[\![\neg\delta]\!]^{g,\mathsf{I}} = \mathsf{T}$ *iff* $[\![\delta]\!]^{g,\mathsf{I}} = \mathsf{F}$*,*
for each $\delta_1, \delta_2 \in D^\Sigma$*,*
$[\![\delta_1 \wedge \delta_2]\!]^{g,\mathsf{I}} = \mathsf{T}$ *iff* $[\![\delta_1]\!]^{g,\mathsf{I}} = \mathsf{T}$ *and* $[\![\delta_2]\!]^{g,\mathsf{I}} = \mathsf{T}$*,*
$[\![\delta_1 \vee \delta_2]\!]^{g,\mathsf{I}} = \mathsf{T}$ *iff* $[\![\delta_1]\!]^{g,\mathsf{I}} = \mathsf{T}$ *or* $[\![\delta_2]\!]^{g,\mathsf{I}} = \mathsf{T}$*,*
$[\![\delta_1 \rightarrow \delta_2]\!]^{g,\mathsf{I}} = \mathsf{T}$ *iff* $[\![\delta_1]\!]^{g,\mathsf{I}} = \mathsf{F}$ *or* $[\![\delta_2]\!]^{g,\mathsf{I}} = \mathsf{T}$*,*
$[\![\delta_1 \leftrightarrow \delta_2]\!]^{g,\mathsf{I}} = \mathsf{T}$ *iff either*
$[\![\delta_1]\!]^{g,\mathsf{I}} = \mathsf{T}$ *and* $[\![\delta_2]\!]^{g,\mathsf{I}} = \mathsf{T}$*, or*
$[\![\delta_1]\!]^{g,\mathsf{I}} = \mathsf{F}$ *and* $[\![\delta_2]\!]^{g,\mathsf{I}} = \mathsf{F}$*.*

Finally, an interpretation is a *model* of a theory iff all formulae of that theory are *true* in that interpretation.

Definition 10 (truth)**.** *A formula δ is **true** in an interpretation I iff $[\![\delta]\!]^{g,\mathsf{I}} = \mathsf{T}$ for each $g \in \mathsf{G}_\mathsf{I}$.*

Definition 11 (model)**.** *For each signature Σ, for each theory θ^Σ, a Σ interpretation I is a **model** of θ^Σ iff each $\delta \in \theta^\Sigma$ is true in I.*

Appendix B Translation into standard FOL

Translation of $\mathrm{L_{DG}}$ into FOL, the language of standard First Order Logic, is relatively straightforward, so we only sketch it here.

B.1 Signature

Standard FOL signature is usually taken to be a triple, $\langle Fu, Rel, Ar\rangle$, where Fu is a set of function symbols, Rel is a set of relation symbols, and Ar is a function assigning arities to both kinds of symbols. The translation of an $\mathrm{L_{DG}}$ signature $\Sigma_{\mathrm{DG}} = \langle S_{\mathrm{DG}}, \sqsubseteq, A_{\mathrm{DG}}, F_{\mathrm{DG}}, R_{\mathrm{DG}}, Ar_{\mathrm{DG}}\rangle$ into a FOL signature $\Sigma_{\mathrm{FOL}} = \langle Fu_{\mathrm{FOL}}, Rel_{\mathrm{FOL}}, Ar_{\mathrm{FOL}}\rangle$ is defined as follows:

(B2) a. $Fu_{\mathrm{FOL}} \stackrel{df}{\equiv} A_{\mathrm{DG}}$
b. $Rel_{\mathrm{FOL}} \stackrel{df}{\equiv} R_{\mathrm{DG}} \cup S_{\mathrm{DG}}$
c. $Ar_{\mathrm{FOL}} \stackrel{df}{\equiv} (A_{\mathrm{DG}} \times \{1\}) \cup Ar_{\mathrm{DG}} \cup (S_{\mathrm{DG}} \times \{1\})$

That is, FOL functions are the $\mathrm{L_{DG}}$ attributes, FOL relations are the $\mathrm{L_{DG}}$ relations and sorts, and FOL arities are the same as $\mathrm{L_{DG}}$ arities defined for $\mathrm{L_{DG}}$ relations, as well as arity 1 assigned to $\mathrm{L_{DG}}$ sorts now interpreted as relations and to $\mathrm{L_{DG}}$ attributes interpreted as functions. (See (C13)–(C14) below for an illustration.)

This translation preserves all non-logical symbols, but loses information about the partial order of sorts and about attribute appropriateness conditions, so this information needs to be added explicitly via formulas; this is explicated in §B.3 below.

B.2 Syntax

$\mathrm{L_{DG}}$ formulae, defined in Definition 4 above, differ from standard FOL formulae only by the presence of special atomic formulae $\tau : \sigma$ and by the look of terms defined in Definition 3. Hence, the syntax translation function Tr from $\mathrm{L_{DG}}$ to FOL may be defined as follows (using $\rightsquigarrow$ instead of $=$, already used in formulae, to avoid confusion):

(B3) translation of terms:
a. for each $x \in V$, $Tr(x) \rightsquigarrow x$,
b. for each $\alpha \in A$ and each $\tau \in T^\Sigma$, $Tr(\tau\,\alpha) \rightsquigarrow \alpha(Tr(\tau))$.

(B4) translation of formulae:
a. for each $\sigma \in S$, for each $\tau \in T^\Sigma$, $Tr(\tau : \sigma) \rightsquigarrow \sigma(Tr(\tau))$,
b. for each $\tau_1, \tau_2 \in T^\Sigma$, $Tr(\tau_1 = \tau_2) \rightsquigarrow Tr(\tau_1) = Tr(\tau_2)$,
c. for each $\rho \in R$, for each $\tau_1, \ldots, \tau_{Ar(\rho)} \in T^\Sigma$, $Tr(\rho(\tau_1, \ldots, \tau_{Ar(\rho)})) \rightsquigarrow \rho(Tr(\tau_1), \ldots, Tr(\tau_{Ar(\rho)}))$,
d. for each $x \in V$, for each $\delta \in D^\Sigma$, $Tr(\exists x.\delta) \rightsquigarrow \exists x.\, Tr(\delta)$, (also $\forall$)
e. for each $\delta \in D^\Sigma$, $Tr(\neg\delta) \rightsquigarrow \neg\, Tr(\delta)$,
f. for each $\delta_1, \delta_2 \in D^\Sigma$, $Tr((\delta_1 \wedge \delta_2)) \rightsquigarrow (Tr(\delta_1) \wedge Tr(\delta_2))$. (also $\vee, \rightarrow, \leftrightarrow$)

For example, the "First Valents of Verbs are Subjects" Logical-level statement in (91a), repeated below as (B5), may be translated into FOL as in (B6) (keeping font conventions).

(B5) First Valents of Verbs are Subjects (L_{DG})
$\forall x.\forall y.\,(x\ \textsc{cat} : \mathit{verb} \wedge x\ \textsc{val}\ \textsc{first} = y) \rightarrow y\ \textsc{rel} : \mathit{subj}$

(B6) First Valents of Verbs are Subjects (FOL)
$\forall x.\forall y.\,(\mathit{verb}(\textsc{cat}(x)) \wedge \textsc{first}(\textsc{val}(x)) = y) \rightarrow \mathit{subj}(\textsc{rel}(y))$

B.3 Semantics

The interpretation of FOL formulae translated from L_{DG} formulae as sketched above is standard. However, in order for the FOL theory to be satisfied by the same models as the corresponding L_{DG} theory, it must be extended with formulae simulating the effects of the L_{DG} signature which were lost in the translation above, namely, that:

(B7) a. each model object is of exactly one minimal sort,
b. if an object is of a certain non-maximal sort, then it must be of its immediate supersort,
c. partial functions corresponding to particular attributes must be constrained as prescribed by the L_{DG} signature (more on this below).

As to (B7a), assuming that minimal sorts are $S_{min} = \{s_1^m, s_2^m, s_3^m, \ldots, s_k^m\}$, the following formula should be part of the theory:

(B8) $\forall x.\,(s_1^m(x) \wedge \neg s_2^m(x) \wedge \neg s_3^m(x) \wedge \ldots \wedge \neg s_k^m(x)) \vee$
$(\neg s_1^m(x) \wedge s_2^m(x) \wedge \neg s_3^m(x) \wedge \ldots \wedge \neg s_k^m(x)) \vee$
$(\neg s_1^m(x) \wedge \neg s_2^m(x) \wedge s_3^m(x) \wedge \ldots \wedge \neg s_k(x)) \vee$
$\ldots \vee$
$(\neg s_1^m(x) \wedge \neg s_2^m(x) \wedge \neg s_3^m(x) \wedge \ldots \wedge s_k^m(x))$

As to (B7b), at least one formula should be added for each sort other than $\top$. Assuming that such a non-maximal sort s_1 has an immediate supersort s_2, the formula should be:[66]

(B9) $\forall x.\, s_1(x) \rightarrow s_2(x)$

As to (B7c), appropriateness conditions say which attributes are defined for which sorts, and what appropriate values of such attributes are. This translates into a single formula for any ⟨sort, attribute⟩ pair $\langle s, a\rangle$ for which F is defined. If, according to the L_{DG} signature, $F(s, a)$ is defined and its value is s', then the following formula needs to be added to the theory:

(B10) $\forall x.\, s(x) \rightarrow s'(a(x))$

Moreover, if $\{s_1^a, \ldots, s_n^a\}$ is the set of such sorts s that $F(s, a)$ is defined, then the following formula should be added:[67]

(B11) $\forall x.\,(\exists y.\, a(x) = y) \rightarrow (s_1^a(x) \vee \ldots \vee s_n^a(x))$

While this translation procedure from L_{DG} to FOL is cumbersome, its sole aim is to demonstrate that sort hierarchies and attribute specifications assumed in L_{DG} signatures do not extend the underlying logic beyond that of FOL.

---

[66] Note that nothing in the definition of L_{DG} signatures precludes the possibility of "multiple inheritance", i.e., of a given sort having more than one immediate supersort. Within HPSG, this possibility is crucially taken advantage of in some analyses, e.g., the analysis of gerunds in Malouf 1998. Formally, s_2 is an immediate supersort of s_1 iff $s_1 \sqsubsetneq s_2 \wedge \neg\exists s_3.\, s_1 \sqsubsetneq s_3 \wedge s_3 \sqsubsetneq s_2$ (where, in turn, $s_1 \sqsubsetneq s_2 \stackrel{df}{\equiv} s_1 \sqsubseteq s_2 \wedge s_1 \neq s_2$).

[67] In practice, a much simpler formula mentioning only the *maximal* sorts appropriate for the attribute would suffice. For example, assuming the sort hierarchy in (37)–(38) in the main text, (i) below could be simplified to (ii).

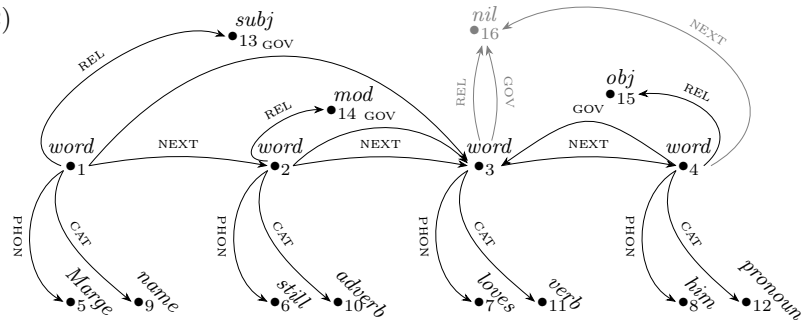
(i) $\forall x.\,(\exists y.\,\textsc{num}(x) = y) \rightarrow (\mathit{noun}(x) \vee \mathit{name}(x) \vee \mathit{common}(x) \vee \mathit{count}(x) \vee \mathit{mass}(x) \vee \mathit{pronoun}(x) \vee \mathit{verb}(x) \vee \mathit{auxiliary}(x) \vee \mathit{modal}(x) \vee \mathit{be}(x) \vee \mathit{do}(x) \vee \ldots \vee \mathit{lverb}(x) \vee \mathit{determiner}(x))$

(ii) $\forall x.\,(\exists y.\,\textsc{num}(x) = y) \rightarrow (\mathit{noun}(x) \vee \mathit{verb}(x) \vee \mathit{determiner}(x))$

Appendix C Formal model

Formally, the FOL model visualized in §3.1 as (23), repeated below as (C12) with additional numbering of objects added, is the triple $M = \langle D, \Sigma, I \rangle$ in (C13) such that D is the domain, Σ is the FOL signature, and I is the interpretation function.

(C12)



(C13) $M = \langle D, \Sigma, I \rangle$, where:

a. $D = \{\bullet_1, \bullet_2, \ldots, \bullet_{16}\}$

b. $\Sigma = \langle F, R, A \rangle$, where:

i. $F = \{\text{PHON}, \text{CAT}, \text{REL}, \text{GOV}, \text{NEXT}\}$

ii. $R = \{\textit{word}, \textit{name}, \textit{adverb}, \textit{verb}, \textit{pronoun}, \textit{subj}, \textit{obj}, \textit{mod}, \textit{Marge}, \textit{still}, \textit{loves}, \textit{him}, \textit{nil}\}$

iii. $A = \{\langle \text{PHON}, 1\rangle, \langle \text{CAT}, 1\rangle, \ldots, \langle \text{NEXT}, 1\rangle, \langle \textit{word}, 1\rangle, \langle \textit{name}, 1\rangle, \ldots, \langle \textit{nil}, 1\rangle\}$

c. $I = \{\langle \text{PHON}, \{\langle \bullet_1, \bullet_5\rangle, \langle \bullet_2, \bullet_6\rangle, \langle \bullet_3, \bullet_7\rangle, \langle \bullet_4, \bullet_8\rangle\}\rangle,$
$\langle \text{CAT}, \{\langle \bullet_1, \bullet_9\rangle, \langle \bullet_2, \bullet_{10}\rangle, \langle \bullet_3, \bullet_{11}\rangle, \langle \bullet_4, \bullet_{12}\rangle\}\rangle,$
$\ldots,$
$\langle \text{NEXT}, \{\langle \bullet_1, \bullet_2\rangle, \langle \bullet_2, \bullet_3\rangle, \langle \bullet_3, \bullet_4\rangle, \langle \bullet_4, \bullet_{16}\rangle\}\rangle,$
$\langle \textit{word}, \{\bullet_1, \bullet_2, \bullet_3, \bullet_4\}\rangle,$
$\langle \textit{name}, \{\bullet_9\}\rangle,$
$\ldots,$
$\langle \textit{nil}, \{\bullet_{16}\}\rangle\}$

The domain of the model, defined in (C13a), is the set of objects in the model. The FOL signature, defined in (C13b), is a triple consisting of function symbols, relation symbols, and a function assigning arities to both kinds of symbols; here, all functions and relations are monadic. Finally, the interpretation function, defined in (C13c), assigns partial functions (i.e., appropriate sets of pairs) to function symbols and sets of objects to unary predicate symbols.

If this model corresponded to an L_{DG} theory with an L_{DG} signature (hierarchy of sorts, as in (24) in §3.2) and used additional relation symbols (e.g., `precede` defined in (26) there), then the this FOL signature and interpretation would have to be extended correspondingly, as defined in Appendix B: the relation symbols R in the signature would also need to contain $\top$, *category*, *relation*, etc. (all of arity 1), as well as `precede` (of arity 2), etc., and the interpretation function I would have to provide the appropriate interpretation of these symbols, i.e., it would also have to contain pairs such as in (C14).

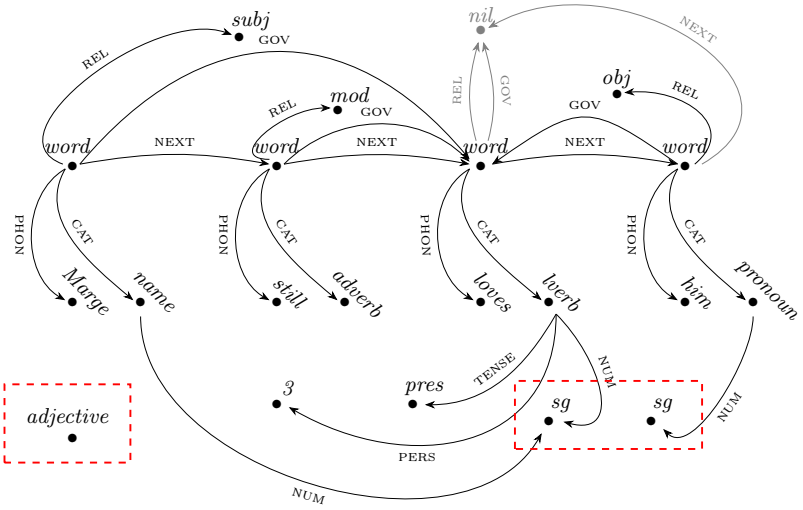
(C14) $I = \{\ldots,$
$\langle \top, \{\bullet_1, \bullet_2, \ldots, \bullet_{16}\}\rangle,$
$\langle \textit{category}, \{\bullet_9, \bullet_{10}, \bullet_{11}, \bullet_{12}\}\rangle,$
$\langle \textit{relation}, \{\bullet_{13}, \bullet_{14}, \bullet_{15}\}\rangle,$
$\langle \texttt{precede}, \{\langle \bullet_1, \bullet_2\rangle, \langle \bullet_2, \bullet_3\rangle, \langle \bullet_3, \bullet_4\rangle\}\rangle,$
$\ldots\}$

Appendix D Model constraints

Apart from framework constraints, defining the general topology of structures assumed in a given dependency approach (exemplified in §3.2), and actual linguistic constraints (in §4), it makes sense to formulate some axioms that are even more abstract than framework constraints, in order to ensure the minimality of models.

Consider the following model (D15), which is similar to that in (41) in §4.1, but lacks that model's minimality property.

(D15)



There are two problems with this model, signalled with red dashed squares. First, there is a spurious object of sort *adjective*, which is not integrated with the actual structure of the sentence *Marge still loves him*, but also does not violate any of the constraints introduced in the main text. (In particular, it does not follow from anything so far that objects of sort *category* are allowed in the model only when they are values of the CAT attribute.) Second, there are two objects representing the singular number, while one would suffice for the analysis.

Analogous potential problems with HPSG models were discussed in Richter 2007, and their solutions assuming lean models similar to those assumed here were proposed in Przepiórkowski 2021; in what follows, we port these solutions to MTDG.

The first problem, with non-integrated objects, is solved by the following requirement that all objects are connected by attributes:[68]

(D16) Component Principle

a. 'There is an object from which all model objects can be reached via attributes.'

b. $\exists x.\forall y.\,\texttt{accessible\_from}(y,\,x)$

[68] The relation `accessible_from` in (D17) is based on the relation `component` in Sailer 2003: 115–116. Technically speaking, there are two problems with the constraint in (D16)–(D17). First, as is well known, graph connectivity cannot be expressed in FOL *in full generality*, i.e., allowing for models of any cardinality. However, we assume that only finite models of utterances are of interests to linguists. Second, even when considering only finite models, the definition of `accessible_from` in (D17) does not work as intended when cycles in the graph defined by attributes are allowed. In the models assumed here such cycles are in fact allowed, e.g., see (C12) and the cycle from $\bullet_3$ via NEXT to $\bullet_4$ and back to $\bullet_3$ via GOV. This problem can be solved by replacing "$a \in A$" with "$a \in A\backslash\{\textsc{gov}\}$", i.e., by excluding GOV from the attributes considered in the definition of components. This is harmless given the kinds of structures defined by other formulae, because GOV can only lead from *word* objects to *word* objects, and all *word* objects can be reached from the first one via NEXT anyway. (Further modifications of this constraint are needed in the extended signature assumed in the main text, where values of VAL may also introduce cycles.)

(D17) a. 'An object y is accessible from an object x iff they are in fact the same object or y is accessible from some object accessible from x via an attribute.'

b. $\forall x.\forall y.\, \texttt{accessible\_from}(y,\, x) \;\leftrightarrow$
$(y = x \;\vee \bigvee_{\text{A} \in A} \texttt{accessible\_from}(y,\, x\ \text{A}))$

In the last formula, $\bigvee_{\text{A} \in A} \texttt{accessible\_from}(y,\, x\ \text{A})$ is a disjunction over attributes A as defined in the L_{DG} signature (see Appendix A.1 above). For example, if $A = \{\text{PHON}, \text{CAT}, \text{REL}, \text{GOV}, \text{NEXT}\}$, then this subformula is defined as in (D18).

(D18) $\bigvee_{\text{A} \in A} \texttt{accessible\_from}(y,\, x\ \text{A}) \;\stackrel{df}{\equiv}$
$\texttt{accessible\_from}(y,\, x\ \text{PHON}) \vee \texttt{accessible\_from}(y,\, x\ \text{CAT}) \vee$
$\texttt{accessible\_from}(y,\, x\ \text{REL}) \vee \texttt{accessible\_from}(y,\, x\ \text{GOV}) \vee$
$\texttt{accessible\_from}(y,\, x\ \text{NEXT})$

Note that the model in (D15) does not satisfy the Component Principle (D16) as the object of sort *adjective* is not accessible from any other object.

The second problem is solved with Sailer's (2003: 116) General Identity Principle, which requires that objects that "look the same" (formally `are_copies`) are in fact the same.

(D19) General Identity Principle

a. 'If two objects look the same, then they are the same object.'

b. $\forall x.\forall y.\, \texttt{are\_copies}(x,\, y) \rightarrow x = y$

(D20) a. 'Two objects look the same iff they are of the same species and have the same values of corresponding attributes.'

b. $\forall x.\forall y.\, \texttt{are\_copies}(x,\, y) \;\leftrightarrow$
$\left(\bigvee_{s \in S} (x : s \wedge y : s) \wedge \bigwedge_{\text{A} \in A} ((\exists z.\, x\ \text{A} = z) \rightarrow \texttt{are\_copies}(x\ \text{A},\, y\ \text{A})\right)$

In the last formula, S refers to the set of species assumed in the L_{DG} signature, for example, the leaves of the sort hierarchy in (24) in the main text, and A – to the set of attributes. Hence, two conditions have to be met for two objects to be copies. First, they must have the same species (either both are of species *word*, or both are of species *name*, or...). Second, for each attribute, if that attribute is present on both objects (given that these objects are of the same species, it must be present on both or on neither, so it is sufficient to check its presence on one), the two values of this attribute must be copies.

Returning to the model in (D15), the two objects of sort *sg* look the same (they have the same species, *sg*, and the same values of all – zero – corresponding attributes), so they should be the same object according to the General Identity Principle (D19), but they are not. In contrast, the original model in (41) in §4.1 satisfies both constraints.

References

Adger, David. 2003. *Core syntax: A minimalist approach*. Oxford: Oxford University Press.

Ajdukiewicz, Kazimierz. 1935. Die syntaktische Konnexität. *Studia Philosophica* 1:1–27. English translation in Storrs McCall, editor (1967), *Polish Logic: 1920–1939*, pp. 207–231, Oxford University Press, Oxford.

Apresjan, Jurij D., Igor M. Boguslavsky, Leonid L. Iomdin, Alexander V. Lazursky, Vladimir Z. Sannikov, Victor G. Sizov, and Leonid L. Tsinman. 2003. ETAP-3 linguistic processor: A full-fledged NLP implementation of the Meaning ⇔ Text Theory. In *First International Conference on Meaning–Text Theory (MTT'2003)*, 279–288. URL http://cl.iitp.ru/bibitems/ETAP_and_MTT.pdf.

Arnold, Doug, Miriam Butt, Berthold Crysmann, Tracy Holloway King, and Stefan Müller, ed. by. 2016. *Proceedings of the joint 2016 conference on Head-driven Phrase Structure Grammar and Lexical Functional Grammar*. Stanford, CA: CSLI Publications. URL https://proceedings.hpsg.xyz/issue/view/27.

Babby, Leonard H. 2009. *The syntax of argument structure*. Cambridge: Cambridge University Press.

Bejček, Eduard, Eva Hajičová, Jan Hajič, Pavlína Jínová, Václava Kettnerová, Veronika Kolářová, Marie Mikulová, Jiří Mírovský, Anna Nedoluzhko, Jarmila Panevová, Lucie Poláková, Magda Ševčíková, Jan Štěpánek, and Šárka Zikánová. 2013. Prague Dependency Treebank 3.0.

Bender, Emily M., Dan Flickinger, and Stephan Oepen. 2002. The Grammar Matrix: An open-source starter-kit for the rapid development of cross-linguistically consistent broad-coverage precision grammars. In *Proceedings of the Workshop on Grammar Engineering and Evaluation at the 19th International Conference on Computational Linguistics*, ed. by John Carroll, Nelleke Oostdijk, and Richard Sutcliffe, 8–14. Taipei, Taiwan.

Beuck, Niels, Arne Köhn, and Wolfgang Menzel. 2011. Incremental parsing and the evaluation of partial dependency analyses. In *Proceedings of the International Conference on Dependency Linguistics (Depling 2011)*, ed. by Kim Gerdes, Eva Hajičová, and Leo Wanner, 290–299. Barcelona, Spain. URL https://depling.org/proceedingsDepling2011/papers/proceedingsDepling2011.pdf.

Boas, Hans, and Ivan A. Sag, ed. by. 2012. *Sign-based construction grammar*. Stanford, CA: CSLI Publications.

Boeckx, Cedric, ed. by. 2011. *The oxford handbook of linguistic minimalism*. Oxford: Oxford University Press.

Borsley, Robert D., and Adam Przepiórkowski, ed. by. 1999. *Slavic in Head-driven Phrase Structure Grammar*. Stanford, CA: CSLI Publications. URL http://csli-publications.stanford.edu/site/1575861747.shtml.

Bouma, Gosse, Robert Malouf, and Ivan A. Sag. 2001. Satisfying constraints on extraction and adjunction. *Natural Language and Linguistic Theory* 19:1–65.

Bresnan, Joan. 1978. A realistic Transformational Grammar. In *Linguistic theory and psychological reality*, ed. by Morris Halle, Joan Bresnan, and George A. Miller, 1–59. Cambridge, MA: MIT Press.

Bresnan, Joan, ed. by. 1982a. *The mental representation of grammatical relations*. Cambridge, MA: MIT Press.

Bresnan, Joan. 1982b. The passive in lexical theory. In Bresnan (1982a), 3–86.

Bresnan, Joan, Ash Asudeh, Ida Toivonen, and Stephen Wechsler. 2016. *Lexical-functional syntax*. Blackwell Textbooks in Linguistics. Wiley-Blackwell, 2nd edition.

Bruening, Benjamin. 2010. Ditransitive asymmetries and a theory of idiom formation. *Linguistic Inquiry* 41:519–562.

Butt, Miriam, Helge Dyvik, Tracy Holloway King, Hiroshi Masuichi, and Christian Rohrer. 2002. The parallel grammar project. In *Proceedings of the COLING 2002 Workshop on Grammar Engineering and Evaluation*, 1–7. Taipei.

By, Tomas. 2004. English dependency grammar. In *Proceedings of the Workshop on Recent Advances in Dependency Grammar*, 64–69. Geneva, Switzerland: COLING. URL https://aclanthology.org/W04-1509/.

Börjars, Kersti, Rachel Nordlinger, and Louisa Sadler. 2019. *Lexical-Functional Grammar: An introduction*. Cambridge: Cambridge University Press.

Carnie, Andrew. 2013. *Syntax: A generative introduction*. Oxford: Blackwell, 3rd edition.

Chomsky, Noam. 1957. *Syntactic structures*. The Hague: Mouton.

Chomsky, Noam. 1995. *The minimalist program*. Cambridge, MA: MIT Press.

Chomsky, Noam. 2000. Minimalist inquiries: The framework. In *Step by step*, ed. by Roger Martin, David Michaels, and Juan Uriagereka, 89–155. Cambridge, MA: MIT Press.

Copestake, Ann. 2002. *Implementing typed feature structure grammars*. Stanford, CA: CSLI Publications.

Croft, William. 2001. *Radical construction grammar: Syntactic theory in typological perspective*. Oxford: Oxford University Press.

Crouch, Dick, Mary Dalrymple, Ron Kaplan, Tracy King, John Maxwell, and Paula Newman. 2011. XLE documentation. https://ling.sprachwiss.uni-konstanz.de/pages/xle/doc/xle_toc.html.

Culicover, Peter W., and Ray Jackendoff. 2005. *Simpler Syntax*. Oxford University Press.

Dalrymple, Mary, ed. by. 2023. *Handbook of Lexical Functional Grammar*. Berlin: Language Science Press. URL https://langsci-press.org/catalog/book/312.

Dalrymple, Mary, John J. Lowe, and Louise Mycock. 2019. *The Oxford reference guide to Lexical Functional Grammar*. Oxford: Oxford University Press.

Davis, Anthony, and Jean-Pierre Koenig. 2024. The nature and role of the lexicon in HPSG. In Müller et al. (2024), 133–184. URL https://langsci-press.org/catalog/book/478.

Davis, Anthony, Jean-Pierre Koenig, and Stephen Wechsler. 2024. Argument structure and linking. In Müller et al. (2024), 335–390. URL https://langsci-press.org/catalog/book/478.

Debusmann, Ralph, Denys Duchier, and Geert-Jan M. Kruijff. 2004. Extensible dependency grammar: A new methodology. In *Proceedings of the Workshop on Recent Advances in Dependency Grammar*, 70–76. Geneva, Switzerland: COLING. URL https://aclanthology.org/W04-1510/.

Dekhtyar, Michael, Alexander Dikovsky, and Boris Karlov. 2015. Categorial dependency grammars. *Theoretical Computer Science* 579:33–63.

Ferrer-I-Cancho, Ramon, and Carlos Gómez-Rodríguez. 2016. Crossings as a side effect of dependency lengths. *Complexity* 21:320–328. URL https://onlinelibrary.wiley.com/doi/abs/10.1002/cplx.21810.

Findlay, Jamie Y., Roxanne Taylor, and Anna Kibort. 2023. Argument structure and mapping theory. In Dalrymple (2023), 699–778. URL https://langsci-press.org/catalog/book/312.

Foth, Kilian, Tomas By, and Wolfgang Menzel. 2006. Guiding a constraint dependency parser with supertags. In *Proceedings of the 21st International Conference on Computational Linguistics and 44th Annual Meeting of the Association for Computational Linguistics*, 289–296. Sydney, Australia.

Gaifman, Haim. 1965. Dependency systems and phrase-structure systems. *Information and Control* 8:304–337.

Gazdar, Gerald, Ewan Klein, Goeffrey K. Pullum, and Ivan A. Sag. 1985. *Generalized Phrase Structure Grammar*. Cambridge / Cambridge, MA: Blackwell / Harvard University Press.

Gibson, Edward. 2025. *Syntax: A cognitive approach*. Cambridge, MA: MIT Press.

Gibson, Edward. 2026. Dependency syntax as the simplest theory of grammar. *Trends in Cognitive Sciences* xx:1–13.

Goldberg, Adele E. 1995. *Constructions: A construction grammar approach to argument structure*. Chicago, IL: Chicago University Press.

Groß, Thomas, and Timothy Osborne. 2009. Toward a practical dependency grammar theory of discontinuities. *SKY Journal of Linguistics* 22:43–90. URL https://journal.fi/finjol/article/view/152927.

Haegeman, Liliane, and Jacqueline Guéron. 1999. *English grammar: A generative perspective*. Oxford: Blackwell.

Haspelmath, Martin. 2007. Coordination. In *Language typology and syntactic description. Volume II: Complex constructions*, ed. by Timothy Shopen, 1–51. Cambridge: Cambridge University Press, 2nd edition.

Hays, David G. 1964. Dependency theory: A formalism and some observations. *Language* 40:511–525.

Hewitt, John, and Christopher D. Manning. 2019. A structural probe for finding syntax in word representations. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, ed. by Jill Burstein, Christy Doran, and Thamar Solorio, 4129–4138. Minneapolis, Minnesota: Association for Computational Linguistics. URL https://aclanthology.org/N19-1419/.

Hromei, Claudiu Daniel, Danilo Croce, and Roberto Basili. 2024. `U-DepPLLaMA`: Universal Dependency parsing via auto-regressive large language models. *Italian Journal of Computational Linguistics* 10:21–38.

Hudson, Richard. 1984. *Word grammar*. Oxford: Blackwell.

Hudson, Richard. 1990. *English Word Grammar*. Oxford: Blackwell.

Hudson, Richard. 2007. *Language networks: The new Word Grammar*. Oxford: Oxford University Press.

Hudson, Richard. 2010. *An introduction to Word Grammar*. Cambridge: Cambridge University Press.

Hudson, Richard. 2024. HPSG and Dependency Grammar. In Müller et al. (2024), 1531–1580. URL https://langsci-press.org/catalog/book/478.

Hudson, Richard, Nikolas Gisborne, Thomas Hikaru Clark, Eva Duran Eppler, Willem Hollmann, Andrew Rosta, and Graeme Trousdale. 2026. The syntactic constraint on English auxiliary contraction. *Journal of Linguistics* 62:357–388.

Höhle, Tilman N. 1994. Spurenlose extraction. Unpubl. ms., University of Tübingen, reprinted in Müller et al. 2019.

Höhle, Tilman N. 1999. An architecture for phonology. In Borsley and Przepiórkowski (1999), 61–90. URL http://csli-publications.stanford.edu/site/1575861747.shtml, reprinted in Müller et al. 2019.

Jacobson, Pauline. 1999. Towards a variable-free semantics. *Linguistics and Philosophy* 22:117–184.

Johnson, Mark. 1994. Two ways of formalizing grammars. *Linguistics and Philosophy* 17:221–248.

Joshi, Aravind K. 1985. Tree adjoining grammars: How much context sensitivity is required to provide reasonable structural descriptions? In *Natural language parsing: Psychological, computational, and theoretical perspectives*, ed. by David Dowty, Lauri Karttunen, and Arnold M. Zwicky. Cambridge: Cambridge University Press.

Joshi, Aravind K. 1987. An introduction to Tree Adjoining Grammars. In *Mathematics of language*, ed. by Alexis Manaster-Ramer. Amsterdam: John Benjamins.

Jumelet, Jaap, Leonie Weissweiler, Joakim Nivre, and Arianna Bisazza. 2026. MultiBLiMP 1.0: A massively multilingual benchmark of linguistic minimal pairs. *Transactions of the Association for Computational Linguistics* 14:193–216. URL https://aclanthology.org/2026.tacl-1.10/.

Kahane, Sylvain. 1997. Bubble trees and syntactic representations. In *Proceedings of Mathematics of Language 5*, ed. by Tilman Becker and Hans-Ulrich Krieger, 70–76. DFKI, Saarbrücken.

Kaplan, Ronald M. 1995. The formal architecture of Lexical-Functional Grammar. In *Formal issues in Lexical-Functional Grammar*, ed. by Mary Dalrymple, Ronald M. Kaplan, John T. Maxwell, III, and Annie Zaenen, 7–27. Stanford, CA: CSLI Publications.

Kaplan, Ronald M. 2017. Preserving grammatical functions in LFG. In *The very model of a modern linguist*, ed. by Victoria Rosén and Koenraad De Smedt, number 8 in Bergen Language and Linguistics Studies, 127–142. Bergen: University of Bergen Library. URL https://bells.uib.no/index.php/bells/article/view/1342.

Kaplan, Ronald M., and Joan Bresnan. 1982. Lexical-Functional Grammar: A formal system for grammatical representation. In Bresnan (1982a), 173–281.

Kay, Paul. 1998. An informal sketch of a formal architecture for construction grammar. In *Proceedings of the Joint Conference on Formal Grammar, Head-Driven Phrase Structure Grammar, and Categorial Grammar*, ed. by Gosse Bouma, Geert-Jan M. Kruijff, and Richard T. Oehrle, 175–184. Saarbrücken: Universität des Saarlandes.

Kay, Paul, and Charles J. Fillmore. 1997. Grammatical constructions and linguistic generalizations: the *What's X Doing Y?* construction. *Language* 75:1–33.

Kilgarriff, Adam, Vít Baisa, Jan Bušta, Miloš Jakubíček, Vojtěch Kovář, Jan Michelfeit, Pavel Rychlý, and Vít Suchomel. 2014. The Sketch Engine: Ten years on. *Lexicography* 1:7–36.

Kilgarriff, Adam, Pavel Rychlý, Pavel Smrž, and David Tugwell. 2008. The Sketch Engine. In *Practical lexicography. A reader*, ed. by Thierry Fontenelle, 297–306. Oxford: Oxford University Press.

King, Paul John. 1989. A logical formalism for Head-driven Phrase Structure Grammar. Ph.D. dissertation, University of Manchester.

King, Paul John. 1994. An expanded logical formalism for Head-driven Phrase Structure Grammar. Arbeitspapiere des Sonderforschungsbereichs 340 Bericht Nr. 59, Seminar für Sprachwissenschaft, Universität Tübingen.

King, Paul John. 1999. Towards truth in HPSG. In Kordoni (1999), 301–352.

Kogkalidis, Konstantinos. 2023. Dependency as modality, parsing as permutation: A neurosymbolic perspective on categorial grammars. Ph.D. dissertation, Utrecht University.

Köhn, Arne, and Wolfgang Menzel. 2013. Incremental and predictive dependency parsing under real-time conditions. In *Proceedings of the International Conference Recent Advances in Natural Language Processing RANLP 2013*, ed. by Ruslan Mitkov, Galia Angelova, and Kalina Bontcheva, 373–381. Hissar, Bulgaria: INCOMA Ltd. Shoumen, BULGARIA. URL https://aclanthology.org/R13-1048/.

Kordoni, Valia, ed. by. 1999. *Tübingen studies in Head-driven Phrase Structure Grammar*. Arbeitspapiere des Sonderforschungsbereichs 340, Bericht Nr. 132. Tübingen: Universität Tübingen.

Kuhlmann, Marco. 2013. Mildly non-projective dependncy grammar. *Computational Linguistics* 30:355–387.

Kübler, Sandra, Ryan McDonald, and Joakim Nivre. 2009. *Dependency parsing*. Morgan & Claypool.

Lahm, David. 2016. Refining the semantics of lexical rules in HPSG. In Arnold et al. (2016), 360–379. URL https://proceedings.hpsg.xyz/issue/view/27.

Lambek, Joachim. 1958. The mathematics of sentence structure. *American Mathematical Monthly* 65:154–170.

Lewis, Mike, and Mark Steedman. 2014. A* CCG parsing with a supertag-factored model. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, ed. by Alessandro Moschitti, Bo Pang, and Walter Daelemans, 990–1000. Doha, Qatar: Association for Computational Linguistics. URL https://aclanthology.org/D14-1107/.

Link, Godehard. 1983. The logical analysis of plurals and mass terms. In *Meaning, use and interpretation of language*, ed. by Rainer Bäuerle, Christoph Schwarze, and Arnim von Stechow, 302–323. Berlin: Walter de Gruyter.

Malouf, Robert. 1998. Mixed categories in the hierarchical lexicon. Doctoral Dissertation, Stanford University.

de Marneffe, Marie-Catherine, Christopher D. Manning, Joakim Nivre, and Daniel Zeman. 2021. Universal Dependencies. *Computational Linguistics* 47:255–308. URL https://doi.org/10.1162/coli_a_00402.

de Marneffe, Marie-Catherine, and Joakim Nivre. 2019. Dependency Grammar. *Annual Review of Linguistics* 5:197–218.

Maruyama, Hiroshi. 1990. Structural disambiguation with constraint propagation. In *28th Annual Meeting of the Association for Computational Linguistics*, 31–38. Pittsburgh, Pennsylvania, USA: Association for Computational Linguistics. URL https://aclanthology.org/P90-1005/.

McCawley, James D. 1968. Concerning the base component of a transformational grammar. *Foundations of Language* 4:243–269.

Mel'čuk, Igor. 1988. *Dependency syntax: Theory and practice*. Albany, NY: The SUNY Press.

Mel'čuk, Igor. 2006. Explanatory combinatorial dictionary. In *Open problems in linguistic and lexicography*, ed. by Giandomenico Sica, 225–355. Monza: Polimetrica.

Mel'čuk, Igor. 2009. Dependency in natural language. In Polguère and Mel'čuk (2009), 1–110.

Mel'čuk, Igor. 2021. *Ten studies in dependency syntax*. Berlin / Boston: De Gruyter Mouton.

Mel'čuk, Igor, and Alexander Zholkovsky. 1984. *Explanatory combinatorial dictionary of modern Russian. Semantico-syntactic studies of Russian vocabulary*. Vienna: Wiener Slawistischer Almanach.

Meurers, W. Detmar. 2001. On expressing lexical generalizations in HPSG. *Nordic Journal of Linguistics* 24:161–217.

Meurers, W. Detmar, Gerald Penn, and Frank Richter. 2002. A web-based instructional platform for contraint-based grammar formalisms and parsing. In *Proceedings of the ACL-02 Workshop on Effective Tools and Methodologies for Teaching Natural Language Processing and Computational Linguistics*, 19–26. Philadelphia, PA: Association for Computational Linguistics. URL https://aclanthology.org/W02-0103/.

Miller, Philip H. 1999. *Strong generative capacity: The semantics of linguistic formalism*. Stanford, CA: CSLI Publications.

Moot, Richard. 2023. Do model-theoretic grammars have fundamental advantages over proof-theoretic grammars? https://hal.science/lirmm-04285668/.

Moot, Richard, and Christian Retoré. 2012. *The logic of categorial grammars: A deductive account of natural language syntax and semantics*. Number 6850 in Lecture Notes in Computer Science. Berlin / Heidelberg: Springer-Verlag.

Müller, Stefan. 2015. The CoreGram project: Theoretical linguistics, theory development and verification. *Journal of Language Modelling* 3:21–86.

Müller, Stefan. 2023. *Grammatical theory: From Transformational Grammar to constraint-based approaches*. Berlin: Language Science Press, 5th edition. URL http://langsci-press.org/catalog/book/380.

Müller, Stefan. 2026. Dependency or phrase structure grammars. Unpublished manuscript, https://hpsg.hu-berlin.de/~stefan/Pub/dg-vs-psg.html, version of 27 May 2026.

Müller, Stefan, Anne Abeillé, Robert D. Borsley, and Jean-Pierre Koenig, ed. by. 2024. *Head-driven Phrase Structure Grammar: The handbook*. Berlin: Language Science Press, 2nd edition. URL https://langsci-press.org/catalog/book/478.

Müller, Stefan, Marga Reis, and Frank Richter, ed. by. 2019. *Beiträge zur deutschen Grammatik: Gesammelte Schriften von Tilman N. Höhle*. Berlin: Language Science Press.

Nefdt, Ryan M. 2024. *The philosophy of theoretical linguistics: A contemporary outlook*. Cambridge: Cambridge University Press.

Nefdt, Ryan M., and Giosué Baggio. 2024. Notational variants and cognition: The case of dependency grammar. *Erkenntnis* 89:2867–2897.

Nguyen, Minh Van, Viet Lai, Amir Pouran Ben Veyseh, and Thien Huu Nguyen. 2021. Trankit: A light-weight transformer-based toolkit for multilingual natural language processing. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*.

Osborne, Timothy. 2006. Shared material and grammar: Toward a Dependency Grammar theory of non-gapping coordination for English and German. *Zeitschrift für Sprachwissenschaft* 25:39–93.

Osborne, Timothy. 2008. Major constituents and two dependency grammar constraints on sharing in coordination. *Linguistics* 46:1109–1165.

Osborne, Timothy. 2019. *A dependency grammar of English: An introduction and beyond*. Amsterdam / Philadelphia: John Benjamins.

Osborne, Timothy. 2026. Syntactic predicates. *Journal of Linguistics* 62:389–423.

Osborne, Timothy, and Kim Gerdes. 2019. The status of function words in dependency grammar: A critique of Universal Dependencies (UD). *Glossa: A Journal of General Linguistics* 4.

Partee, Barbara H., Alice ter Meulen, and Robert E. Wall. 1993. *Mathematical methods in linguistics*. Dordrecht: Kluwer.

Patejuk, Agnieszka, and Adam Przepiórkowski. 2016. Reducing grammatical functions in Lexical Functional Grammar. In Arnold et al. (2016), 541–559. URL https://proceedings.hpsg.xyz/issue/view/27.

Patejuk, Agnieszka, and Adam Przepiórkowski. 2023. Category mismatches in coordination vindicated. *Linguistic Inquiry* 54:326–349.

Polguère, Alain, and Igor Mel'čuk, ed. by. 2009. *Dependency in linguistic description*. Amsterdam: John Benjamins.

Pollard, Carl, and Ivan A. Sag. 1994. *Head-driven Phrase Structure Grammar*. Chicago, IL: Chicago University Press / CSLI Publications.

Popel, Martin, David Mareček, Jan Štěpánek, Daniel Zeman, and Zděněk Žabokrtský. 2013. Coordination structures in dependency treebanks. In *Proceedings of the 51st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 517–527. Sofia, Bulgaria. URL http://www.aclweb.org/anthology/P13-1051.

Przepiórkowski, Adam. 1999a. Case assignment and the complement-adjunct dichotomy: A non-configurational constraint-based approach. Ph.D. dissertation, Universität Tübingen. URL http://nlp.ipipan.waw.pl/~adamp/Dissertation/.

Przepiórkowski, Adam. 1999b. On complements and adjuncts in Polish. In Borsley and Przepiórkowski (1999), 183–210. URL http://nlp.ipipan.waw.pl/~adamp/Papers/1999-adjuncts/.

Przepiórkowski, Adam. 2016. How *not* to distinguish arguments from adjuncts in LFG. In Arnold et al. (2016), 560–580. URL https://proceedings.hpsg.xyz/issue/view/27.

Przepiórkowski, Adam. 2021. Three improvements to the HPSG model theory. In *Proceedings of the HPSG 2021 Conference*, ed. by Stefan Müller and Nurit Melnik, 165–185. Frankfurt/Main University Library. URL https://proceedings.hpsg.xyz/.

Przepiórkowski, Adam. 2022. Coordination of unlike grammatical cases (and unlike categories). *Language* 98:592–634.

Przepiórkowski, Adam. 2023. LFG and HPSG. In Dalrymple (2023), 1861–1918. URL https://langsci-press.org/catalog/book/312.

Przepiórkowski, Adam, and Agnieszka Patejuk. 2018. Arguments and adjuncts in Universal Dependencies. In *Proceedings of the 27th International Conference on Computational Linguistics (COLING 2018)*, 3837–3852. Santa Fe, NM. URL https://aclanthology.org/C18-1324/, (Best position paper at COLING 2018).

Przepiórkowski, Adam, and Agnieszka Patejuk. 2019. Nested coordination in Universal Dependencies. In *Proceedings of the Third Workshop on Universal Dependencies (UDW, SyntaxFest 2019)*, ed. by Alexandre Rademaker and Francis Tyers, 58–69. Association for Computational Linguistics. URL https://aclweb.org/anthology/W19-8007/.

Przepiórkowski, Adam, and Agnieszka Patejuk. 2025. Prenominal adverbs, or apparent selectional violations in coordination. *Linguistic Inquiry* Early Access:1–29.

Przepiórkowski, Adam, and Michał Woźniak. 2023. Conjunct lengths in English, Dependency Length Minimization, and dependency structure of coordination. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ed. by Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, 15494–15512. Toronto, Canada: Association for Computational Linguistics. URL https://aclanthology.org/2023.acl-long.864.

Pullum, Geoffrey K., and Barbara C. Scholz. 2001. On the distinction between model-theoretic and generative-enumerative syntactic frameworks. In *Logical aspects of computational linguistics: 4th international conference*, ed. by Philippe de Groote, Glyn Morrill, and Christian Retoré, volume 2099 of *Lecture Notes in Artificial Intelligence*, 17–43. Berlin: Springer-Verlag.

Qi, Peng, Yuhao Zhang, Yuhui Zhang, Jason Bolton, and Christopher D. Manning. 2020. Stanza: A Python natural language processing toolkit for many human languages. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, 101–108. URL https://nlp.stanford.edu/pubs/qi2020stanza.pdf.

Radford, Andrew. 2004. *Minimalist syntax; exploring the structure of English*. Cambridge: Cambridge University Press.

Richter, Frank. 1999. RSRL for HPSG. In Kordoni (1999), 74–115.

Richter, Frank. 2004. A mathematical formalism for linguistic theories with an application in Head-driven Phrase Structure Grammar. Ph.D. dissertation (defended in 2000), Universität Tübingen.

Richter, Frank. 2007. Closer to the truth: A new model theory for HPSG. In Rogers and Kepser (2007), 99–108.

Richter, Frank. 2024. Formal background. In Müller et al. (2024), 93–131. URL https://langsci-press.org/catalog/book/478.

Robinson, Jane J. 1970. Dependency structures and transformational rules. *Language* 46:259–285.

Rogers, James. 1994. Studies in the logic of trees with applications to grammar formalisms. Ph.D. dissertation, University of Delaware.

Rogers, James. 1997. "Grammarless" phrase structure grammar. *Linguistics and Philosophy* 20:721–746.

Rogers, James, and Stephan Kepser, ed. by. 2007. *Model-theoretic syntax at 10*. European Summer School on Logic, Language and Information (ESSLLI 2007).

Sag, Ivan A., Gerald Gazdar, Thomas Wasow, and Steven Weisler. 1985. Coordination and how to distinguish categories. *Natural Language and Linguistic Theory* 3:117–171.

Sag, Ivan A., Thomas Wasow, and Emily M. Bender. 2003. *Syntactic theory: A formal introduction*. Stanford, CA: CSLI Publications, 2nd edition.

Sailer, Manfred. 2003. Combinatorial semantics and idiomatic expressions in Head-driven Phrase Structure Grammar. Ph.D. dissertation (defended in 2000), Universität Tübingen.

Schröder, Ingo, Wolfgang Menzel, Kilian Foth, and Michael Schulz. 2000. Modeling dependency grammar with restricted constraints. *Traitement Automatique des Langues (T.A.L.)* 41:97–126.

Sgall, Petr, Eva Hajičová, and Jarmila Panevová. 1986. *The meaning of the sentence in its semantic and pragmatic aspects*. Dordrecht: Reidel.

Sgall, Petr, Ladislav Nebeský, Alla Goralčíková, and Eva Hajičová. 1969. *A functional approach to syntax in generative description of language*. New York: Elsevier Science B.V.

Stabler, Edward. 1996. Derivational minimalism. In *Logical Aspects of Computational Linguistics (LACL'96)*, ed. by Christian Retoré, 68–95. Berlin: Springer-Verlag.

Stabler, Edward. 2011. Computational perspectives on minimalism. In Boeckx (2011), 617–642.

Stanojević, Miloš, and Edward Stabler. 2018. A sound and complete left-corner parsing for Minimalist Grammars. In *Proceedings of the Eight Workshop on Cognitive Aspects of Computational Language Learning and Processing*, ed. by Marco Idiart, Alessandro Lenci, Thierry Poibeau, and Aline Villavicencio, 65–74. Melbourne: Association for Computational Linguistics. URL https://aclanthology.org/W18-2809/.

Steedman, Mark. 1987. Combinatory grammars and parasitic gaps. *Natural Language and Linguistic Theory* 5:403–439.

Steedman, Mark. 2000. *The syntactic process*. Cambridge, MA: MIT Press.

Steedman, Mark. 2024. *Combinatory categorial grammar: An introduction*. Philadelphia / Edinburgh: The SOMESUCH Press. Draft 3.1 of August 14, 2024, https://homepages.inf.ed.ac.uk/steedman/papers/ccg/esslli24.pdf.

Sugayama, Kensei, and Richard Hudson, ed. by. 2006. *Word grammar: New perspectives on a theory of language structure*. London: Continuum.

Tesnière, Lucien. 1959. *Éléments de syntaxe structurale*. Paris: Klincksieck.

Tesnière, Lucien. 2015. *Elements of structural syntax*. Amsterdam: John Benjamins.

Torr, John, Miloš Stanojević, Mark Steedman, and Shay B. Cohen. 2019. Wide-coverage neural A* parsing for Minimalist Grammars. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, ed. by Anna Korhonen, David Traum, and Lluís Màrquez, 2486–2505. Florence, Italy: Association for Computational Linguistics. URL https://aclanthology.org/P19-1238/.

Van Eynde, Frank. 2024. Nominal structures. In Müller et al. (2024), 295–334. URL https://langsci-press.org/catalog/book/478.

Vijay-Shanker, K., David J. Weir, and Aravind K. Joshi. 1987. Characterizing structural descriptions produced by various grammatical formalisms. In *Proceedings of the 25th Annual Meeting of the Association for Computational Linguistics*, 104–111. Stanford, CA.

Wood, Mary McGee. 1993. *Categorial grammars*. London: Routledge.

Yadav, Himanshu, Samar Husain, and Richard Futrell. 2022. Assessing corpus evidence for formal and psycholinguistic constraints on nonprojectivity. *Computational Linguistics* 48:375–401. URL https://aclanthology.org/2022.cl-2.5/.